MINISTRY OF EDUCATION AND TRAINING
UNIVERSITY OF ECONOMICS HO CHI MINH CITY

UEH
UNIVERSITY

INFORMATION SYSTEMS PROJECT REPORT

SkyRescue – AI-Enabled Aerial Accident Detection and Rescue Coordination System

FULL NAME	STUDENT ID
Trần Minh Hoàng	31251026701
Lê Minh Triết	31251023501
Trần Ngọc Thảo Anh	31251026259
Lê Thái Bảo	31251026566
Đặng Nguyễn Phương Nghi	31251020720

Table of Contents

CHAPTER 1. PROJECT INTRODUCTION 4

1.1. Project Overview 4

1.1.1. Post-Disaster Rescue Context and Project Motivation 4

1.1.2. System Objectives 4

1.1.3. Research Scope and Target Users 5

1.2. Business Problem Description 6

1.2.1. Information Intake and Rescue Dispatch Workflow from Aerial Imagery 6

1.2.2. Limitations of Traditional Manual Inspection and Assessment Methods 7

1.2.3. Proposed Solution: Integrated Computer Vision (AI) and Multi-Criteria Decision Support System (DSS + MIS) 8

1.3. System Requirements Analysis 9

1.3.1. Functional Requirements 9

1.3.2. Non-Functional Requirements 10

1.3.3. Core Business Rules 10

1.4. Technologies and Supporting Tools 11

1.4.1. Computer Vision Technologies 11

1.4.2. Multi-Criteria Decision-Making Methods 12

1.4.3. System Development Platform 13

1.5. Report Structure 14

CHAPTER 2. SYSTEM ANALYSIS 15

2.1. Business Analysis 15

2.1.1. Stakeholders 15

2.1.2. Roles and Needs of Each Stakeholder in the Emergency Response Chain 15

2.1.3. Overall Business Workflow 16

2.2. Data Flow Analysis 18

2.2.1. System Context Diagram 18

2.2.2. Data Flow Diagram Level 1 19

2.2.3. Data Flow Diagram Level 2 20

2.3. Use Case Analysis 25

2.3.1. System Use Case Diagram and Role-Based Decomposition 25

2.3.2. Main Use Case Specifications 28

2.4. Analysis of Core Business Functions 29

2.4.1. Multi-Layer Image Intake and Validation 29

2.4.2. Incident Scene Classification 30

2.4.3. Victim Detection and Trauma Pose Recognition 31

2.4.4. Domain-Specific Disaster Severity Assessment 32

2.4.5. Priority Queue Ranking via AHP / TOPSIS 32

2.4.6. Optimal Rescue Team Recommendation and Matching 33

2.5. Decision Support System (DSS) Requirements Analysis 34

2.5.1. Defining Decision Criteria (Severity, Geodesic Distance, Rescue Team Availability) 34

2.5.2. Algorithmic Transparency and Explainability Requirements 35

CHAPTER 3. SYSTEM DESIGN 35

3.1. System Architecture Design 35

3.1.1. Overall System Architecture 35

3.1.2. Main Components and Functions 36

3.1.3. Communication and Data Exchange Mechanisms 37

3.2. Business Process Design (Swimlane Activity Diagram) 38

3.2.1. Image Data Intake, Validation, and Error Code Handling Workflow (ERR_*) 40

3.2.2. Multi-Stage AI Inference Workflow 40

3.2.3. AHP / TOPSIS Queue Priority Scoring Workflow 41

3.2.4. Rescue Dispatch, Progress Monitoring, and Incident Closure Workflow 42

3.3. Analytical and Decision Support Model Design 43

3.3.1. Incident Scene Classification Model 43

3.3.2. Human Detection and Victim Pose Classification Model 44

3.3.3. Domain-Specific Severity Assessment Algorithms 44

3.3.4. Criterion Weight Determination via Analytic Hierarchy Process 45

3.3.5. TOPSIS Priority Queue Ranking Algorithm 46

3.3.6. Rescue Team Compatibility Scoring Algorithm 47

3.4. Database Design 49

3.4.1. Target Relational Schema 49

3.4.2. Core Entities and Relationships 50

3.4.3. As-Built Storage Architecture (Relational Architecture on SQLite WAL with Infrastructure Extensions) 52

3.4.4. Mapping Between JSON State Keys and the Target Relational Schema 53

3.5. Data Flow and Information Security Design 55

3.5.1. Session Management and Role-Based Access Control 55

3.5.2. Append-Only Audit Trail for Post-Incident Accountability 56

3.5.3. Automated Incident Digital Dossier Generation 57

CHAPTER 4. USER INTERFACE DESIGN AND SYSTEM EVALUATION 58

4.1. System User Interface Overview (Real-Time Synchronized Dual-Portal System) 58

4.2. Field Reporter Portal Interface 59

4.2.1. Image Upload, Automated EXIF GPS Extraction, and Browser AML Geolocation Interface 59

4.2.2. Supplemental Field Notes and Layer 1 Validation Feedback Interface 60

4.2.3. Submission History and Personal Rescue Progress Tracking Interface (My Reports) 61

4.3. Command Dispatch Station Interface 62

4.3.1. GIS Tactical Map Dashboard for Incident and Rescue Resource Monitoring 62

4.3.2. Visual Priority Queue Interface 63

4.3.3. Detailed Incident Verification and AI Evidence Viewer Interface 65

4.3.4. Rescue Team Recommendation and Dispatch Order Confirmation Interface 66

4.3.5. System Configuration, Rescue Team Roster Management, and Digital Dossier Export Interface 68

4.4. Deployment Results and Experimental Evaluation 72

4.4.1. Experimental Results on the Multi-Regional Drone Image Dataset 72

4.4.2. Performance and Response Time Evaluation 77

4.4.3. Consistency and Sensitivity Evaluation of the DSS Ranking Algorithm 81

4.4.4. Evaluation of Strengths and Limitations During Testing 82

CHAPTER 5. CONCLUSION AND FUTURE WORK..... 834

5.1. Achievements 834

5.2. System Limitations..... 845

5.3. Future Development Directions..... 855

CHAPTER 1. PROJECT INTRODUCTION

1.1. Project Overview

1.1.1. Post-Disaster Rescue Context and Project Motivation

In recent years, under the impact of global climate change, natural disasters such as storms, floods, landslides, earthquakes, and fires have occurred with increasing frequency and complex developments. In emergency response operations, Unmanned Aerial Vehicles (UAVs / Drones) have become indispensable tools thanks to their ability to rapidly access isolated and hazardous areas to capture aerial scene imagery with wide coverage.

However, the growing number of aerial reconnaissance vehicles has also led to an explosion of visual data transmitted back to the command center. Currently, scene screening and assessment still rely primarily on manual human effort: dispatchers must manually inspect each image to identify incident types and search for traces of victims. This manual approach reveals severe limitations, causing information bottlenecks and risking the loss of the critical "golden hour" for rescue.

Motivated by this reality, the project "SkyRescue - An AI-Integrated Aerial Imagery System for Disaster Monitoring, Detection, and Rescue Dispatch" is developed to automate image preprocessing using Computer Vision and apply a Decision Support System (DSS) to quantify the dispatch workflow, thereby enhancing rescue efficiency in emergency situations.

1.1.2. System Objectives

The core objective of the project is to build a software platform that comprehensively supports the rescue response chain, shifting from intuition-based processing to a data-driven command model:

- Shorten incident classification and detection time: Apply a two-stage deep learning pipeline to automate disaster category recognition while simultaneously detecting and counting victims from UAV imagery with rapid processing speed (measured in seconds).

- Support scientifically grounded multi-criteria dispatch decisions: Build a hierarchical Decision Support System (DSS):

+ Urgency assessment: Combine the Incident Severity Score (evaluated across domain-specific disaster models) alongside the Reinforced Code Red Life-Critical Fast Track (prioritizing human life above geographic distance by pinning critical emergency incidents to the top of the queue through 3 automated defense layers to eliminate 100% false alarms: High-Confidence Gate, Hazard Context Gate, and Golden Hour Escalation), geodesic access distance, and local resource scarcity through Multi-Criteria Decision-Making algorithms (AHP / TOPSIS, fixed weights 0.648 / 0.230 / 0.122, CR = 0.003) to establish queue priority.

+ Dispatch resource optimization: Recommend the optimal rescue unit through Hard Constraints filtering on terrain and vehicle conditions, Soft Constraints scoring for specialized equipment, and an audited Deadlock Override mechanism when strict physical constraints cannot be simultaneously satisfied.

- Ensure supreme human control (Human-on-the-Loop) with Account-Aware Hybrid Trust Verification: The system distinguishes two Reporter account types (Verified Accounts vs. Unverified Citizen Accounts) and routes reports through two verification flows. High-confidence emergency reports from Verified Accounts auto-verify directly to the Matching Team state to eliminate manual screening bottlenecks, whereas Unverified Citizen reports require Coordinator verification in the Admin Review Inbox. Crucially, the system never autonomously dispatches rescue units; every dispatch requires explicit human confirmation (via the 1-Click Unified Triage Workspace or C5 screen). Furthermore, if a Coordinator wants to edit the incident type of an Auto-Verified incident, clicking Edit Type routes the incident back to the Verify screen so it must be re-verified before matching or dispatching.

- Optimize operational management and monitoring: Provide a comprehensive overview of incident resolution progress, rescue team performance statistics, resource overload warnings, and automated Digital Dossiers for each rescue operation to support post-incident auditing.

1.1.3. Research Scope and Target Users

- Research Scope:

+ Dual-portal actor architecture with two Reporter account tiers: The system is separated into two independent interaction workspaces: the Reporter Portal (supporting two account types: Verified Reporter / Specialist CTV / Registered UAV accounts and Unverified Citizen accounts, with streamlined single-field-note intake) and the Dispatch Station (for monitoring the GIS tactical map, reviewing Unverified cases, managing Auto-Verified Matching Team cases, and confirming single-primary-team dispatch orders).

- Experimental Data Scope: Due to limitations in accessing real-time aerial footage during active disasters, the research is conducted at two levels:

+ AI Component Evaluation: Inherit and independently test across four international benchmark open aerial datasets: AIDER (general disaster classification), C2A (human detection and density estimation), FLAME2 (fire severity assessment), and UAVs-TEBDE (collapsed building recognition).

- Integrated System Workflow Experimentation: Utilize the national Vietnam dataset spanning 7 province views (Ho Chi Minh City, Ha Noi, Da Nang, Can Tho, Khanh Hoa, Lam Dong, and Quang Nam) within the nationwide operational bounding box, seeded with 20 real-EXIF aerial incident cases (EXT01-EXT20: 15 open active cases comprising 11 verified and 4 in the Admin Review Inbox, plus 5 in Resolved Cases) and 18 multi-specialty rescue teams across 21 municipal sectors, to demonstrate feasibility, low latency, and end-to-end continuity from report intake to status-only incident closure.

- Target System Users:

+ Field Reporter (2 Account Types on /reporter/): Comprises (1) Verified / Authenticated Accounts (Verified Reporters, Specialist Collaborators / CTV, and Registered UAV Operators) whose high-confidence AI emergency reports are eligible for Hybrid Trust Auto-Verification directly to Matching Team, and (2) Other / Unverified Citizen Accounts whose reports enter the Unverified review queue for Coordinator verification. To minimize cognitive load during emergencies, Reporters only upload a scene photograph with GPS coordinates and enter a single optional free-text field note (without manual scene classification or manual victim counting).

+ Rescue Dispatcher (Dispatcher / SAR Coordinator on /dispatch/): Personnel monitoring the tactical dashboard, verifying Unverified Citizen reports on the Admin Review screen (C9R), reviewing Auto-Verified cases in Matching Team (with the ability to click Edit Type to route an Auto-Verified case back to the Verify screen for re-verification if its category needs adjustment), and confirming single-primary-team dispatch orders via the Unified Triage Workspace.

1.2. Business Problem Description

1.2.1. Information Intake and Rescue Dispatch Workflow from Aerial Imagery

In modern disaster response campaigns, utilizing Unmanned Aerial Vehicles (UAVs / Drones) for scene reconnaissance has become a vital vanguard procedure. From an operational perspective, the response chain from aerial image data to field rescue execution typically undergoes four main stages:

- Stage 1 - Streamlined Image Collection and Layer 1 Gatekeeping: Verified Reporters (UAV Operators / Specialists) or Unverified Citizens capture emergency scenes and submit the raw image along with spatio-temporal metadata (EXIF or browser GPS coordinates, capture timestamp) and a single optional free-text field note (no manual scene tag or victim count is required). The system automatically triggers Layer 1 gatekeeping filters to validate file format, size (≤ 35 MB), image blur/luminance, perceptual dHash near-duplicate status, and EXIF-vs-live GPS consistency.

- Stage 2 - AI Vision Screening and Hybrid Trust Verification Routing (2 Flows): Validated images are processed by the Two-Stage AI Pipeline to classify the disaster category (Fire, Flood, Collapsed Building, Traffic Accident, Normal), localize victims and trauma postures (Upright vs. Non-Upright), and compute the domain-specific Severity Score. Next, the Hybrid Trust Router executes two flows based on account type: (Flow 1) If submitted by a Verified Account with an emergency classification, AI confidence ≥ 85%, and zero risk/integrity flags, the incident is Auto-Verified directly to Matching Team status; (Flow 2) All reports submitted by Unverified Citizen accounts (as well as reports classified as Normal, AI confidence < 85%, or containing any risk/integrity flags) strictly remain in Unverified status, mandating explicit manual Coordinator verification on the Admin Review screen (C9R) before advancing to team matching. Notably, if an incident is Auto-Verified to Matching Team and the Coordinator wishes to change its incident type, clicking Edit Type routes the case back to the Verify screen (C9R) where the Coordinator edits the type and explicitly re-verifies it.

- Stage 3 - Reinforced Code Red Fast Track, AHP / TOPSIS Priority, and Team Matching: Incidents meeting life-threatening indicators are evaluated through the Reinforced Code Red Fast Track (life.critical = true) controlled by 3 defense layers: (Layer 1) High-Confidence Gate requiring ≥ 1 non-upright casualty with Conf ≥ 0.70 or Mass Casualty Incident MCI ≥ 3 victims with Conf ≥ 0.75; (Layer 2) Hazard Context Gate confirming real disaster scenes (Fire, Collapse, Flood, Traffic) while locking out Normal scenes; and (Layer 3) Golden Hour Escalation automatically promoting unassigned victim incidents waiting ≥ 45 minutes. Code Red incidents are pinned to the top of the queue under the fast-track lane (#CR-n). In parallel, standard incidents are ranked by the MCDM engine combining Incident Severity (C1), geodesic distance (C2), and available team count (C3) via AHP / TOPSIS (weights 0.648, 0.230, 0.122; CR = 0.003), establishing the strict queue order: Life Risk (Code Red) ⟶AHP Priority ⟶Waiting Time.

- Stage 4 - Field Deployment and Status Updating: A dispatch order is issued from the command center, transitioning the assigned unit's status to On Mission. Upon receiving field completion reports, the Coordinator triggers the Finalize & Resolve function to transition the incident status to Resolved, mark the dispatch order as Completed, release the rescue unit back to Available status, and automatically generate a closed-loop Digital Dossier.

1.2.2. Limitations of Traditional Manual Inspection and Assessment Methods

Although UAV imagery provides a rich visual data source, traditional manual monitoring and command methods expose critical bottlenecks when confronting wide-scale disasters:

- Bottlenecks in Image Inspection: Following a disaster, hundreds to thousands of high-resolution images arrive continuously. Requiring dispatchers to manually inspect each image with the naked eye to locate victims easily leads to cognitive overload and prolonged delays in victim detection.

- Missing the Rescue "Golden Hour": In search and rescue, the first few dozen minutes are decisive for victim survival. Slow manual analysis stalls priority list establishment, directly consuming precious time before response units receive orders to depart.

- Subjective Assessment Lacking Standardized Metrics: Different dispatchers may form varying intuitive judgments when viewing scene photographs. The absence of a standardized quantitative scoring system causes truly critical incidents to occasionally be queued behind less urgent incidents that were simply reported earlier.

- Fragmented and Disconnected Resource Dispatch: Commanders struggle to simultaneously synthesize multiple dynamic variables: team availability, specialized unit capabilities, and travel distances. This easily results in overlapping assignments or capability mismatches—dispatching a distant unit while overlooking a nearby one, or overloading a single team with excessive concurrent missions.

1.2.3. Proposed Solution: Integrated Computer Vision (AI) and Multi-Criteria Decision Support System (DSS + MIS)

To thoroughly overcome these limitations, this project proposes SkyRescue—an integrated platform combining Artificial Intelligence (AI), a Decision Support System (DSS), and a Management Information System (MIS) to digitize and modernize the entire workflow:

- Automating Pre-Verification via Computer Vision and Hybrid Trust Routing: The system deploys a low-latency Two-stage AI Pipeline where Reporter intake requires only a photo and a single optional free-text note. Stage 1 classifies the disaster context, while Stage 2 detects, counts, and classifies victim poses using Bounding Boxes. Trusted reports from Verified Accounts (confidence $\geq 85\%$, no flags) are Auto-Verified directly to Matching Team, while Unverified Citizen reports are queued for Admin Review— eliminating bottlenecks while preserving strict quality control.

- Objectifying Triage and Dispatch via AHP / TOPSIS and Reinforced Code Red Fast Track: Combining the Reinforced Code Red Life-Critical Fast Track (governed by 3 automated defense layers: High-Confidence Gate, Hazard Context Gate, and Golden Hour Escalation at ≥ 45 min) with the AHP / TOPSIS algorithm (weights 0.648, 0.230, 0.122; CR = 0.003), the system integrates objective incident severity, spatial distance, and rescue force availability into a transparent queue ordered by Life Risk (Code Red) $\longrightarrow$AHP Priority $\longrightarrow$Waiting Time. In parallel, the Dispatch Engine evaluates Hard and Soft Constraints to recommend the single optimal primary rescue unit inside a Unified Triage Workspace.

- Comprehensive Tactical Lifecycle Control via a Management Information System (MIS): The central dashboard visualizes all incident hotspots on a real-time tactical map. Adhering to the Human-in-the-loop safety principle, AI and DSS serve strictly as analytical and recommendation engines, while final verification and dispatch authorization remain exclusively with the human dispatcher. The entire rescue lifecycle is continuously monitored, recorded in an immutable Audit Trail, and compiled into a closed-loop Digital Dossier for operational sign-off, executive reporting, and disaster data digitization.

1.3. System Requirements Analysis

Designed to resolve information overload and support timely decision-making in rescue operations, the system is built upon a cohesive combination of functional requirements, non-functional requirements, and core business rules.

1.3.1. Functional Requirements

The system is organized around two primary actors (Reporter & SAR Coordinator) with the following core functional groups:

- FR-01 - Two-Tier Account Intake, Single Field Note, and Layer 1 Validation: Supports two Reporter account types (Verified Reporter / Specialist CTV / Registered UAV accounts vs. Unverified Citizen accounts) uploading UAV or ground images (≤ 35 MB) via the Reporter Portal. Collects only one optional free-text field note (fieldNote / accessNote), removing manual scene classification and manual victim counting. Automatically extracts EXIF GPS metadata (with browser Geolocation API fallback) and checks perceptual dHash near-duplicates and EXIF-vs-live GPS distance.
- FR-02 - Incident Scene Classification: From the acquired image and coordinate data, the system analyzes and classifies the scene into five categories (Fire, Collapsed Building, Flood, Traffic Accident, Normal). Each classification result is accompanied by a Confidence Score to guide routing for subsequent in-depth analysis.
- FR-03 - Victim Detection and Trauma Pose Recognition: Based on the identified scene context, the AI scans the image to accurately localize each victim using Bounding Boxes. Simultaneously, it classifies victim postures into two groups-Upright and Non-Upright-to provide a preliminary assessment of life-threatening criticality.
- FR-04 - Hybrid Trust Verification, Reinforced Code Red Fast Track, and AHP / TOPSIS Queue Ranking: Routes incoming reports via two account-aware verification flows: Verified Accounts with emergency AI confidence ≥ 85% and zero flags Auto-Verify directly to Matching Team, whereas all Unverified Citizen reports strictly enter the Unverified Review Inbox requiring manual Coordinator verification without exception. If a Coordinator clicks Edit Type on an Auto-Verified incident, the system routes it back to the Verify screen for mandatory re-verification. For queue ranking, life-critical cases are pinned to the Reinforced Code Red Fast-Track lane (#CR-n) under 3 automated defense gates: (1) High-Confidence Gate (≥ 1 Non-Upright victim with Conf ≥ 0.70, or MCI ≥ 3 victims with Conf ≥ 0.75); (2) Hazard Context Gate (confirmed actual disaster, locking out Normal scenes); and (3) Golden Hour Escalation (auto-promoting any incident with victims waiting ≥ 45 min without team assignment, logging codeRedReason: 'Golden Hour Escalation: Nạn nhân chờ quá 45 phút'). Incident Severity S (C1) is synthesized with geodesic distance (C2) and available team counts (C3) via AHP / TOPSIS (weights 0.648, 0.230, 0.122; CR = 0.003), preserving the exact queue hierarchy: Life Risk (Code Red) ⟶AHP Priority ⟶Waiting Time.
- FR-05 - Single Primary Rescue Team Recommendation and Dispatch: Evaluates available forces against strict Hard Constraints (operational status, disaster specialty matching, vehicle/terrain accessibility) and ranks eligible candidates using Soft Constraints (equipment compatibility, team headcount capacity, and travel distance) to identify the single most

suitable primary rescue team, supported by an audited Deadlock Override when severe physical constraints cannot be simultaneously satisfied.
- FR-06 - Digital Dossier and Operational Audit Management: Captures the entire rescue lifecycle across 15 relational domain tables (130 logical attributes plus 54 implementation/projection columns in server/schema.sql) indexed by immutable public codes (INC-XXXX for reports, DSP-XXXX for dispatches). Executing Finalize & Resolve performs a clean, status-only operational closure (transitioning the incident to Resolved, the dispatch order to Completed, and the rescue team back to Available without requiring casualty-tally or debrief forms) while recording full operator accountability in the append-only audit trail and FEMA ICS-209 Digital Dossier.

1.3.2. Non-Functional Requirements

- NFR-01 - Performance and Processing Latency: The system must ensure processing latency ≤ 200 ms during Layer 1 gatekeeping validation. End-to-end AI Pipeline Inference Time-from scene classification to victim and pose detection-must not exceed 1 s per single image frame. Furthermore, incident state synchronization and notifications between the server and the dispatch station interface must achieve near real-time latency ≤ 2 s via Server-Sent Events (SSE).

- NFR-02 - Decision Transparency and Explainability: The system must never operate as a black box. All priority rankings and dispatch recommendations must be accompanied by an algorithmic trace (DSS Trace) that explicitly displays criterion weights (AHP) and relative closeness matrices (TOPSIS) so dispatchers clearly understand the rationale behind every recommendation.

- NFR-03 - Availability and Concurrency: The system must maintain high concurrent read/write capability without resource locking bottlenecks, ensuring stability and uninterrupted operation even when receiving surges of simultaneous reports from multiple disaster zones.

- NFR-04 - Information Security and Access Control: The security architecture strictly enforces Role-Based Access Control (RBAC) to separate privileges between Reporters and Coordinators. User passwords are protected using the modern scrypt hashing algorithm, and active sessions are managed via authentication cookies configured with HttpOnly and SameSite protection flags to prevent session hijacking attacks (XSS, CSRF).

1.3.3. Core Business Rules

The system operates under three vital business rules to guarantee clean input data and ensure supreme authority always remains with human operators:

- BR-01 - Multi-Layer Gatekeeping and Streamlined Intake (Layer 1 Validation): Every image submitted to the system must pass Layer 1 filtering before entering the AI Pipeline. Input images must conform to JPEG / PNG formats (≤ 35 MB), pass blur and luminance checks, contain valid GPS coordinates within the active Geofence Bounding Box (flagging

conflicts > 1 km between EXIF GPS and live browser GPS), and have a capture timestamp within 30 minutes of server time. Both cryptographic SHA-256 hashing and 64-bit perceptual dHash checks are enforced to block exact and near-duplicate submissions. Reporters provide only a single optional free-text field note, eliminating manual scene tags and manual victim counts.

- BR-02 - Safe-Case Bypass Mechanism (Normal Bypass Gate): If the AI classification model identifies an image as "Normal" with confidence exceeding the safety threshold (Conf ≥ Threshold), the system automatically routes the record to passive storage for rapid closure rather than pushing it into the emergency priority queue, allowing dispatchers to focus 100% of their capacity on genuinely critical incidents.

- BR-03 - Human-on-the-Loop Control, Hybrid Trust Two-Flow Verification, and Edit-Type Re-Verification Guardrail: AI models and DSS algorithms act strictly as an intelligent decision support assistant and never autonomously issue rescue dispatch orders. Under the Hybrid Trust model, emergency reports from Verified Accounts (Verified Reporter, Specialist CTV, Registered UAV) with AI confidence ≥ 85% and zero risk flags are Auto-Verified directly to Matching Team status, whereas reports from Unverified Citizen accounts (or low-confidence / flagged reports) remain Unverified until manually approved by a Coordinator on the Admin Review screen (C9R). Even when an incident is Auto-Verified to Matching Team, final team dispatch always requires human Coordinator confirmation (via 1-Click Unified Workspace or C5). Crucially, if a Coordinator wants to edit the incident type of an Auto-Verified incident, they must click Edit Type, which routes the incident back to the Verify screen (C9R) so the Coordinator can modify the type and explicitly re-verify the incident before team matching and dispatch.

1.4. Technologies and Supporting Tools

1.4.1. Computer Vision Technologies

The SkyRescue system addresses the emergency rescue triage problem by analyzing aerial images captured by UAVs/drones through a multi-stage Computer Vision pipeline developed using the PyTorch library:

- Incident Scene Classification Model (Model 1):

Uses the EfficientNet-B0 network architecture (checkpoint aider_classifier.pth), trained and fine-tuned on the AIDER (Aerial Image Dataset for Emergency Response) aerial disaster dataset. EfficientNet-B0 was selected because of its Compound Scaling mechanism, which simultaneously balances network depth, width, and image resolution. This architecture delivers high classification accuracy while maintaining a relatively small number of parameters (~5.3 million), enabling inference times of under 500 ms on a standard CPU. The model classifies images into five categories:

+ Collapsed building
+ Fire
+ Flood
+ Traffic accident

+ Normal

- Human and Victim Pose Detection Model (Model 2):

Uses the YOLOv8m network architecture (checkpoint v8m_up2_best.pt), trained on the C2A (Casualty and Action in Aerial Imagery) dataset. The model accurately localizes each victim in the scene by predicting bounding box coordinates and classifies their poses into two categories:

+ Upright (Standing / Moving): Indicates that a person is conscious and potentially capable of evacuating independently.
+ Non-Upright (Lying / Crawling / Collapsed): An important indicator of victims who may be severely injured, suffering from smoke inhalation, unconscious, or trapped beneath debris. This category is standardized and stored in the database under the label Other.

- Domain-Specific Severity Assessment Wrappers:

Rather than applying a single subjective formula to all incidents, the system implements four specialized modules to calculate severity scores $S \in [0.0, 1.0]$, corresponding to different types of disasters:

+ Collapsed Building Branch: Uses the MobileNetV3-Small network (collapsed_building_best_v2.pt), fine-tuned on the UAVs-TEBDE dataset, to classify structural damage into three severity levels: Intact (0.0), Damaged (0.5), and Collapsed (1.0).
+ Fire Branch: Uses the MobileNetV3-Small network (best_fire_mobilenetv3_phase2.pth) to classify three danger levels based on the presence of flames and smoke: No Flame / No Smoke (0.0), Flame / No Smoke (0.5), and Flame / Smoke (1.0).
+ Flood Branch: Uses a geometric measurement algorithm based on the reduction in the bounding-box aspect ratio h/w of an upright person compared with a non-flooded baseline ($BASE$) to estimate water depth.
+ Traffic Accident Branch: Applies a Sliding Window algorithm (4m x 4m) to calculate the highest victim density according to the Fruin Level of Service (Fruin LOS: A to F), combined with the number of deformed vehicles (best_vehicle.pt).

1.4.2. Multi-Criteria Decision-Making Methods

To ensure that the rescue queue is prioritized objectively, the system combines two classical multi-criteria decision-making techniques:

- Analytic Hierarchy Process (AHP):

Developed by Thomas L. Saaty, AHP is used to determine the weights of three core criteria: Incident Severity (C_1), Distance to the Nearest Rescue Team (C_2), and Number of Rescue Teams Currently Available (C_3), with derived weights w_sev = 0.648, w_dist = 0.230, and w_free = 0.122. This is achieved through a pairwise comparison matrix and a mathematical consistency check (Consistency Ratio CR = 0.003 < 0.1). In alignment with the underserved area doctrine, incidents with greater isolation (larger distance) and fewer available teams receive higher priority alongside high severity.

- Technique for Order Preference by Similarity to Ideal Solution (TOPSIS):
Based on the weight vector derived from AHP, TOPSIS normalizes the data matrix of pending incidents and identifies the positive ideal solution (A+) and the negative ideal solution (A-). The priority ranking of each incident is then determined based on its relative closeness coefficient (Ci).

1.4.3. System Development Platform

SkyRescue is developed as a web-based application consisting of two main interfaces: the Reporter Portal and the Dispatch Station. Both interfaces share a common backend. The primary technologies used include FastAPI, SQLite, Server-Sent Events (SSE), HTML5, CSS and JavaScript.

The backend is developed using Python 3.11 and FastAPI and is hosted on a Uvicorn server. FastAPI provides RESTful APIs for authentication, session management, report submission, image uploads and status updates. Data is primarily exchanged in JSON format, while images are uploaded to the server via HTTP.

The database is implemented on SQLite with Write-Ahead Logging (WAL) enabled, adopting a fully relational architecture comprising 15 core domain tables (governing reports, predictions, severity assessments, rescue teams, dispatches, victims, and audit logs) supplemented by operational infrastructure tables (such as sessions for authenticated state management). Primary keys utilize 64-bit integer identifiers paired with human-readable operational codes (e.g., report_id with INC-XXXX, dispatch_id with DSP-XXXX). Image assets are managed in the server's local media directory while their paths, checksums, and metadata are indexed relationally.

Data synchronization is implemented using SSE combined with polling. When data changes, the backend sends a notification containing the latest data version through SSE. Upon receiving this notification, the frontend requests the updated data through the API. Additionally, the system checks the data version every two seconds via the /api/version endpoint.

The frontend is developed using HTML5, CSS and vanilla JavaScript. HTML5 defines the structure of web pages, CSS controls the visual presentation and JavaScript handles user interactions, API requests and dynamic updates to displayed data. The Dispatch Station integrates the Leaflet library to provide an interactive map for rescue coordination and dispatch operations.

AI processing is supported by PyTorch, Torchvision, Ultralytics, OpenCV, NumPy, SciPy and Pillow. These libraries are used for image processing, incident classification, person detection and severity assessment. AI tasks are submitted to an asynchronous processing queue managed by the backend.

The system is deployed using Docker and Docker Compose. The backend, frontend and AI components run within a single application container. The SQLite database and uploaded images are stored in Docker volumes to ensure data persistence across container restarts.

1.5. Report Structure

This report is organized into five main chapters, comprehensively reflecting the research, analysis, design, and experimental evaluation of the SkyRescue system:

- Chapter 1 - Project Introduction: Presents the urgency and context of technology application in disaster rescue; defines objectives and research scope around the dual-portal architecture; describes the real-world business problem; analyzes functional requirements, non-functional requirements, and core business rules (Layer 1 Gatekeeping, Human-in-the-loop); and introduces the foundational technologies employed (Computer Vision, AHP / TOPSIS multi-criteria DSS, FastAPI, SQLite WAL, SSE).

- Chapter 2 - System Analysis: Conducts an in-depth analysis of stakeholders and the end-to-end business workflow from image intake to incident closure; models information flows through the Context Diagram and Data Flow Diagrams (DFD Level 1 and Level 2); constructs Use Case diagrams and specifications; and details key business functions (5-class disaster classification, victim pose detection, priority scoring, and rescue team matching).

- Chapter 3 - System Design: Specifies the layered system architecture and real-time synchronization mechanisms; designs operational workflows via a Swimlane Activity Diagram; formulates analytical algorithms (EfficientNet-B0 on AIDER, YOLOv8m on C2A, AHP weight matrix, TOPSIS ranking, and Team Match Score); and details the Relational database design on SQLite WAL comprising 15 core domain tables plus infrastructure tables alongside information security mechanisms (RBAC, append-only Audit Trail, Digital Dossier).

- Chapter 4 - User Interface Design and System Evaluation: Presents the visual interface design for both subsystems—the Reporter Portal and the Dispatch Station (with its GIS Tactical Map); reports experimental test results on multi-regional UAV imagery; evaluates technical performance metrics (Layer 1 latency $\leq$ 200 ms, AI inference time $\leq$ 1 s, SSE synchronization $\leq$ 2 s); and verifies the consistency and sensitivity of the DSS ranking algorithm.

- Chapter 5 - Conclusion and Future Work: Summarizes the research achievements and technical deliverables in solving automated, transparent rescue triage; candidly discusses existing system limitations (simulated data scale, camera angle dependency, and the 1:1 Report-Incident modeling convention within the 15-table scope); and proposes practical directions for future expansion (separating an independent Incident entity, integrating live UAV video streams, and connecting real-time IoT vehicle telemetry).

CHAPTER 2. SYSTEM ANALYSIS

2.1. Business Analysis

2.1.1. Stakeholders

In the emergency rescue response chain of the SkyRescue system, seamless coordination among participants is vital. The system identifies four key stakeholders participating in the business workflow:

- Field Reporters (2 Account Types: Verified Accounts vs. Unverified Citizens): Personnel and citizens submitting photographic evidence, GPS coordinates, and a single optional free-text field note through the Reporter Portal (/reporter/). The system explicitly differentiates two account tiers: (1) Verified / Authenticated Accounts (Verified Reporters, Specialist CTVs, Registered UAV Operators) whose high-confidence emergency reports are eligible for automated trust verification directly to Matching Team; and (2) Unverified Citizen / Other Accounts whose reports require Coordinator verification in the Admin Review Inbox.

- Rescue Dispatchers (SAR Coordinators): Operational duty personnel at the central command station (/dispatch/). They verify Unverified Citizen reports, monitor Auto-Verified incidents in Matching Team (with the authority to click Edit Type on any Auto-Verified incident to route it back to the Verify screen for re-verification), oversee the Code Red and AHP / TOPSIS priority queue, and confirm single-primary-team dispatch orders via the Unified Triage Workspace.
- Command Board / System Managers: Macro-level operational leadership. Rather than handling individual incidents directly, this group monitors the overall tactical picture on the GIS map, coordinates inter-regional resources, configures alert thresholds, manages dispatch rule sets (Hard and Soft Constraints along with Context-Aware Scoring Weights in the Rule Engine), and conducts post-incident auditing and statistical analysis via the MIS reporting module on the Dispatch Station.
- Field Rescue Teams: Rapid response units (firefighting, emergency medical services, urban search and rescue, water rescue). Within the system's dual-actor portal architecture, rescue teams are managed as internal operational resource records. They receive dispatch orders from the command center, mobilize directly to the scene to rescue victims, and report mission completion back to the Dispatcher via tactical communication channels so the Dispatcher can execute the Finalize & Resolve closure operation on the system.

2.1.2. Roles and Needs of Each Stakeholder in the Emergency Response Chain

Stakeholder	Primary Role	Key System Needs
Field Reporter (2 Account Types: Verified Reporter / UAV / CTV vs.	- Upload disaster scene imagery with EXIF / live GPS coordinates and capture timestamps. - Provide a single optional	- Fast, zero-friction submission with only 1 optional free-text note. - Verified accounts get instant Auto-Verify to Matching Team (when AI Conf $\geq$ 85%); Unverified Citizen accounts get transparent review tracking.

Unverified Citizen)	free-text field note (no manual scene tag or victim count).	
Rescue Dispatcher (SAR Coordinator)	- Verify Unverified Citizen reports on the Admin Review screen (C9R). - Review Auto-Verified cases in Matching Team (or click Edit Type to route back to C9R and re-verify). - Authorize single-primary-team dispatch orders.	- Hybrid Trust filtering that auto-verifies trusted high-confidence reports while holding citizen reports for review. - Reinforced Code Red (3-layer defense) pinning & AHP/TOPSIS queue. - 1-Click Unified Triage Workspace.
Command Board (System Managers)	- Monitor the macro disaster picture and operational load. - Coordinate strategic resources across districts. - Configure operational rules and conduct post-incident audits.	- GIS tactical map dashboard and real-time KPI metrics (response time, resolution rate). - Early warnings for resource overload or wide-scale disasters. - Automated export of summary reports and FEMA ICS-209 Digital Dossiers.
Field Rescue Team (Operational Resource Record)	- Mobilize to the incident epicenter as the single assigned primary unit. - Communicate field mission completion back to the command center (represented as an operational record rather than a login actor).	- Receive advance aerial visual context, exact GPS coordinates, and AI-estimated victim counts from single-unit dispatch orders. - Rapid status-only Finalize & Resolve closure by the Coordinator (automatically releasing the unit back to Available status without manual debrief forms).

2.1.3. Overall Business Workflow

The business workflow of SkyRescue covers the complete lifecycle of an emergency rescue operation—from initial photo report submission to field mission completion and dossier archiving—across five consecutive stages:

- Stage 1 - Streamlined Intake and Layer 1 Pre-Verification: A field reporter logs in under one of the two account types (Verified Account or Unverified Citizen Account), uploads a scene image to the Reporter Portal, and optionally enters a single free-text field note (manual scene classification and manual victim counting are completely removed). The system extracts EXIF GPS (or browser Geolocation fallback), checks Geofence compliance, timestamp freshness, SHA-256 and perceptual dHash near-duplicates, and EXIF-vs-live GPS distance before assigning a unique incident identifier (INC-XXXX).

- Stage 2 - AI Vision Analysis, Hybrid Trust Routing, and Urgency Quantification: Valid reports enter the Two-stage AI Pipeline. Model 1 classifies the disaster category (Fire, Collapsed Building, Flood, Traffic Accident, Normal), and Model 2 detects victim counts and trauma postures (Upright vs. Non-Upright). The DSS computes domain severity S_raw, evaluates Code Red fast-track status, and executes AHP / TOPSIS ranking (weights 0.648, 0.230, 0.122; CR = 0.003). Simultaneously, the Hybrid Trust Router evaluates the reporter's account tier: (Flow 1) Verified Accounts with an emergency prediction, AI confidence ≥ 85%, and zero risk flags are Auto-Verified directly to Matching Team status; (Flow 2) All Unverified Citizen reports, Normal scenes, confidence < 85%, or flagged reports are placed in Unverified status, requiring mandatory manual Coordinator review before advancing to team matching.

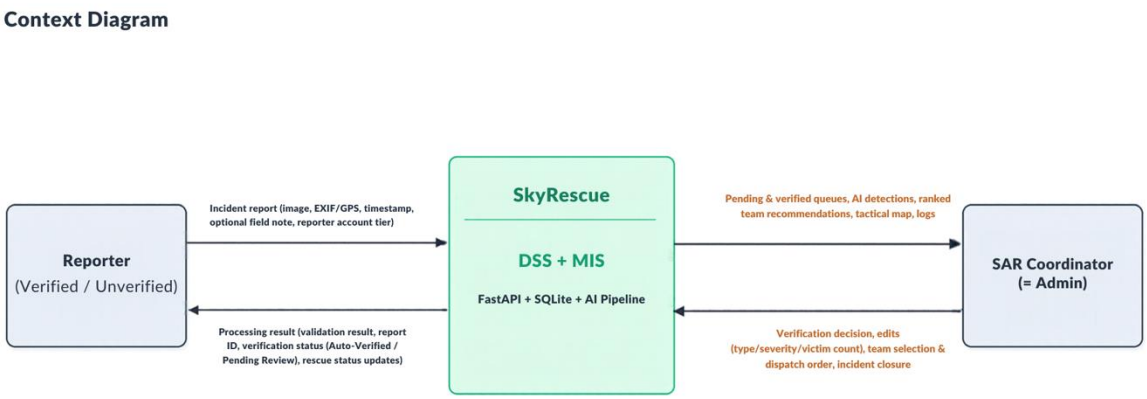
- Stage 3 - Human-on-the-Loop Verification, Edit-Type Re-Verification Guardrail, and Dispatch Authorization: For Flow 2 (Unverified) reports, the Dispatcher opens the Admin Review screen (C9R) to inspect bounding boxes, adjust parameters if needed, and click Verify to transition the incident to Matching Team. For Flow 1 (Auto-Verified) reports, the incident is already in Matching Team with a pre-ranked primary rescue unit ready for 1-click dispatch confirmation in the Unified Triage Workspace (C2) or C5. Crucially, if the Dispatcher wishes to change the incident type of an Auto-Verified incident, clicking Edit Type routes the incident back to the Verify screen (C9R) so the Dispatcher must edit the type and explicitly re-verify it before matching and confirming dispatch. Once confirmed, the dispatch order (DSP-XXXX) transitions the incident to Team Assigned (In Progress) and the selected primary rescue team to Busy (On Mission).

- Stage 4 - Field Operations and Progress Synchronization: The dispatch order is transmitted immediately to field rescue forces, and the assigned rescue team's status transitions to On Mission. Field approach and rescue progress are communicated by the rescue team back to the command center. The command center continuously monitors developments on the digital tactical map (GIS Dashboard) via real-time synchronization (Server-Sent Events - SSE).

- Stage 5 - Incident Closure and Digital Dossier Generation: Upon receiving confirmation from the rescue team that the scene is secure, the Dispatcher triggers the Finalize & Resolve function to confirm mission completion. The system immediately transitions the incident status to Resolved, updates the dispatch order to Completed, releases the rescue team back to Available status, and automatically packages the entire evidence chain (original image, AI analysis outputs, AHP / TOPSIS trace, human audit log, response timestamps) into a closed-loop Digital Dossier compliant with FEMA ICS-209 standards for inspection and post-operation review.

2.2. Data Flow Analysis

2.2.1. System Context Diagram

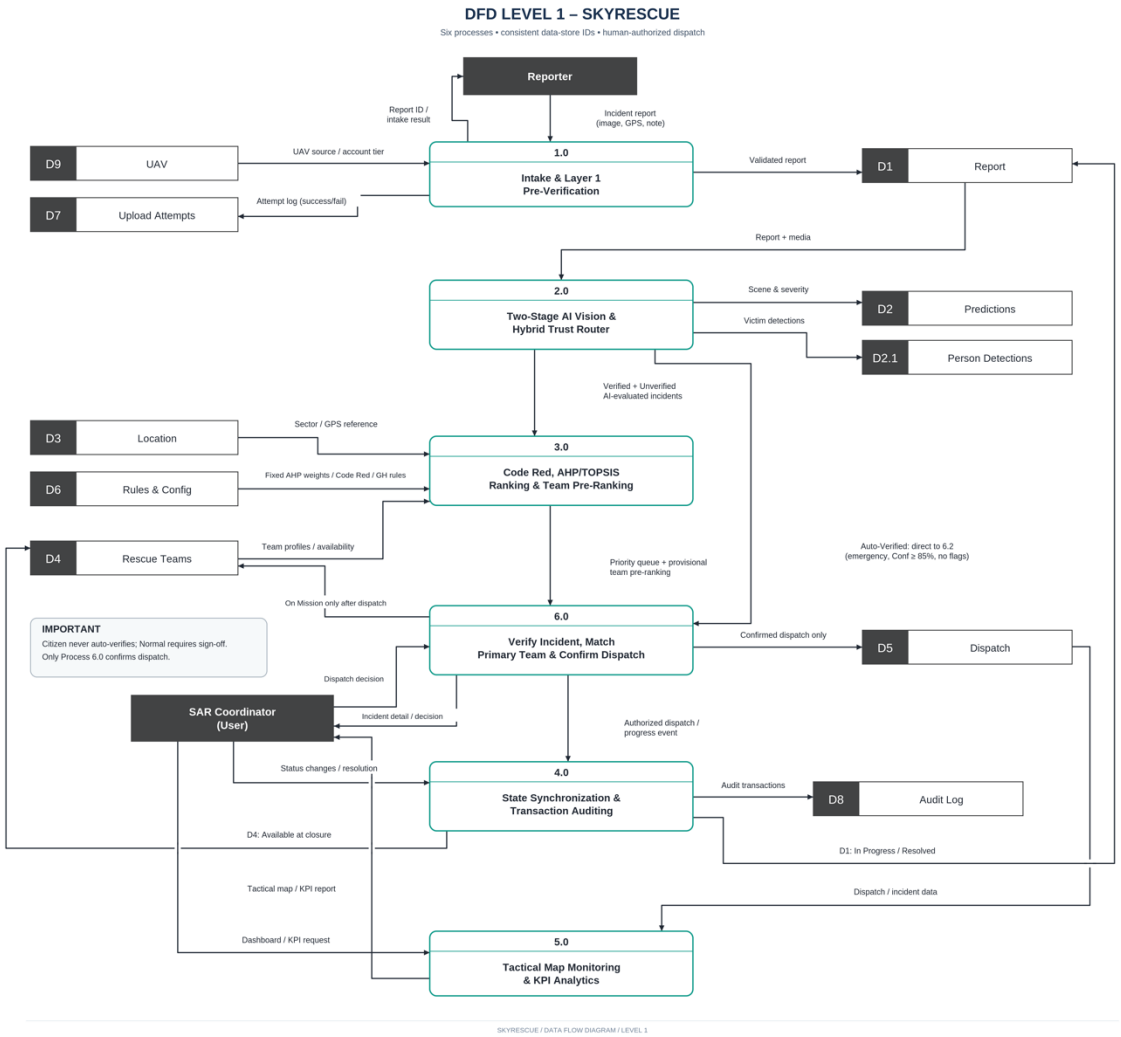


The Context Diagram clearly defines the boundary of the SkyRescue system and its two independent external actors:

- Reporter (2 Account Types: Verified Reporter / UAV / CTV vs. Unverified Citizen): The actor initiating the data flow via the Reporter Portal (/reporter/). Provides aerial or ground scene photographs, EXIF / live GPS coordinates, account tier identity, and a single optional free-text field note (without manual scene classification or victim counting), and receives Layer 1 feedback, an INC-XXXX tracking code, trust routing status (Auto-Verified vs. Pending Review), and real-time SSE lifecycle updates.

- Dispatch Admin (Command Dispatcher / SAR Coordinator): The controlling actor at the Dispatch Station (/dispatch/). Monitors the Code Red fast-track lane and AHP / TOPSIS priority queue, verifies Unverified Citizen reports on C9R, reviews Auto-Verified incidents in Matching Team (routing back to C9R via Edit Type whenever an Auto-Verified incident's category must be changed and re-verified), confirms single-primary-team dispatch orders via the 1-Click Unified Workspace or C5, and signs off on mission resolution.

2.2.2. Data Flow Diagram Level 1



The system is decomposed into six core processes interacting with six centralized data stores:

- Process 1.0 (Intake & Layer 1 Pre-Verification Gatekeeping): Accepts photo uploads and a single optional free-text field note from the two Reporter account types, validates file integrity (≤ 35 MB), EXIF / live GPS coordinates (flagging GPS conflicts > 1 km), capture timestamps, and SHA-256 / perceptual dHash duplicate checks. Valid report records are stored in data store D1 Report.

- Process 2.0 (Two-Stage AI Vision & Hybrid Trust Verification Router): Executes Model 1 (AIDER disaster classification) and Model 2 (human and pose detection), computes domain Severity Score, and routes verification status via two account-aware flows: Verified Accounts with emergency confidence ≥ 85% and no flags Auto-Verify directly to Matching Team (in D5), whereas Unverified Citizen reports (or low-confidence / flagged cases) are set to Unverified for manual Admin Review.

- Process 3.0 (Reinforced Code Red Fast Track, AHP / TOPSIS Ranking & Team Pre-Ranking): Cross-references GPS coordinates against D8/D3 Location, evaluates the 3-layer Reinforced Code Red Fast Track (High-Confidence Gate, Hazard Context Gate, and Golden

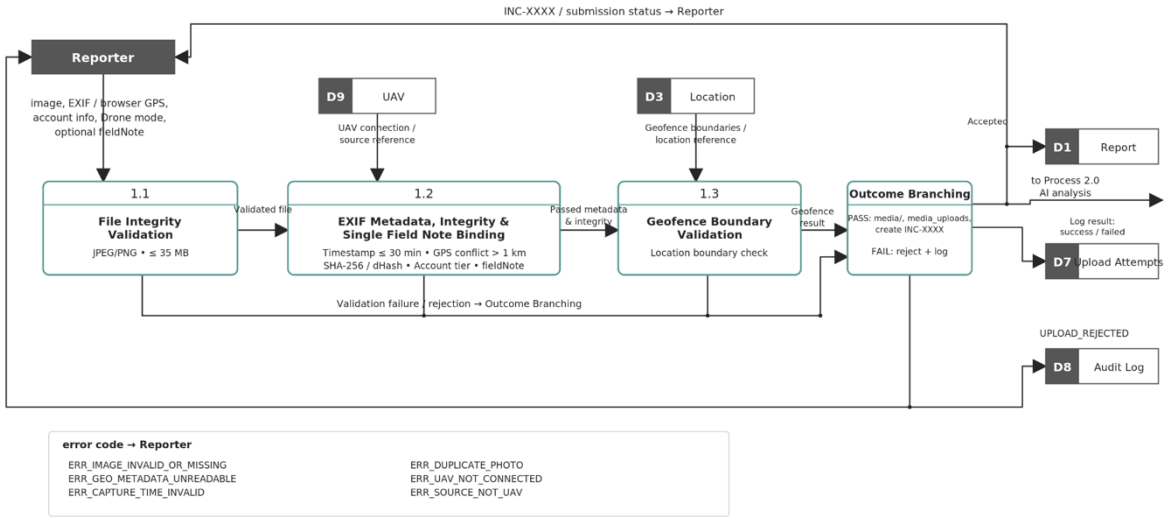
Hour Escalation at ≥ 45 minutes) to pin life-critical cases (#CR-n), feeds Incident Severity S (C1), geodesic distance to the nearest available capable team (C2), and available team count (C3) into the AHP / TOPSIS engine (weights 0.648, 0.230, 0.122; CR = 0.003), and pre-ranks candidate primary rescue teams enforcing the queue order: Life Risk (Code Red) ⟶AHP Priority ⟶Waiting Time.

- Process 4.0 (State Synchronization & Transaction Auditing): Manages the operational status lifecycle of incidents and dispatch orders in data store D5 Dispatch, processes mission completion sign-off (Finalize & Resolve), and releases rescue forces back to available pool.

- Process 5.0 (Tactical Map Monitoring & KPI Analytics): Extracts data from data stores D5 Dispatch and D6 System & AHP Config to visualize the GIS tactical map, compile KPI statistics, export Digital Dossiers, and allow configuration of alert thresholds, AHP weights, and dispatch rule sets.

- Process 6.0 (Verify Incident, Match Primary Team & Confirm Dispatch): Receives Unverified reports into Process 6.1 (Admin Review C9R) for manual verification to Matching Team, while receiving Auto-Verified reports directly into Process 6.2 (Match & Select Primary Team). If the Coordinator clicks Edit Type on an Auto-Verified incident in Process 6.2, the incident is routed back to Process 6.1 so the Coordinator can edit the type and re-verify it. Process 6.3 executes human-confirmed dispatch of a single selected primary rescue team.

2.2.3. Data Flow Diagram Level 2



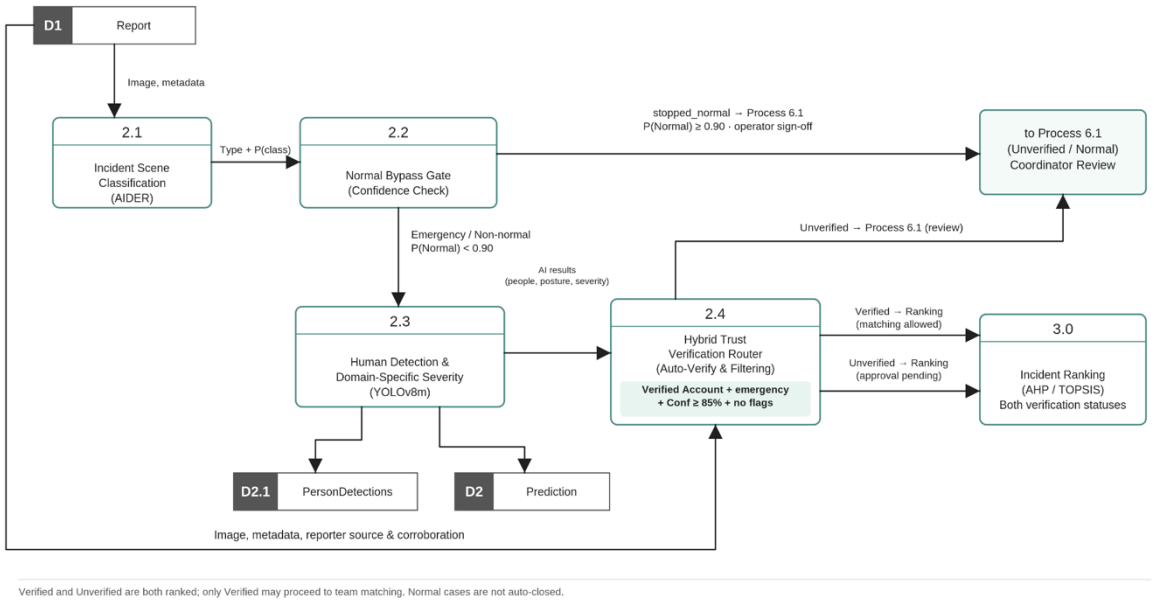
Detailed Breakdown of Process 1.0: Ensuring clean input data and blocking invalid submissions before invoking the AI pipeline:

1.1. File Integrity Validation: Verifies image file format (JPEG / PNG) and upload size (≤ 35 MB).

1.2. EXIF Metadata, Integrity & Single Field Note Binding: Extracts capture timestamp (within 30 minutes of server time), checks EXIF GPS against browser live GPS (flagging discrepancies > 1 km), computes SHA-256 and 64-bit dHash near-duplicate fingerprints, binds the Reporter's account trust tier (Verified vs. Unverified Citizen), and stores the single optional free-text field note (fieldNote).

1.3. Geofence Boundary Validation: Verifies whether EXIF GPS coordinates (or fallback browser GPS coordinates) fall within the registered operational Bounding Box.

Outcome Branching: If the submission passes all validation gates at /api/uploads, the server saves the image file to the media/ directory and records its metadata in the media_uploads infrastructure table with an uploadToken to link with the incident report in data store D1 Report (while logging a successful attempt with result = 'success' in the upload_attempts table) before triggering the AI queue. Conversely, if validation fails at any step, the server returns the corresponding error codes, logs an UPLOAD_REJECTED audit event during state synchronization, and records the failed upload attempt (result = 'failed' with error_code, SHA-256 fingerprint, and metadata) in the upload_attempts table for security monitoring and lookup.



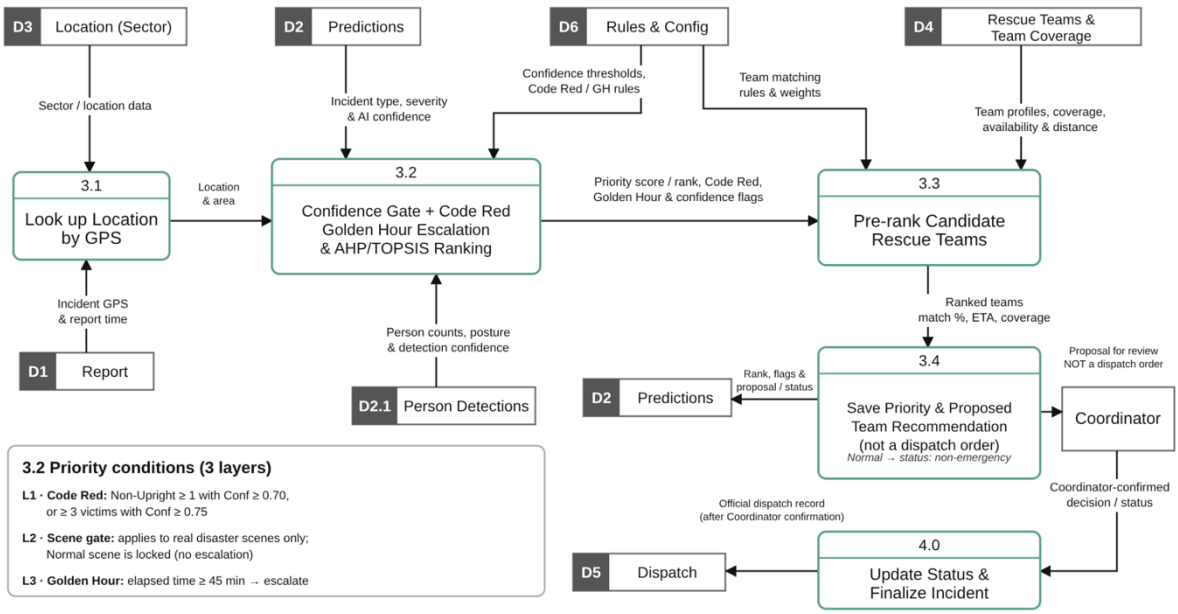
Detailed Breakdown of Process 2.0:

2.1. General Disaster Scene Classification (Model 1): Uses EfficientNet-B0 to predict probability distributions across five categories (Normal, Fire, Flood, Collapsed, Traffic).

2.2. Severity Router & Normal Bypass Filter: If P(Normal) ≥ 0.90 → routes to non-emergency closure; for disaster incidents → triggers human detection and domain-specific severity scoring in parallel.

2.3. Victim Localization & Pose Classification (Model 2): The YOLOv8m model scans the image to return Bounding Box coordinates for each person and assigns a posture label (Upright or Non-Upright), storing records in D2.1 PersonDetections.

2.4. Domain-Specific Severity Assessment & Hybrid Trust Verification Router: Computes domain severity S_raw, then routes the incident through the Hybrid Trust Router: (Flow 1) Verified Accounts with emergency AI confidence ≥ 85% and zero review flags are Auto-Verified directly to Matching Team status; (Flow 2) All Unverified Citizen reports, Normal/Unclassified cases, confidence < 85%, or flagged cases strictly remain in Unverified status for manual Coordinator review (no auto-verification for Citizen accounts under any circumstance).



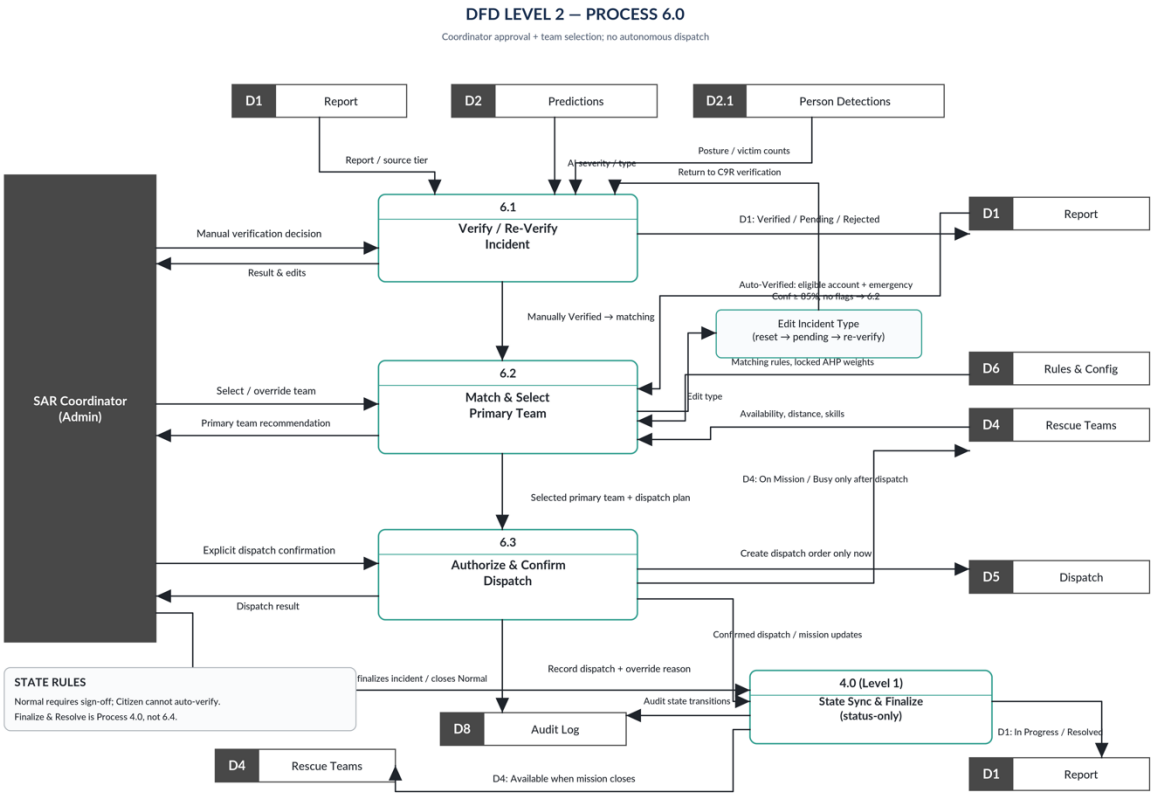
Detailed Breakdown of Process 3.0:

3.1. Look Up Location by GPS Coordinates: Receives GPS coordinates (latitude, longitude) from data store D2 Prediction, cross-references district boundaries in data store D3 Location to determine the administrative sector (area) where the incident occurred, verifies operational Geofence compliance, and forwards sector information to Process 3.2.

3.2. Rank Incident Priority via Reinforced Code Red Fast Track & AHP / TOPSIS: Evaluates the 3-layer Reinforced Code Red filter (Layer 1: Non-Upright ≥ 1 with Conf ≥ 0.70 or MCI Victims ≥ 3 with Conf ≥ 0.75; Layer 2: Real disaster context, locking out Normal; Layer 3: Golden Hour Escalation at ageMinutes ≥ 45) to set life.critical = true and pin emergencies to the top lane (#CR-n). In parallel, it combines Incident Severity S (from D2/D3 Prediction) with geodesic distance and available team counts via fixed AHP weights (0.648 for Severity, 0.230 for distance, 0.122 for available teams; CR = 0.003) and TOPSIS, sorting the operational queue strictly by: Life Risk (Code Red) ⟶AHP Priority ⟶Waiting Time.

3.3. Propose Optimal Rescue Team: Based on the priority_rank and capability profiles of rescue forces from D4 Rescue Team, the process executes team matching: filtering through mandatory safety Hard Constraints, applying contextual Soft Constraints bonuses, and computing the Team Match Score (based on specialty, distance, and capacity) to identify the single most suitable unit (proposed team).

3.4. Initialize and Save Dispatch Record: Receives the proposed team from Process 3.3 to create a provisional dispatch record. For images classified as Normal (from D2 Prediction), the process records a non-emergency closure state without triggering force deployment. All recommendation data and tactical parameters are saved in data store D5 Dispatch, ready for Dispatcher verification and authorization.



Detailed Breakdown of Process 6.0:

6.1. Evidence Inspection, Verification & Edit-Type Re-Verification (Admin Review Gate C9R): Receives Flow 2 Unverified reports for human inspection of scene imagery, bounding boxes, and field notes; clicking Verify transitions the incident to Matching Team (or closes Normal cases to Resolved). Crucially, Process 6.1 also receives Auto-Verified incidents routed back from Process 6.2 whenever a Dispatcher clicks Edit Type to change the incident category— requiring the Dispatcher to modify the type on C9R and explicitly re-verify the incident before returning to Process 6.2.

6.2. Rule-Based Single Primary Team Matching (Receiving Verified & Auto-Verified Cases): Receives both manually verified incidents from Process 6.1 and Flow 1 Auto-Verified incidents directly from data store D5 (status = Matching Team). If the Dispatcher

decides to change the incident type of an Auto-Verified case while in Process 6.2, clicking Edit Type routes the case back to Process 6.1 for re-verification. Otherwise, Process 6.2 evaluates candidate rescue units:

- Applies Hard Constraints (HC-02, HC-03, HC-04) to automatically exclude ineligible teams (e.g., heavy fire trucks blocked by narrow alleys), while supporting a Deadlock Override mechanism that requires a mandatory audit justification if zero teams satisfy all hard constraints.

- Applies Soft Constraints (SC-01, SC-03, SC-04) to award priority bonus points based on population density context and specialized equipment.

- Computes the composite Team Match Score (Specialty 50%, Distance 35%, Capacity 15%) → Recommends the single most optimal rescue team.

6.3. Authorize Dispatch Order (Confirm Dispatch): The Dispatcher approves the order, writes the record to data store D5 Dispatch Orders, updates the incident status to In Progress, and transitions the assigned team's status to On Mission.

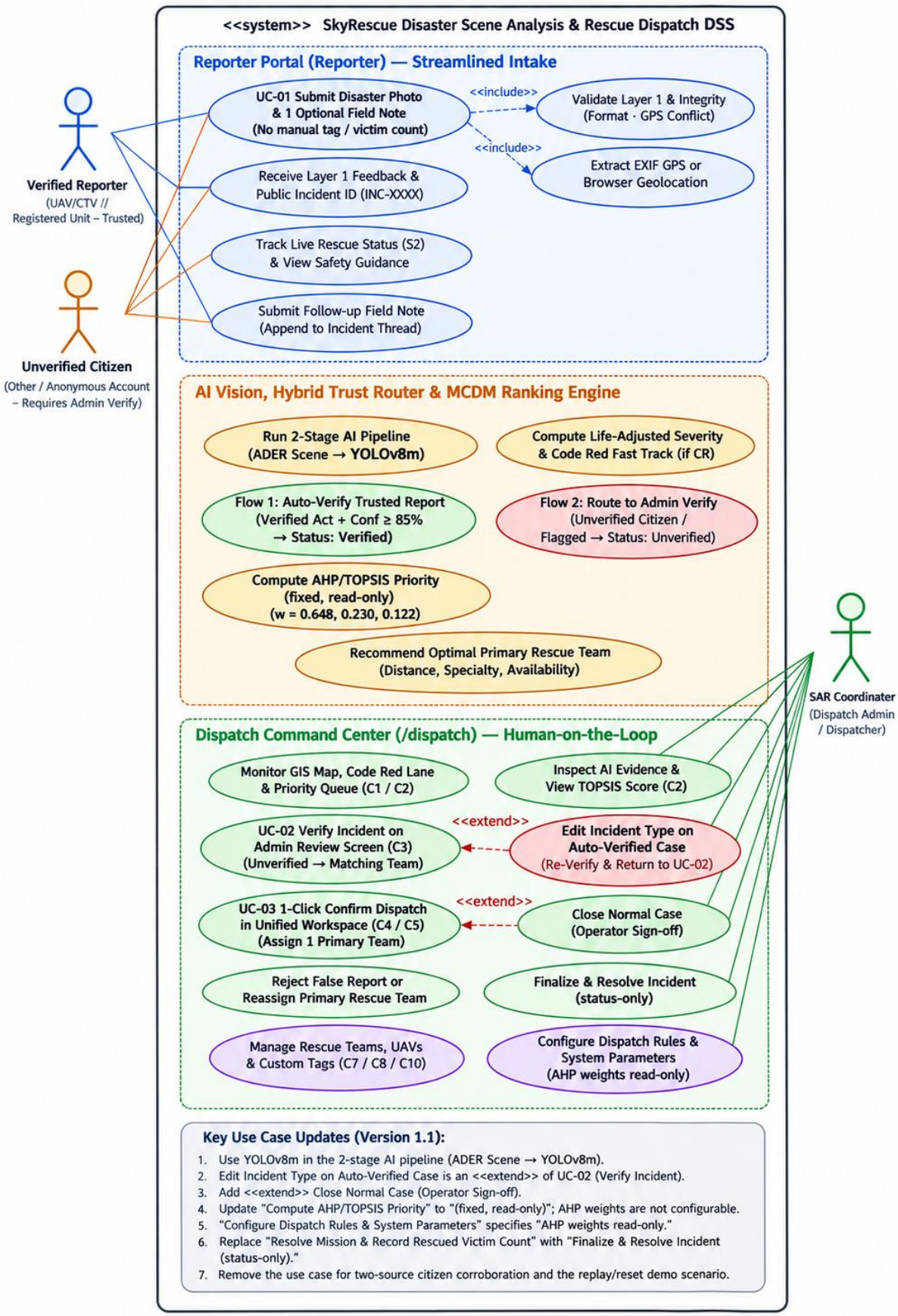
6.4. Field Sign-Off & Dossier Closure (Finalize & Resolve): Upon receiving mission completion confirmation from the field, the Dispatcher triggers Finalize & Resolve to transition the incident status to Resolved, update the dispatch order to Completed, release the rescue team back to Available status, record the audit trail, and generate a closed-loop Digital Dossier.

2.3. Use Case Analysis

2.3.1. System Use Case Diagram and Role-Based Decomposition

SkyRescue — UML Use Case Diagram (v1.1)

2 Reporter Account Types · Hybrid Trust Verification · Edit-Type Re-Verification · 1-Click Workspace Dispatch



Key Use Case Updates (Version 1.1):

1. Use YOLOv8m in the 2-stage AI pipeline (ADER Scene → YOLOv8m).
2. Edit Incident Type on Auto-Verified Case is an <<extend>> of UC-02 (Verify Incident).
3. Add <<extend>> Close Normal Case (Operator Sign-off).
4. Update "Compute AHP/TOPSIS Priority" to "(fixed, read-only)"; AHP weights are not configurable.
5. "Configure Dispatch Rules & System Parameters" specifies "AHP weights read-only."
6. Replace "Resolve Mission & Record Rescued Victim Count" with "Finalize & Resolve Incident (status-only)."
7. Remove the use case for two-source citizen corroboration and the replay/reset demo scenario.

The overall Use Case Diagram models all interactions between external actors and the SkyRescue system boundary (integrating DSS + MIS). The system distinguishes two Use Case categories: user-interactive web Use Cases (blue) and automated background AI pipeline Use Cases (green). Functions are partitioned into two independent operational workspaces:

- Reporter Portal Subsystem (/reporter/ - 2 Account Types: Verified Reporter/UAV/CTV vs. Unverified Citizen): Serves trusted reconnaissance accounts and general citizens submitting emergency reports from the field.

+ UC-01 Upload Image & Submit Report with 1 Optional Field Note (Primary Use Case): Allows users under either account tier to upload a disaster scene photo with GPS and enter only a single optional free-text field note (no manual scene tag or manual victim count).

+ Validate Image (EXIF, GPS, 30m) (<<include>> relationship): Automatically triggers Layer 1 validation (file format, SHA-256 deduplication, 30-minute capture freshness check, and EXIF GPS coordinate extraction).

+ Classify Incident, Compute Severity & Route Hybrid Trust (<<include>> executed by AI & Trust Router): Runs Model 1 and Model 2, computes Incident Severity and Code Red status, and routes Verified Account reports (Conf ≥ 85%, no flags) via Flow 1 (Auto-Verified to Matching Team) while routing Unverified Citizen reports via Flow 2 (Unverified Inbox).

+ Rank Incidents (AHP/TOPSIS) & Recommend Teams (<<include>> relationship): Automatically computes preliminary queue priority scores and compiles candidate rescue teams.

+ View Report History & Status (Independent Use Case): Allows reporters to track the submission history and rescue progress of their submitted reports (My Reports).

- Dispatch Station Subsystem (/dispatch/): Dedicated exclusively to duty dispatchers (Dispatcher / Dispatch Admin) at the central command center.

+ View Queue & Incident Details: Monitors real-time queues of verified and unverified incidents.

+ View Tactical Map: Visualizes all active disaster hotspots on the GIS tactical map.

+ Verify Incident (UC-02 on C9R): Executes Human-on-the-Loop verification for Unverified Citizen reports (and for Auto-Verified reports whose type is edited), transitioning approved emergencies to Matching Team, accompanied by two <<extend>> relationships:

- <<extend>> Edit Incident Type / Data (with Mandatory Re-Verification for Auto-Verified Cases): Allows the Dispatcher to override the incident category, victim count, or severity score. If an incident was previously Auto-Verified to Matching Team and the Dispatcher clicks Edit Type, the incident is routed back to the Verify Incident screen (C9R) where the Dispatcher must edit the type and explicitly re-verify it.

- <<extend>> Close Normal Case: Directly closes false alarms or non-emergency (Normal) cases without generating a dispatch order.

+ Assign Single Primary Team & Confirm Dispatch (UC-03 in Unified Workspace C2 / C5): Applies Hard Constraints and Soft Constraints to recommend the single optimal primary rescue team; the Dispatcher reviews and confirms the official dispatch order via 1-Click Workspace Dispatch or C5.

+ Finalize & Resolve Incident: Confirms mission completion after the rescue team reports that the scene is secure; automatically transitions the incident to Resolved, marks the dispatch order as Completed, releases the rescue team to Available status, and generates the Digital Dossier.

+ Manage Rescue Teams Roster: Manages the directory, equipment inventory, and availability status of rescue units.

+ Configure Regional Dataset & Dispatch Rules (Read-Only AHP Weights): Switches between the national dataset and 7 province views, adjusts Soft Constraint weights, creates one-sided custom equality rules (CR-xx) over the 6 seeded read-only catalog tags, and displays the fixed AHP decision weights (0.648 / 0.230 / 0.122, CR = 0.003) read-only in Configuration and DSS Trace.

+ View Reports & Digital Dossier: Queries and exports summary reports and closed-loop Digital Dossiers compliant with FEMA ICS-209 standards.

+ View Audit Log: Traces the immutable data modification history (who changed what, when, and from which old value to which new value).

2.3.2. Main Use Case Specifications

- UC-01: Upload Scene Image and Submit Incident Report with Single Optional Field Note+ Actor: Reporter (2 Account Types: Type 1 Verified Reporter / Specialist CTV / Registered UAV vs. Type 2 Unverified Citizen).+ Description: Receives a scene photograph (up to 35 MB) and a single optional free-text field note (fieldNote; manual scene classification and manual victim counting are removed), automatically extracts EXIF GPS coordinates (or falls back to browser geolocation), checks perceptual dHash near-duplicates and EXIF-vs-live GPS consistency, and validates Geofence compliance. Creates a Report record (INC-XXXX) and triggers the background AI pipeline and Hybrid Trust Router (which Auto-Verifies Type 1 Verified Account emergency reports with AI confidence ≥ 85% and no flags directly to Matching Team, while placing Type 2 Unverified Citizen reports into Unverified status for Admin Review).+ Exception Handling: Images violating file format rules, exceeding 35 MB, duplicating an existing report, or falling outside the operational coverage area are rejected and logged in the upload_attempts table.

- UC-02: Inspect, Verify, and Re-Verify Incident (Hybrid Trust & Edit-Type Re-Verification Gate)+ Actor: Dispatch Admin (Duty Dispatcher / SAR Coordinator).+ Description: For Unverified reports (from Unverified Citizen accounts, low-confidence predictions, or flagged

uploads), the Admin opens the Admin Review screen (C9R) to cross-check the scene photograph against AI detection outputs (disaster category, victim Bounding Boxes, Severity Score) and approves verification to transition the incident to Matching Team. Crucially, if an incident was Auto-Verified to Matching Team (from a Verified Account) and the Admin wishes to change its incident category, clicking Edit Type routes the incident back to the Verify screen (C9R), requiring the Admin to modify the type and explicitly click Verify again before team matching and dispatch.+ Alternative Flow: If the Admin determines the scene is non-emergency or a false alarm (Normal), the incident is closed directly to Resolved status without deploying rescue forces.

- UC-03: Recommend Single Primary Rescue Team and Confirm Dispatch (Unified Triage Workspace & C5)+ Actor: Dispatch Admin.+ Description: For incidents in Matching Team status (either manually verified via UC-02 or Auto-Verified via Hybrid Trust), the system evaluates Hard Constraints (HC-02, HC-03, HC-04) and Soft Constraints (SC-01, SC-03, SC-04) to compute the Team Match Score and recommend the single optimal primary rescue team. The Admin confirms dispatch either via 1-Click Confirm Dispatch in the Unified Triage Workspace (C2) or on the Dispatch Approval screen (C5), creating a dispatches record (DSP-XXXX), transitioning the selected primary team to Busy / On Mission, and updating the incident to Team Assigned / In Progress.+ Exception Handling (Deadlock Override): When zero teams satisfy all Hard Constraints, the system halts automatic recommendation and allows the Admin to manually select the nearest unit with a mandatory tactical justification recorded in the audit_log table.

- UC-04: Close Incident and Generate Digital Dossier (Finalize & Resolve Incident)
+ Actor: Dispatch Admin (SAR Coordinator).
+ Description: Upon receiving mission completion confirmation from the assigned primary rescue unit, the Admin triggers the status-only Finalize & Resolve action on the dispatch console.
+ Outcome: The system executes a status-only closure (without casualty-tally or debrief input forms): it transitions the incident status to Resolved, marks the single dispatches record as Completed (recording completed_at and confirmed_by_user_id), releases the primary rescue team back to Available status, appends the closure event to the audit_log table, and compiles the full evidence chain (including the server AHP/TOPSIS priority snapshot) into a FEMA ICS-209 Digital Dossier for lookup or PDF / CSV export.

2.4. Analysis of Core Business Functions

2.4.1. Multi-Layer Image Intake and Validation

The image intake and validation function acts as the Layer 1 Gatekeeper to guarantee data integrity, authenticity, and spam prevention before forwarding images to deep learning models:
- Binary File and Size Validation: Inspects file magic bytes to verify authentic JPEG or PNG formats, supporting a maximum upload file size of ≤ 35 MB (accommodating high-resolution imagery from enterprise drones such as DJI Matrice 30T / Mavic 3 Enterprise).

- Automated EXIF Metadata Extraction: The server/exif.py module automatically parses image header tags, including WGS-84 geographic coordinates (Latitude, Longitude), UAV flight altitude (GPSAltitude), actual capture timestamp (DateTimeOriginal), and device telemetry (cameraMake, cameraModel, uavSerialNumber).
- Advanced Mobile Location (AML) Browser Geolocation Fallback: The system prioritizes GPS coordinates embedded in the original image's EXIF header. When an image is captured by a device lacking GPS hardware or has stripped EXIF tags, the system activates an AML fallback mechanism via the browser's HTML5 Geolocation API to lock onto the reporter's live field coordinates.
- Bounding-Box Geofence Enforcement: Verifies that incident coordinates fall strictly within the rectangular bounding box (minLat, maxLat, minLng, maxLng) of the active operational region, incorporating a safety tolerance margin (GEOFENCE_MARGIN_DEG = 0.5°).
- Strict Server-Side Validation Error Codes:
    + ERR_IMAGE_INVALID_OR_MISSING: Invalid image file format or file size exceeding 35 MB.
    + ERR_GEO_METADATA_UNREADABLE: GPS coordinates are unreadable or fall outside the active region's geographic Bounding Box.
    + ERR_CAPTURE_TIME_INVALID: Capture timestamp is invalid, lies in the future, or exceeds the ≤ 30-minute Data Freshness window.
    + **ERR_DUPLICATE_PHOTO (plus Drone-Mode ERR_UAV_NOT_CONNECTED / ERR_SOURCE_NOT_UAV):** Rejects exact duplicate uploads via cryptographic SHA-256 fingerprinting and flags near-duplicates via 64-bit perceptual dHash comparison (evaluated strictly against uploads still referenced by active state so demo resets never cause false nearDuplicate flags), complemented by EXIF-vs-live GPS conflict flagging (> 1 km). In Drone mode, Layer 1 also enforces registered-UAV connection (ERR_UAV_NOT_CONNECTED) and camera make/serial matching (ERR_SOURCE_NOT_UAV).
- Violation History Logging (upload_attempts): Every upload rejected by Layer 1 is automatically logged into the upload_attempts table (recording IP address, error code, timestamp, and SHA-256 fingerprint) for cybersecurity monitoring and Denial-of-Service protection (Rate Limiting: 30 requests/minute/IP).
- Incident Identifier Initialization and Account-Aware Hybrid Trust Routing: Every report passing Layer 1 is assigned a unique public identifier (INC-XXXX) with its single optional field note stored in the database. Following AI inference, the Hybrid Trust Router sets the workflow status to Matching Team (verificationStatus = verified) strictly if the report comes from a Verified Account with an emergency classification (confidence ≥ 85%) and zero review flags. For all reports submitted by Unverified Citizen accounts (as well as Normal, low-confidence, or flagged reports), the status is initialized to Unverified, requiring mandatory manual Coordinator verification before advancing to team matching.

2.4.2. Incident Scene Classification

This function performs preliminary disaster category recognition from an aerial perspective using Model 1 (EfficientNet-B0):

- Disaster Categorization: Classifies each image into one of five operational classes: Collapsed building, Fire, Flood, Traffic accident, and Normal.
- Normal Bypass Gate Mechanism:
    + When the model predicts the Normal label with high confidence ($P \geq 0.90$), the system identifies the scene as safe or a false alarm. It automatically sets the stopped_normal flag, assigns a severity score of 0, and completely bypasses Model 2 (Human Detection), conserving valuable GPU/CPU resources and accelerating case closure.
    + The report is still retained in the queue for a Dispatcher to verify and sign off on closure, upholding transparency and never silently discarding citizen submissions.
- Uncertainty Margin Handling ($P(\text{Normal}) < 0.90$): When the top-probability label is Normal but confidence is below 0.90, the system does not auto-bypass; instead, it automatically routes the image to the second-highest disaster category and triggers the needs_review flag to mandate human inspection.
- Model Failure or Timeout Fail-Safe: If the AI process encounters an exception or exceeds the 60-second execution timeout, the system performs a graceful catch, assigns the Unclassified status, and locks dispatch functionality until a Dispatcher manually specifies the incident category.

2.4.3. Victim Detection and Trauma Pose Recognition

This function addresses human detection in disaster zones and quantifies trauma criticality using Model 2 (C2A YOLOv8m):
- Victim Localization and Counting: Accurately localizes the Bounding Box coordinates of each victim in the scene, yielding the total visible person count (N_{person}).
- Binary Survival Pose Classification:
    + Upright (Standing / Moving): Indicates individuals who remain mobile and potentially capable of self-evacuation.
    + Non-Upright (Lying / Crawling / Collapsed): A critical indicator showing victims who may be severely injured, suffering from smoke inhalation, unconscious, or trapped beneath rubble.
- Critical Posture Ratio Index ($R_{non\text{-}upright}$):

$$R_{non\text{-}upright} = \frac{N_{non\text{-}upright}}{max(1, N_{person})}$$

- High-Urgency Visual Alert: A high proportion of non-upright victims immediately triggers visual warning indicators, rendering prominent red-bordered bounding boxes on the tactical interface to alert the Dispatcher during triage verification, while the posture ratio contributes directly to the domain severity assessment without relying on unvalidated survival decay formulas.

2.4.4. Domain-Specific Disaster Severity Assessment

Rather than applying a uniform formula across all disasters, the system analyzes the physical characteristics of each incident category to compute a standardized severity score $S \in [0.0, 100.0]$:

- Collapsed Building Branch: Analyzes three levels of structural destruction using MobileNetV3-Small (Intact: 0.0, Damaged: 0.5, Collapsed: 1.0). Crucially, it incorporates an anti-miss Fail-safe Threshold that triggers the needs_review flag whenever $P(\text{Collapsed}) \geq 0.04$ for cases initially predicted as Intact or Damaged, preventing buried victims from being overlooked.
- Fire Branch: Uses MobileNetV3-Small to classify three hazard levels based on flame and smoke presence: No Flame / No Smoke (0.0), Flame / No Smoke (0.5), and Flame / Smoke (1.0).
- Flood Branch: Compares the bounding-box aspect ratio (h/w) of upright persons against a dry-land baseline (BASE) to estimate water submersion depth. For flood scenes lacking a measurable upright human reference, the system applies a conservative fail-safe rule: assigning the maximum score of 1.0 (100 points) and setting the needs_review flag for manual Dispatcher verification.
- Traffic Accident Branch: Quantifies accident scale through a spatial sliding-window algorithm (default 4 m x 4 m window, 1 m stride, Ground Sample Distance computed from UAV altitude) operating on victim detections from Model 2 (C2A). It identifies the densest local casualty cluster to determine pedestrian density (people/m2), which is mapped to Fruin Level of Service (LOS) standards (Grades A through F corresponding to severity scores 0.0, 0.2, 0.4, 0.6, 0.8, and 1.0), optionally factored with detected damaged vehicles.

2.4.5. Priority Queue Ranking via AHP / TOPSIS

This function fulfills the core mission of a DSS: ranking all active emergency incidents (including both Unverified reports and verified/Auto-Verified Matching Team incidents) into a unified priority queue featuring the Reinforced Code Red Life-Critical Fast Track (#CR-n) and Incident Severity Score S as Criterion C1, strictly organized by the queue hierarchy: Life Risk (Code Red) $\longrightarrow$AHP Priority $\longrightarrow$Waiting Time:

- Enforcing the Underserved & Isolated Area Priority Doctrine:
  + Within the TOPSIS algorithm, the system establishes a deliberate priority polarity:
    • Distance to Rescue Team Criterion: Greater distance receives higher priority (distance_benefit = True).
    • Available Rescue Teams Criterion: Fewer available teams receive higher priority (free_teams_benefit = False).
  + Operational Rationale: Suburban districts, remote areas, or sectors experiencing severe rescue force depletion are where victims suffer the greatest disadvantage in response reachability. Elevating the priority of these cases enables Dispatchers to issue dispatch orders earlier to compensate for long travel distances, preventing fatalities caused by geographic isolation.

- Coordinating Server-Authoritative Ranking and Client Optimistic Preview:
    + The Python backend (backend_ml/ahp_topsis.py) acts as the authoritative Single Source of Truth, persisting official TOPSIS scores into the database.
    + The client-side JavaScript module (shared-state.js) maintains a lightweight linear approximation for instantaneous UI responsiveness (< 5 ms), with the distance and free-teams variables inverted in the exact same direction to guarantee 100% ranking consistency with the Python server.

2.4.6. Optimal Rescue Team Recommendation and Matching

Once an incident reaches Matching Team status (either through Hybrid Trust Auto-Verification for Verified Accounts or manual Coordinator verification on C9R— subject to the rule that clicking Edit Type on an Auto-Verified case routes it back to C9R for re-verification), the system scans the internal rescue unit Roster to generate deployment recommendations:

- Recommending a Single Primary Response Unit: Avoiding complex multi-unit coalition scheduling, the system selects the single best-matched rescue unit based on its composite Team Match Score.
- Compatibility Evaluation Structure:
    + Specialty & Equipment Fit (50%): Evaluates alignment between the verified incident category and the unit's core specialization (Firefighting for Fire, USAR for Collapsed Building, Water Rescue for Flood, Traffic / Medical Rescue for Traffic Accident), combined with equipment bonuses (+5 points for medical/triage gear when Non-Upright casualties are present, or +5 points for UAV/thermal imaging gear when severity ≥ 85).
    + Access Distance & Same-District Bonus (35%): Computed from the Haversine distance between the unit's base station and the incident epicenter (decaying linearly across a 15 km operational radius), plus a +10 local sector bonus if the unit is stationed within the same district as the incident.
    + Headcount Capacity Score (15%): Ratio of the unit's actual personnel (headcount) to the required field manpower threshold $M_{req} = max(4, N_{person} \times 2)$
    + Soft Constraints Adjustment: The base compatibility score is adjusted by Δ_{soft} derived from contextual rules (such as SC-01 prioritizing compact agile vehicles for narrow alleys, SC-03 prioritizing local units in ultra-dense sectors, and SC-04 prioritizing heavy hydraulic extrication gear).
- team_coverage Protection Radius (M:N Relationship): In the target relational model, the team_coverage junction table defines each unit's service jurisdiction based on a 10 km coverage radius initialized from base station coordinates in ahp_topsis.py.
- Dispatch Deadlock Override Mechanism:
    + In emergency situations where physical constraints are violated (e.g., an alley is too narrow for fire trucks, or floodwaters are too deep for wheeled vehicles) resulting in zero teams satisfying 100% of mandatory conditions, the system activates the Deadlock Override mechanism.

    + This allows the commander to forcibly assign the nearest rescue unit under an on-foot operation plan, while mandating an explicit operational justification recorded in the Audit Log.

- Instantaneous State Transition: As soon as the Dispatcher clicks Confirm Dispatch, the selected rescue team's status immediately transitions to On Mission, and the incident status transitions to In Progress.

- Mission Closure (Status-Only Finalize & Resolve): Upon mission completion, the Coordinator triggers Finalize & Resolve to execute a clean, status-only closure: transitioning the incident to Resolved, updating the single Dispatch record to Completed (recording completed_at and confirmed_by_user_id), releasing the rescue team back to Available status, and writing an immutable audit event (without requiring casualty-tally or debrief remark forms).

2.5. Decision Support System (DSS) Requirements Analysis

2.5.1. Defining Decision Criteria (Severity, Geodesic Distance, Rescue Team Availability)

To ensure accurate and comprehensive dispatch prioritization, the DSS problem is structured around three core criteria:

- Criterion 1: Incident Severity Score (C_1):
    + Scale: Continuous value in [0.0, 1.0] (or [0, 100]) computed directly from domain-specific severity assessment algorithms (or uses the Coordinator's manual override score), without artificial multiplicative factors, preserving mathematical consistency.
    + Role: The most critical criterion, carrying the dominant weight in determining rescue priority.
- Criterion 2: Geodesic Distance (C_2):
    + Scale: Haversine straight-line distance (in km) from the incident GPS coordinates to the base station of the nearest available specialized rescue team.
    + Role: Measures the geographic isolation of the incident scene.
- Criterion 3: Available Specialized Teams (C_3):
    + Scale: The count of specialized rescue teams currently in Available status whose coverage area encompasses the incident location.
    + Role: Quantifies local rescue resource scarcity within the affected sector.

Through an expert AHP pairwise comparison matrix, the relative importance among criteria is established: human life criticality (C1) is weighted 3 times more important than distance (C2) and 5 times more important than available team count (C3), yielding fixed weights $w_{sev} = 0.648$, $w_{dist} = 0.230$, and $w_{free} = 0.122$ with a Saaty consistency ratio $CR = 0.003 < 0.1$.

2.5.2. Algorithmic Transparency and Explainability Requirements

A medical and emergency rescue Decision Support System must transparently explain why one incident is prioritized over another to satisfy legal accountability and post-incident auditing:

- DSS Trace Evidence Panel:
    + On the incident detail screen, the system provides a DSS Trace Panel displaying raw criterion values, vector-normalized values, component AHP / TOPSIS scores, and triggered business rules.
- Visual Evidence Overlay:
    + The interface renders the original scene photograph overlaid with victim detection Bounding Boxes color-coded by posture-Upright (green) and Non-Upright (orange-red)-enabling Dispatchers to verify AI accuracy within seconds.
- Supreme Human Authority Principle (Human-in-the-Loop & Override):
    + The DSS never autonomously dispatches rescue vehicles.
    + The SAR Coordinator retains full authority to override any AI output: modifying the incident category, adjusting the severity score, or updating the victim count. Crucially, even if an incident from a Verified Account was Auto-Verified directly to Matching Team, if the Coordinator wants to change its incident type, clicking Edit Type routes the incident back to the Verify screen (C9R) where the Coordinator must edit the type and explicitly re-verify the incident.
    + Whenever a Coordinator modifies these parameters and re-verifies the incident, the system immediately recalculates Severity Score, Code Red status, AHP / TOPSIS queue priority, and candidate primary team rankings using the human-verified values while preserving the original AI predictions intact in the audit_log table and Digital Dossier.

CHAPTER 3. SYSTEM DESIGN

3.1. System Architecture Design

3.1.1. Overall System Architecture

SkyRescue is designed using a layered architecture consisting of four main layers: Presentation, Application API, AI Engine and Persistence. Each layer is responsible for a specific set of functions. The layers exchange data through the backend to support the workflow from report submission to rescue coordination and dispatch.

The Presentation layer provides two web interfaces: the Reporter Portal and the Dispatch Station. The Reporter Portal allows users to submit incident reports from the field and track rescue progress. The Dispatch Station supports incident verification, AI result review, priority ranking and rescue team dispatch. Both interfaces access shared data through the backend.

The Application API layer is built using FastAPI. This layer receives requests from the frontend and handles the core business logic. Its main functions include user authentication, uploaded image validation, report management, status updates and activity logging. It also communicates with the AI Engine to process images. Data is exchanged through RESTful APIs. Server-Sent Events (SSE) combined with polling are used to synchronize system states across the interfaces.

The AI Engine layer is responsible for analyzing images captured at incident scenes. The system uses models and algorithms located in the backend_ml directory. The processing pipeline includes incident classification, person detection, pose estimation and severity assessment. The results are returned to the backend to update incident reports and support priority assessment. AI tasks are executed through an asynchronous processing queue within the application. Crucially, all deep learning models (EfficientNet-B0, YOLOv8m, MobileNetV3-Small) and decision algorithms run locally on the server CPU, guaranteeing full offline operational capability without dependency on external third-party cloud AI APIs. The only external network dependency is the retrieval of online map tiles (OpenStreetMap / Carto) for Leaflet rendering in the Dispatch Station.

The Persistence layer is responsible for storing system data. The backend uses SQLite with Write-Ahead Logging (WAL) enabled. The database consists of 15 domain tables and 4 infrastructure tables. Relational data is transformed into a JSON-based state representation for communication with the frontend. The infrastructure tables support session management, uploaded image management, AI result storage and data version tracking. Image files are stored in the server's media directory.

In the current implementation, all four layers are integrated within a single application. FastAPI serves both web interfaces, handles API requests and manages the AI processing queue. All components share a common data storage and state synchronization mechanism.

3.1.2. Main Components and Functions

The SkyRescue system consists of four main components: Reporter Portal, Dispatch Station, FastAPI Backend and Background ML Worker. These components work together to support incident reporting, analysis, verification and rescue dispatch operations.

The Reporter Portal is the interface designed for field reporters across two account types: Verified Accounts (Verified Reporter, Specialist CTV, Registered UAV) and Unverified Citizen Accounts. Users upload JPEG or PNG images, extract EXIF or browser GPS coordinates, and enter only a single optional free-text field note (no manual scene classification or victim counting). Before accepting a report, the backend performs Layer 1 validation (including dHash near-duplicate and GPS conflict checks). Validated reports from Verified Accounts with high-confidence emergency AI predictions (≥ 85%, no flags) auto-verify to Matching Team, whereas Unverified Citizen reports enter the Unverified review inbox. Users track live progress through the My Reports section.

The Dispatch Station is the interface designed for dispatch operators. It displays the Code Red life-critical lane (#CR-n), the AHP / TOPSIS priority queue, the GIS tactical map, and the Unified Triage Workspace. Operators verify Unverified Citizen reports on screen C9R, review Auto-Verified incidents in Matching Team (with the Edit Type guardrail that routes an Auto-Verified incident back to C9R for re-verification if the operator wishes to change its incident type), and confirm single-primary-team dispatch orders via 1-Click Workspace Dispatch or screen C5.

The FastAPI Backend serves as the central processing component. It provides APIs for both the Reporter Portal and Dispatch Station. The backend manages authentication, user sessions, report data and incident states. When an image is received, the backend performs Layer 1 validation. The validation process checks image formats, metadata, geographic boundaries, capture timestamps and duplicate submissions. If the report is valid, the backend stores the relevant information and adds an image analysis task to the AI processing queue. The backend also records state changes and provides mechanisms for data synchronization between the interfaces.

The Background ML Worker performs asynchronous image analysis tasks. When a valid report is submitted, an analysis task is added to an asyncio.Queue. The worker retrieves the task and invokes the AI Engine through ml_service. The processing pipeline uses incident classification models, person detection models and severity assessment algorithms. Once processing is complete, the AI results are stored in the database and incorporated into the report state. If the analysis fails, the task is marked as Failed and the classification result is set to Unclassified for review by a dispatch operator.

All components share a common backend and database. Whenever data changes, the backend updates the state version. The frontend receives notifications through SSE or detects changes through polling. It then retrieves the updated data without requiring a full page reload.

3.1.3. Communication and Data Exchange Mechanisms

SkyRescue uses RESTful APIs and Server-Sent Events (SSE) to facilitate data exchange between the frontend and backend. FastAPI provides APIs for both the Reporter Portal and Dispatch Station. Both interfaces share a common data state stored on the server.

RESTful APIs handle requests and data retrieval. The frontend uses HTTP to perform user authentication, image uploads, report submissions, information retrieval and status updates. Data is primarily exchanged in JSON format, while images are uploaded using multipart/form-data. The backend receives requests, processes them and returns the results to the frontend.

The shared state is managed through several key endpoints. GET /api/state returns the current state data and its version number. POST /api/sync receives operations to add, modify or delete data. These operations include set, upsert and delete. The backend processes the changes, stores them in SQLite and increments the state version number.

Server-Sent Events (SSE) are used to notify the frontend of data changes in near real time. The frontend establishes a connection to the /api/events endpoint using EventSource. When the state version changes, the backend sends the updated version number through the SSE connection. The frontend compares the received version with its locally stored version. If a change is detected, the interface calls /api/state to retrieve the latest data.

The system combines SSE with polling to maintain synchronization when the SSE connection is unavailable. The frontend sends requests to /api/version every two seconds to check the current state version. If a newer version is detected, the frontend retrieves the updated data from the backend. The polling mechanism operates continuously in parallel with SSE.

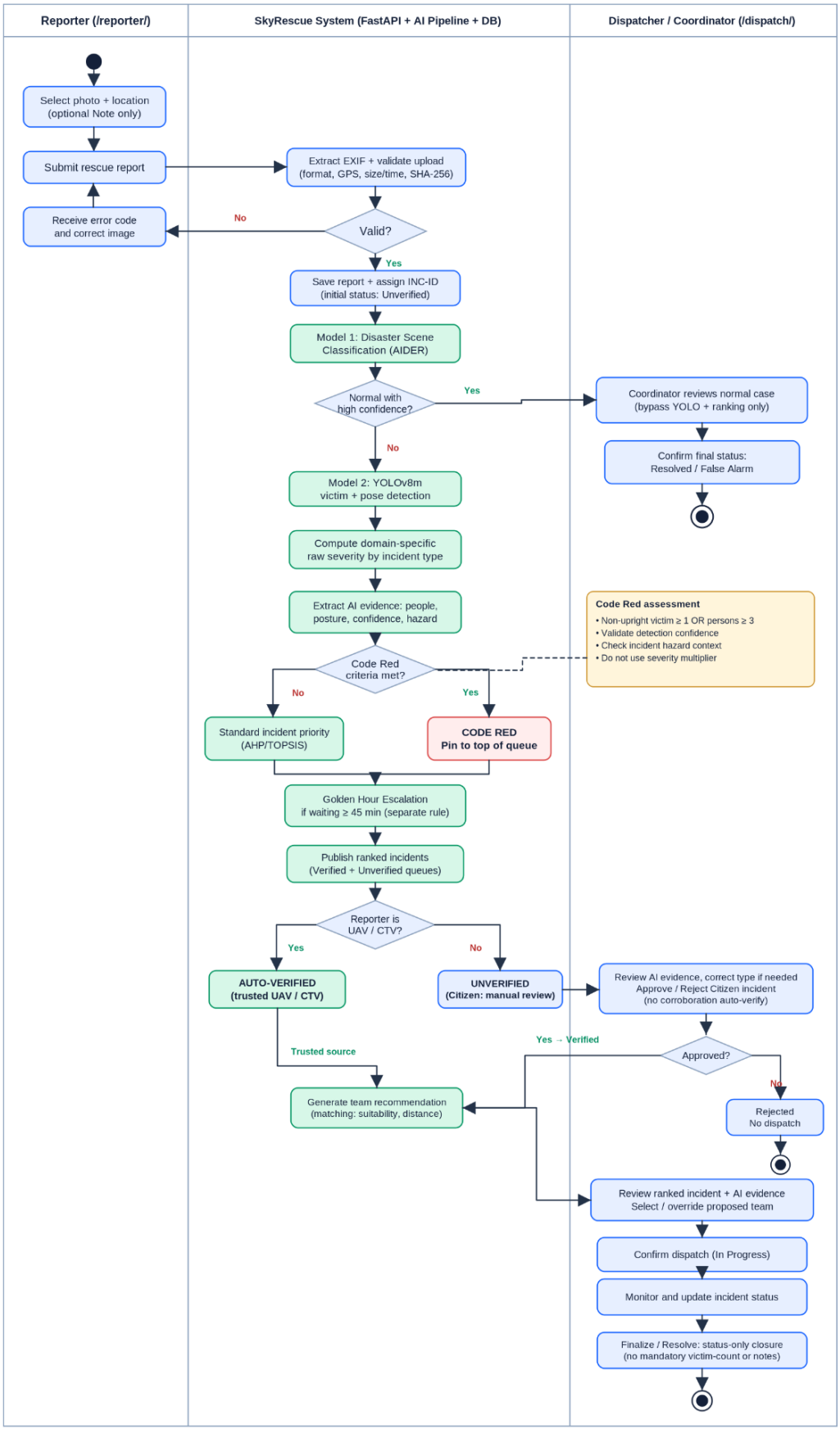
In the current implementation, SSE only transmits state version notifications. The complete state data is still retrieved through RESTful APIs. This mechanism allows the Reporter Portal and Dispatch Station to update their displayed information whenever reports, AI analysis results or dispatch states change without requiring a full page reload.

3.2. Business Process Design (Swimlane Activity Diagram)

The end-to-end business workflow of SkyRescue is modeled through a Swimlane Activity Diagram illustrating sequential interactions across three entity lanes: Field Reporter (Reporter Portal), Core System (FastAPI Backend + AI Pipeline + SQLite DB), and Command Dispatcher (Dispatcher Admin Station).

Swimlane Activity Diagram (v4)

SkyRescue incident lifecycle | Normal Bypass, strict Citizen verification, Code Red and status-only closure



3.2.1. Image Data Intake, Validation, and Error Code Handling Workflow (ERR_*)

- Initialization and Streamlined Submission (2 Account Types & 1 Optional Field Note): A field reporter logs into the Reporter Portal (/reporter/) under one of two account types— (1) Verified Account (Verified Reporter, Specialist CTV, or Registered UAV) or (2) Unverified Citizen Account— selects an aerial or ground photograph (JPEG / PNG), and optionally enters a single free-text field note (fieldNote, such as access route conditions or hazards). Manual scene classification tags, manual victim counts, and hint checkboxes are completely eliminated; scene classification and victim localization are fully delegated to the Two-stage AI Pipeline.

- Layer 1 Technical Validation: Immediately upon receiving the upload request, the FastAPI Backend executes a sequence of metadata validation checks prior to storage:

+ File Format and Size Check: The file must be a valid image format with a size ≤ 35 MB. If violated, the server returns error code ERR_IMAGE_INVALID_OR_MISSING.

+ Geofence Boundary Check: GPS coordinates extracted from EXIF (or browser Geolocation) must fall within the active region's bounding box (coverageBounds). If GPS metadata is missing or out of bounds, the server returns error code ERR_GEO_METADATA_UNREADABLE.

+ Capture Time Freshness Check: The capture timestamp (DateTimeOriginal) must not lie in the future and must not be older than 30 minutes relative to server time. If expired, the server returns error code ERR_CAPTURE_TIME_INVALID.

+ Deduplication, GPS Conflict, and Drone-Mode UAV Verification: Computes both the cryptographic SHA-256 hash (blocking exact duplicates with ERR_DUPLICATE_PHOTO) and a 64-bit perceptual dHash compared only against uploads referenced by the current state (preventing false nearDuplicate flags after a demo reset), while cross-checking EXIF GPS against live browser Geolocation (> 1 km flagged). For Drone-mode uploads, the server additionally verifies that a registered UAV is linked (ERR_UAV_NOT_CONNECTED) and that the EXIF camera make/serial match the connected UAV (ERR_SOURCE_NOT_UAV).

Outcome: If any hard Layer 1 check fails, the corresponding error code is returned to the user for correction; if valid, the system creates an incident record with identifier INC-XXXX and triggers the asynchronous AI pipeline and Hybrid Trust Verification Router.

3.2.2. Multi-Stage AI Inference Workflow

- Stage 1 - Scene Context Classification: The EfficientNet-B0 model classifies the image into one of five classes: Fire, Collapsed Building, Flood, Traffic Accident, or Normal.

Severity Router:

+ If Model 1 predicts the Normal class with probability P(Normal) ≥ 0.90, the system automatically flags stopped_normal, bypasses victim detection, and routes the report directly to the verification queue for rapid operator sign-off and closure.

+ Incident Routing: If P(Normal) < 0.90 or the image belongs to one of the four disaster classes, processing proceeds to Stage 2.

- Stage 2 - Victim Detection & Damage Assessment:

+ The YOLOv8m model (trained on the C2A dataset) detects humans, renders Bounding Boxes, and classifies victim postures (Upright or Non-Upright).

+ The Router activates the corresponding domain-specific sub-model to compute domain severity S_raw in [0.0, 100.0]. Next, the Hybrid Trust Router evaluates two verification flows: (Flow 1) If the report was submitted by a Verified Account, classified as an emergency (Fire, Flood, Collapsed Building, Traffic Accident) with AI confidence ≥ 85%, and has zero review/integrity flags, the system automatically sets verificationStatus = verified and transitions the incident directly to Matching Team; (Flow 2) All reports submitted by Unverified Citizen accounts (as well as reports with confidence < 85%, Normal/Unclassified, or flagged) strictly remain in Unverified status for manual Coordinator review, guaranteeing zero autonomous verification for unverified public submissions.

3.2.3. AHP / TOPSIS Queue Priority Scoring Workflow

- Input Variable Extraction: The system automatically extracts three decision criteria for each active emergency incident:

+ Incident Severity Score (C1): Computed directly from domain-specific severity sub-models S_raw (or the Dispatcher-overridden final_severity_score), representing the physical magnitude of damage without ad-hoc multipliers.

+ Distance: Haversine geodesic distance from the incident to the nearest capable available rescue team.

+ Free Teams: Count of rescue teams currently in Available status covering the affected sector.

- TOPSIS Algorithm Execution:

+ The decision matrix undergoes vector normalization to eliminate measurement unit differences.

+ Normalized values are multiplied by the AHP hierarchical weight vector (0.648 for Severity, 0.230 for Distance, 0.122 for Free Teams; Consistency Ratio CR = 0.003 < 0.1).

+ Applies the Underserved & Isolated Area Priority Doctrine: greater distance and fewer available teams yield higher intervention priority.

+ Computes Euclidean distances to the Positive Ideal Solution (S^+) and Negative Ideal Solution (S^-) to derive the relative Priority Score $\in$ [0, 1].

- Operational Queue Updating & Reinforced Code Red Fast Track: Incidents are evaluated against the Reinforced Code Red Life-Critical Fast Track under the core principle that human life transcends geographic distance. To eliminate 100% false alarms, the system enforces

three automated defense layers:
+ Layer 1 - High-Confidence Gate: Automatically triggers when detecting ≥ 1 motionless/collapsed victim (pose == 'Non-Upright') with AI confidence Conf ≥ 0.70; OR in Mass Casualty Incidents (MCI standard) detecting ≥ 3 victims with Conf ≥ 0.75.
+ Layer 2 - Hazard Context Gate: The incident scene must be confirmed by Model 1 as an actual physical disaster (Fire, Collapsed Building, Flood, Traffic Accident). The system automatically locks and inhibits Code Red for Normal scenes to prevent mistaking sleeping, resting, or sunbathing individuals for disaster casualties.
+ Layer 3 - Golden Hour Escalation: Any incident with detected victims that remains waiting without a dispatched rescue team undergoes automated temporal escalation: (Step 1: 0–30 min) Normal status with life.critical = false, ranked by standard AHP/TOPSIS; (Step 2: 30–45 min) Warning phase transitioning waiting badge to amber (Oldest Waiting) with audio alert on the dispatch console; (Step 3: ≥ 45 min) Forced Escalation automatically flips life.critical = true and generates an automated audit justification: codeRedReason: 'Golden Hour Escalation: Nạn nhân chờ quá 45 phút'.
Any incident triggered as Code Red automatically bypasses the standard AHP queue and is pinned to the top of the C2 Priority Queue in a dedicated crimson fast-track lane (#CR-1, #CR-2...), establishing the immutable operational queue order: Life Risk (Code Red) ⟶AHP Priority ⟶Waiting Time while preserving 100% mathematical consistency of the 3×3 AHP matrix (weights 0.648, 0.230, 0.122; CR = 0.003).

3.2.4. Rescue Dispatch, Progress Monitoring, and Incident Closure Workflow

- Two-Flow Verification Gate & Edit-Type Re-Verification Rule (C2 / C9R): Unverified reports (Flow 2: Unverified Citizen accounts, low-confidence predictions, or flagged cases) enter the Admin Review screen (C9R), where the Dispatcher inspects photographic evidence, Bounding Boxes, and the field note, adjusts parameters if needed, and clicks Verify Report to transition the incident to Matching Team (Normal cases close directly to Resolved). Conversely, Auto-Verified reports (Flow 1: Verified Accounts with emergency confidence ≥ 85% and no flags) bypass manual intake verification and enter Matching Team directly. Crucially, if the Dispatcher wants to edit the incident type of an Auto-Verified incident while it is in Matching Team, clicking Edit Type routes the incident back to the Verify screen (C9R); the Dispatcher must modify the incident type on C9R and explicitly click Verify again to re-verify the incident before team matching and dispatch can proceed.

- Rescue Team Recommendation and Matching (C4 Team Matching):

+ Mandatory Safety Filtering (Hard Constraints): Automatically excludes units violating safety regulations (e.g., blocking heavy fire trucks from narrow alleys under HC-02, or requiring rescue boats for deep flood zones under HC-03). When a deadlock occurs (zero eligible teams), the system allows the Dispatcher to execute a manual Deadlock Override with a mandatory audit justification.

+ Adaptive Bonus Scoring (Soft Constraints): Awards bonus points for agile vehicles entering narrow alleys (SC-01) or local units stationed within ultra-dense sectors (SC-03).

+ Capability Ranking: Computes the composite Team Match Score (Specialty 50%, Distance 35%, Capacity 15%) and recommends the single rescue team with the highest compatibility score.

- Issuing the Dispatch Order (1-Click Unified Workspace or C5 Confirm Dispatch): For any incident in Matching Team status, the Dispatcher can either click Confirm Dispatch directly inside the split-view Unified Triage Workspace on screen C2 (using the top-recommended primary rescue team) or open screen C5 to review full deployment parameters and click Confirm Dispatch. The system generates a dispatch order record (DSP-XXXX), assigns the single selected primary team (status On Mission / Busy), and transitions the incident status to In Progress (Team Assigned).

- Status-Only Incident Closure (Finalize & Resolve): When the assigned primary rescue team completes its field mission and reports back to command, the Dispatcher clicks Finalize & Resolve on the console. The system executes a synchronized status-only closure:
+ Transitions the incident status to Resolved, recording the closure timestamp (closedAt) and authorizing Coordinator (finalizedBy).
+ Updates the single dispatch order record (DSP-XXXX) status to Completed (recording completed_at and confirmed_by_user_id without casualty-tally or debrief fields).
+ Releases the assigned primary rescue team back to Available status to accept new missions.
+ Appends the closure event to the audit_log table and updates the incident Digital Dossier (screen C9D) with the persisted server AHP/TOPSIS snapshot for archiving and FEMA ICS-209 export.

3.3. Analytical and Decision Support Model Design

3.3.1. Incident Scene Classification Model

- Network Architecture: Uses EfficientNet-B0 loaded from checkpoint aider_classifier.pth. This architecture optimizes both accuracy and inference speed through Mobile Inverted Bottleneck Convolution (MBConv) blocks integrated with Squeeze-and-Excitation (SE) attention modules.

- Input and Preprocessing:

+ Normalized input image resolution: 224×224 pixels.
+ Standard ImageNet distribution normalization: $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$.

- Classification Head Structure: Replaces the original final classification layer with a linear projection layer connected to the five disaster classes:

$$model.classifier[1] = nn.Linear(in_features = 1280, out_features = 5)$$

Output label set:

$$Y = \{Collapsed\ building, Fire, Flood, Normal, Traffic\ accident\}$$

- Loss Function and Optimization: Trained with class-weighted Cross-Entropy loss to handle class imbalance, using the AdamW optimizer ($lr=10^{-4}$, weight decay $=10^{-2}$).
- Branching Gate Logic:

$$\text{Branch}(x) = \begin{cases} \text{Bypass Model 2, Severity} = 0, & \text{if } \hat{y} = \text{Normal} \wedge P(\text{Normal}) \geq 0.90 \\ \text{Needs Review, Route to 2nd Class,} & \text{if } \hat{y} = \text{Normal} \wedge P(\text{Normal}) < 0.90 \\ \text{Activate Model 2 \& Severity Router,} & \text{otherwise} \end{cases}$$

3.3.2. Human Detection and Victim Pose Classification Model

- Architecture: Enhanced YOLOv8m (v8m_up2_best.pt), optimized for detecting small-scale human targets from UAV Nadir (vertical) and Oblique camera perspectives.
- Output: Set of bounding boxes $B_k=(x_1,y_1,x_2,y_2,conf_k, pose_k)$ where $pose_k \in$ {Upright,Non-upright}
- Critical Posture Ratio Index ($R_{non\text{-}upright}$):

$$R_{non\text{-}upright} = \frac{N_{non\text{-}upright}}{max(1,N_{person})}$$

3.3.3. Domain-Specific Severity Assessment Algorithms

Each disaster branch computes a normalized severity score $S \in [0.0,1.0]$:

a. Collapsed Building Branch

- Uses a MobileNetV3-Small network fine-tuned on the UAVs-TEBDE dataset (collapsed_building_best_v2.pt) to classify structural damage into three levels:

$$Class \in \{Intact:0.0, Damaged:0.5, Collapsed:1.0\}$$

- Anti-Miss Fail-Safe Rule:

To prevent a genuinely collapsed building from being misclassified as intact or lightly damaged (a dangerous False Negative), the review alert flag needs_review is automatically triggered whenever:

$$P(Collapsed) \geq 0.04 \quad (when\ the\ predicted\ label\ is\ Intact\ or\ Damaged)$$

(The threshold of 0.04 was empirically calibrated on the test set, successfully catching 7/7 misclassified building collapse cases).

b. Fire Branch

- Uses the mobilenet_v3_small (best_fire_mobilenetv3_phase2.pth) model to classify three hazard levels based on flame and smoke presence:

$$Severity_{fire} = \begin{cases} 0.0, & \text{if "No Flame / No Smoke"} \\ 0.5, & \text{if "Flame / No Smoke"} \\ 1.0, & \text{if "Flame / Smoke"} \end{cases}$$

c. Flood Branch

- Measurement Calibration via Upright Human Reference: An upright person exhibits a nearly constant bounding-box aspect ratio h/w when standing on dry ground

($BASE \approx 2.5\text{-}3.2$). When floodwaters submerge the lower body, the visible box height h decreases:

$$ratio = clip\left(1 - \frac{h/w}{BASE}, 0, 1\right)$$

$$Severity_{flood} = max(ratio) \quad across\ all\ reference\ Upright\ persons$$

- Conservative Fail-Safe Principle: If a flood image contains no sufficiently large upright person to serve as a measurement reference, the system automatically assigns a conservative maximum severity score:

$$S_{flood} = 1.0 \quad and\ flag\ needsReview = True$$

d. Traffic Accident Branch

- Crowd Density Calculation via Fruin Level of Service (Fruin LOS): Uses a sliding-window algorithm ($4m \times 4m$, calibrated at a standard 12 m flight altitude ($12m$)) to compute peak human density in the most congested zone (persons/m^2):

+ Grade A (<0.31 $persons/m^2$): $S = 0.0$
+ Grade B (0.31-0.43 $persons/m^2$): $S = 0.2$
+ Grade C (0.43-0.72 $persons/m^2$): $S = 0.4$
+ Grade D (0.72-1.08 $persons/m^2$): $S = 0.6$
+ Grade E (1.08-2.15 $persons/m^2$): $S = 0.8$
+ Grade F (>2.15 $persons/m^2$ - Severe congestion / critical hazard): $S = 1.0$

3.3.4. Criterion Weight Determination via Analytic Hierarchy Process

Determining the weights of the three ranking criteria:

- C1: Incident Severity Score S_raw ∈ [0, 100] (or [0.0, 1.0]), representing the physical damage level of the disaster scene without applying artificial multiplicative factors, preserving 100% mathematical consistency with weights 0.648 / 0.230 / 0.122 and CR = 0.003.
- C_2: Haversine geodesic distance to the nearest available capable rescue team.
- C_3: Number of available specialized rescue teams covering the sector.

Saaty Pairwise Comparison Matrix (A): Established based on domain expert judgments:

- C_1 is 3 times more important than C_2 ($a_{12} = 3.0$).
- C_1 is 5 times more important than C_3 ($a_{13} = 5.0$).
- C_2 is 2 times more important than C_3 ($a_{23} = 2.0$).

$$A = \begin{pmatrix} 1 & 3 & 5 \\ 1/3 & 1 & 2 \\ 1/5 & 1/2 & 1 \end{pmatrix}$$

Weight Vector (w) Calculation:
Using the Normalized Principal Eigenvector method, we obtain:

$$w = \begin{pmatrix} w1 \\ w2 \\ w3 \end{pmatrix} = \begin{pmatrix} 0.648 \\ 0.230 \\ 0.122 \end{pmatrix}$$

Consistency Verification:

- Principal Eigenvalue: $\lambda_{max}=3.003$
- Consistency Index: $CI=\frac{\lambda_{max}-n}{n-1}=\frac{3.003-3}{2}=0.0015$
- Random Index for $n=3$: $RI=0.58$
- Consistency Ratio:

$$CR=\frac{CI}{RI}=\frac{0.0015}{0.58}=0.0026\approx 0.003<0.1 \quad \textit{(Perfectly Consistent)}$$

3.3.5. TOPSIS Priority Queue Ranking Algorithm

Given a batch of m active incidents and $n=3$ decision criteria:
Step 1: Construct the Decision Matrix $X=(x_{ij})_{m\times 3}$:

- xi1: Severity score of incident i ∈ [0, 1], taken from the operational severity score (predictions.final_severity_score, initialized from the immutable predictions.ai_severity_score and editable by the Coordinator). Criterion C1 evaluates the raw incident severity without artificial multipliers, while life-critical emergencies are prioritized via the Reinforced Code Red Fast Track (#CR-n) governed by the 3 defense layers, maintaining the queue order: Life Risk (Code Red) ⟶AHP Priority ⟶ Waiting Time.
- x_{i2}: Geodesic distance from incident i to the nearest available capable rescue team (km). If no available team is found, assign $x_{i2}=max_k(x_{k2})$ and set the no_team_flag flag.
- **x_i3:** Available rescue force capacity covering the sector of incident i (status = 'available') according to explicit team_coverage rows and the 10 km HQ radius rule. Because RescueTeam.capacity stores personnel headcount in the physical schema, the AHP engine derives concurrently deployable sub-units as floor(capacity / 4) (or uses an explicit deployableUnits attribute when present).

Step 2: Vector Normalization:

$$r_{ij}=\frac{x_{ij}}{\sqrt{\sum_{k=1}^{m} x_{kj}^2}}, \quad \forall i=1..m, j=1..3$$

Step 3: Multiply by AHP Weight Vector:

$$v_{ij}=w_j \cdot r_{ij}$$

Step 4: Determine the Positive Ideal Solution (A^+) and Negative Ideal Solution (A^-) under the Underserved & Isolated Area Priority Doctrine:

- Criterion 1 (Severity): Higher severity receives higher priority → Benefit:

$$v_1^+=\max_i(v_{i1}), \quad v_1^-=\min_i(v_{i1})$$

- Criterion 2 (Distance): Greater distance receives higher priority → Benefit:

$$v_2^+ = \max_i(v_{i2}), \quad v_2^- = \min_i(v_{i2})$$

- Criterion 3 (Free Teams): Fewer available teams receive higher priority → Cost:

$$v_3^+ = \min_i(v_{i3}), \quad v_3^- = \max_i(v_{i3})$$

Step 5: Compute Euclidean Distances to Ideal and Anti-Ideal Solutions:

$$D_i^+ = \sqrt{\sum_{j=1}^{3}(v_{ij} - v_j^+)^2}, \quad D_i^- = \sqrt{\sum_{j=1}^{3}(v_{ij} - v_j^-)^2}$$

Step 6: Compute the Relative Closeness Coefficient (C_i):

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}, \quad C_i \in [0,1]$$

The incident with the highest C_i is placed at Rank #1 in the priority queue.

Step 7: Client-Side JS Fallback Synchronization: To ensure instantaneous UI responsiveness under 5 ms when users filter or switch regions in the browser, the client-side JavaScript module (shared-state.js) uses a linear approximation aligned with the server-side TOPSIS polarity:

$$Priority_{JS} = 0.648 \times \text{clamp}\left(\frac{Sev}{100}\right) + 0.230 \times \text{clamp}\left(\frac{Dist}{20}\right) + 0.122 \times \text{clamp}\left(1 - \frac{FreeTeams}{3}\right)$$

This formula ensures smooth visual sorting while awaiting Server-Sent Events (SSE) to deliver the authoritative matrix-computed TOPSIS scores from the Python backend.

3.3.6. Rescue Team Compatibility Scoring Algorithm

Once an incident is verified, the system evaluates all currently Available rescue teams in the Roster to recommend the single most suitable primary response unit.

1. General Multi-Criteria Formula

Each rescue team's compatibility score is computed on a 0-100% scale using a multi-criteria weighted sum combined with soft-constraint adjustments:

$$BaseScore = round(0.50 \cdot S_{specialty} + 0.35 \cdot S_{distance} + 0.15 \cdot S_{capacity})$$

$$TotalScore = clamp(BaseScore + \Delta_{soft}, 0, 100) = max(0, min(100, BaseScore + \Delta_{soft}))$$

2. Detailed Calculation Components

- Component 1: Specialty & Equipment Compatibility Score ($S_{specialty}$ - Weight 50%):

$$S_{specialty} = min(100, S_{base} + bonus)$$

+ Base Specialty Score ($S_{base} \in [75,95]$): Looked up directly from the SPECIALTY_MATRIX[team.id][incident.type] mapping across the multi-specialty rescue units in the roster (covering 18 teams distributed across 7 operational provinces):
    - Primary Specialty Fit: $S_{base} \geq 90$ (e.g., Firefighting unit for Fire achieves 100; USAR unit for Collapsed Building achieves 95; Water

Rescue unit WATER-01 for Flood achieves 100; Medical / Volunteer units for Traffic Accident achieve 90–95).

- Interdisciplinary / Multi-Role Support Fit: $S_{base} \in [65,85]$ (Rapid Response unit SAR-01 achieves 80–95 across most incident categories thanks to versatile combined equipment).
- Cross-Support / Low Fit: $S_{base} \leq 55$ (e.g., a Water Rescue unit assigned to a structural fire receives only 50 points).
- Fallback Rule: For newly added units not yet listed in the static matrix, S_{base} is inferred from equipment keywords (team.equipment): boat/dinghy for Flood (95), firefighting gear for Fire (95), rescue ropes/thermal camera for Collapsed Building (90), ambulance/medical kit for Traffic Accident (90), or a baseline default of 75.

+ Compatible Equipment Bonus ($bonus \in \{0,5,10\}$):
  - Adds +5 points if the incident involves motionless/injured casualties (pose === 'Non-Upright') and the unit carries medical or triage equipment (equipment contains medical or triage).
  - Adds +5 points if the incident severity score is ≥ 85 and the unit is equipped with aerial reconnaissance or thermal imaging gear (equipment contains uav, drone, or thermal).

- Component 2: Access Distance & Local District Bonus Score ($S_{distance}$ - Weight 35%): The access distance d (km) is computed via the Haversine formula between the unit's base station and the incident epicenter, decaying linearly across a standard operational radius of 15 km:

$$S_{dist_raw} = clamp\left(\left(1 - \frac{d}{15}\right) \times 100, 0, 100\right)$$

Additionally, the system awards a local sector bonus (+10 points) if the rescue unit is stationed within the exact district where the incident occurred (team.area === incident.location.district):

$$S_{distance} = min\left(100, round\left(S_{dist_raw} + sameAreaBonus\right)\right), \quad where\ sameAreaBonus = 10$$

- Component 3: Personnel Capacity Score Relative to Victim Count ($S_{capacity}$ - Weight 15%): Evaluates the unit's available personnel relative to the safety manpower threshold required for the AI-detected victim count:

$$M_{req} = max\left(4, 2 \cdot N_{person}\right)$$

(Where $N_{person} = max(1, detectedPeopleCount)$ is the detected victim count; a minimum threshold of 4 rescue personnel is required per sortie, plus 2 additional personnel per detected victim). Given the unit's actual personnel $headcount = team.capacity$ (or team.members):

$$S_{capacity} = min\left(100, round\left(\frac{\text{headcount}}{M_{req}} \times 100\right)\right)$$

- Component 4: Soft Constraints Adjustment (Δ_{soft} from Soft Constraints Engine): The *BaseScore* is adjusted by $\Delta_{soft} \in [-30, +30]$ derived from dynamically configurable business rules:

+ SC-01 (Agile Vehicle Bonus for Narrow Alley): Prioritizes compact motorcycles or rescue vans (compact_moto, rescue_van) when the scene report indicates a narrow entrance (Report.narrow_entrance == true): $\Delta_{soft} = +12$ (for Fire) or $+10$ (for Collapsed Building).
+ SC-03 (Ultra-Dense Sector Local Unit Priority): Prioritizes local units stationed in ultra-dense population sectors (Location.population_density == 'ultra_dense'): $\Delta_{soft} = +8$
+ SC-04 (Heavy Extrication Gear Readiness Bonus): Prioritizes units equipped with heavy hydraulic extrication gear (heavy_debris_gear == true) for structural collapses or multi-vehicle pileups: $\Delta_{soft} = +8$ to $+10$.

3. Visual Classification Badges (Specialty Badge & Tone)
Based on the specialty score $S_{specialty}$, the interface renders an intuitive visual badge to assist Dispatchers in rapid decision-making:

- $S_{specialty} \geq 90$: Displays the Primary Specialty badge (green success tone — primary domain match).
- $75 \leq S_{specialty} < 90$: Displays the High Compatibility badge (neutral tone — multi-role or medical support).
- $S_{specialty} < 75$: Displays the Cross-Support badge (amber warning tone — cross-specialty / use with caution).

The unit achieving the highest composite *TotalScore* while satisfying all Hard Constraints is ranked #1 on the Team Recommendation Card for the Dispatcher to authorize deployment (Confirm Dispatch).

3.4. Database Design

3.4.1. Target Relational Schema

The SkyRescue Entity-Relationship Diagram (ERD) consists of 15 tables. These tables are organized into five functional groups:

- User Account and UAV Management: This group consists of the User and UAV tables. The User table stores user account information and user roles. The UAV table manages registered unmanned aerial vehicles and links each UAV to its owner's account.
- Location and Rescue Resource Management: This group includes the Location, RescueTeam and TeamCoverage tables. The Location table stores information about

geographic areas. The RescueTeam table manages rescue teams, their locations, operational status and capabilities. The TeamCoverage table defines the geographic areas that each rescue team can serve.

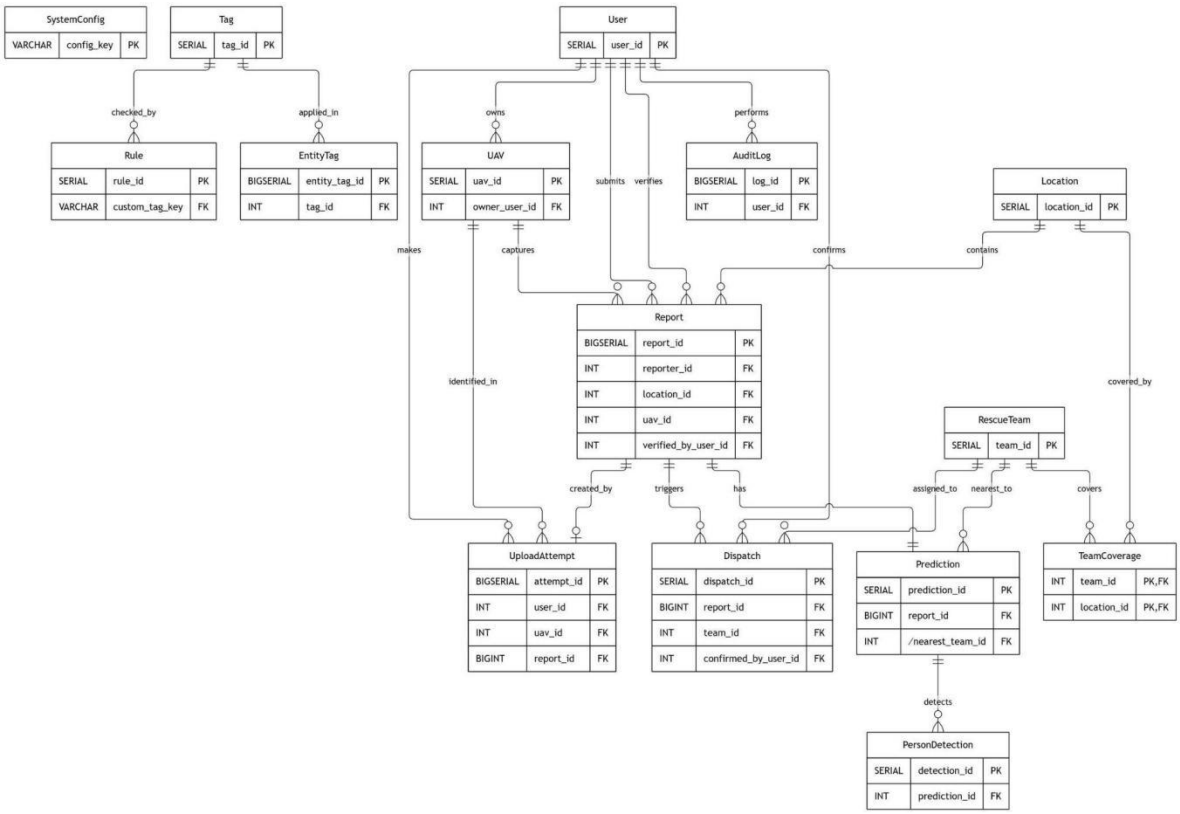
- Incident Reporting, AI Analysis and Dispatch Management: This group consists of the Report, Prediction, PersonDetection and Dispatch tables. The Report table stores incident reports and associated scene images. The Prediction table stores AI analysis results and metrics used to support priority ranking. The PersonDetection table stores information about each person detected in an image. The Dispatch table manages the assignment of rescue teams to incidents.
- **Rule and Configuration Management:** This group includes the Rule, Tag, EntityTag and SystemConfig tables. Built-in Hard Constraints (HC-02, HC-03, HC-04) are evaluated in code and mirrored as projection rows in the Rule table for UI inspection, alongside editable Soft Constraints (SC-01, SC-03, SC-04) and one-sided custom equality rules (CR-xx). The Tag table holds the six seeded read-only catalog tags (narrow_entrance, detected_people_count, access_terrain, population_density, vehicle_type, heavy_debris_gear). The EntityTag table associates tags with reports, locations or rescue teams. The SystemConfig table stores JSON configuration projections (region_meta, settings including settings.matchConfig, counters, and rule_engine_meta).
- Audit Logging and Image Upload Validation: This group consists of the AuditLog and UploadAttempt tables. The AuditLog table records user activities and data changes. The UploadAttempt table stores the outcomes of image upload attempts, including both successful and failed attempts.

In addition to the 15 domain tables, the system uses four infrastructure tables: sessions, media_uploads, ml_results and app_meta. These tables support authentication session management, uploaded image metadata storage, AI processing result storage and data version tracking. They are stored in the same SQLite database but are not part of the 15-table domain model.

Each table has a primary key (PK) that uniquely identifies its records, paired with immutable public codes (report_code INC-XXXX, dispatch_code DSP-XXXX, team_code, uav_code). Foreign keys (FKs) and UNIQUE constraints enforce relational integrity. The physical schema in server/schema.sql implements the 15 logical domain tables (130 logical attributes) plus 54 implementation and JSON-projection columns (184 physical columns in total), storing display labels for incident categories ('Fire', 'Flood', 'Building Collapse', 'Traffic Accident', 'Normal', 'Unclassified'), 'on_mission' for deployed teams (rendered as 'On Mission' in the UI), 'Other' for non-upright postures in person_detections (rendered as 'Non-Upright' in the UI), and 'seeded_sample' as a valid img_source.

3.4.2. Core Entities and Relationships

Entities in SkyRescue are interconnected across the workflow of report intake, AI analysis, and rescue dispatch. Relationships are enforced through foreign keys and junction tables.



User, UAV, and Report: A user can register multiple UAV devices and submit multiple reports. Each report links to its submitter via reporter_id, and optionally associates with a registered UAV device and the incident Location. The verified_by_user_id field identifies the Coordinator who verified the report.

In this data model, the Report entity directly manages incident lifecycle attributes. The operational status is stored in Report.status, and the scene photograph is referenced via img_path. The schema consolidates report and incident data without requiring separate Incident or Media domain tables.

Location, RescueTeam, and TeamCoverage: A geographic region may be served by multiple rescue teams, and a single rescue team may operate across multiple regions. This many-to-many (M:N) relationship is resolved through the TeamCoverage junction table using a composite primary key (team_id, location_id).

Report, Prediction, and PersonDetection: Each report is associated with at most one Prediction record via a UNIQUE constraint on Prediction.report_id. The Prediction table preserves the initial AI severity in ai_severity_score (immutable) and stores the Coordinator-editable operational score in final_severity_score (consumed by AHP/TOPSIS). A single report may contain multiple PersonDetection records (1:N), where sub-pixel bounding-box coordinates are clamped prior to insertion so database CHECK constraints never abort background ML jobs, and non-upright postures are stored as 'Other' (displayed as 'Non-Upright').

Report, RescueTeam, and Dispatch: Each report is associated with at most one Dispatch record via a UNIQUE constraint on Dispatch.report_id (1:0..1 relationship). Each Dispatch links to a single designated primary rescue unit via team_id (there is no team_role column or multi-team support allocation in the demo). The confirmed_by_user_id foreign key references User to record the authorizing Coordinator. Upon operational closure (Finalize & Resolve), the system executes a status-only update: setting Report.status to Resolved, Dispatch.dispatch_status to Completed (with completed_at), and RescueTeam.status back to Available.

Rule, Tag, and EntityTag: Built-in Hard Constraints (HC-02/03/04) are evaluated in backend code while their Rule rows serve as projection mirrors for the UI; Soft Constraints (SC-01/03/04) are editable, and custom rules are one-sided equality rules (CR-xx) over targetEntity, customTagKey, and expectedValue. The Tag table contains six seeded read-only catalog tags. The EntityTag table attaches tag values to Report, Location, or RescueTeam records via a polymorphic (entity_type, entity_id) pair.

AuditLog and UploadAttempt: The AuditLog table records the acting user, action type, target entity, and before/after values, linking to User via user_id. The UploadAttempt table logs the uploader, associated UAV device, filename, and Layer 1 validation outcome. A successful upload attempt links to its resulting Report via a UNIQUE constraint on UploadAttempt.report_id.

SystemConfig stores system-wide configuration as JSON projection rows (region_meta, settings including settings.matchConfig with match parameters 5 / 15 / 10, counters, and rule_engine_meta) without foreign key dependencies. Together, these relationships define SkyRescue's physical relational architecture in server/schema.sql.

3.4.3. As-Built Storage Architecture (Relational Architecture on SQLite WAL with Infrastructure Extensions)

SkyRescue utilizes SQLite configured in Write-Ahead Logging (WAL) mode. The database is structured around a relational schema comprising 15 domain tables and 4 infrastructure tables defined in server/schema.sql, operating by default under DATA_BACKEND=relational.

Domain data persistence is handled across the 15 relational tables managing user accounts, UAVs, geographic locations, rescue teams, reports, AI predictions, dispatch orders, rules, system configurations, and audit logs. Primary keys, foreign keys, and UNIQUE constraints enforce relational integrity and prevent data duplication.

The four infrastructure tables—sessions, media_uploads, ml_results, and app_meta—manage authentication sessions, uploaded file metadata, asynchronous AI task outputs, and global state version tracking, respectively. These infrastructure tables reside in the same SQLite database but remain separate from the 15-table domain schema.

Data exchange with the frontend retains a unified JSON State representation. The backend projects relational records from the 15 domain tables into the JSON structure consumed by the Reporter Portal and Dispatch Station. This bidirectional projection is implemented in server/relational.py, while certain UI-specific attributes are preserved in supplemental JSON columns such as payload_json, report_json, and incident_json.

Data mutations are processed centrally through the backend. The frontend sends set, upsert, or delete operations to the /api/sync endpoint. The backend applies these changes to the JSON State and persists the updated entities back into the relational tables within an atomic SQLite transaction. Upon commit, the global state version counter in app_meta is incremented.

State synchronization relies on this monotonic data version number. Whenever data changes, the backend broadcasts the new version number via Server-Sent Events (SSE). The frontend receives the notification and fetches the updated state via the REST API, supplemented by a 2-second fallback poll on /api/version to guarantee consistency across both portals.

Uploaded image files are stored on the server's filesystem under the media/ directory, while the database stores metadata and relative file paths. In Docker deployments, both the SQLite database and media files are mounted inside the persistent skyrescue-data Docker volume to survive container restarts.

In addition to the default relational mode, the system retains DATA_BACKEND=json as a fallback mode in which the entire operational state is stored as a JSON document inside the state table.

3.4.4. Mapping Between JSON State Keys and the Target Relational Schema

SkyRescue uses the relational SQLite database as its primary persistent store while exchanging data with the frontend via the JSON State structure. To bridge these two representations, the system employs an automated relational-to-JSON mapping layer in server/relational.py.

This mapping layer projects data from the 15 domain tables into the top-level JSON keys expected by the Reporter Portal and Dispatch Station, and decomposes incoming frontend state mutations back into normalized relational rows defined in server/schema.sql.

The table below details the mapping between JSON State keys and relational database tables:

JSON State Key	Relational Table(s)	Mapped Content Description
regionMeta	locations, system_config	Stores region metadata, center coordinates, coverage bounds, and administrative sub-sectors.

uavs	uavs	Stores registered UAV devices, device codes, aircraft models, and registration status.
reports	reports, upload_attempts	Stores submitted incident reports and image upload validation attempt logs.
incidents	reports, predictions	Stores operational incident records, verification status, disaster category, and severity score.
reports[].aiResult	predictions, person_detections	Stores AI inference outputs, confidence scores, severity metrics, and person detection bounding boxes.
incidents[].prioritySnapshot	predictions	Stores AHP / TOPSIS priority scores, geodesic distance to the nearest capable team, and available team counts.
teams	rescue_teams, team_coverage	Stores rescue team profiles, capabilities, operational status, and service coverage districts.
dispatchOrders	dispatches	Stores dispatch orders, assigned rescue units, Team Match Scores, and mission execution status.
settings	system_config, rules	Stores system configuration parameters and configurable dispatch rules.
customTagSchema	tags	Stores custom tag definitions and their corresponding data types.
customAttributes	entity_tags	Associates custom tag values with reports, locations, or rescue teams.
counters	system_config	Stores sequential counters used to generate public business identifiers (INC-XXXX, DSP-XXXX).

audit	audit_log	Stores immutable operational activity logs and before/after data modification traces.

In this architecture, reports and incidents are not separated into two distinct domain tables; instead, both map to the unified reports table, where each row represents an incident lifecycle record. AI inference outputs are normalized into predictions and person_detections, while dispatch assignments are stored in dispatches.

The team_coverage table is populated from each team's primary district, assigned coverage list, and the 10 km operational radius rule during initialization. Custom attributes are managed through the tags and entity_tags tables.

Beyond the JSON State keys, the users table manages authentication accounts, while the four infrastructure tables (sessions, media_uploads, ml_results, app_meta) handle session tokens, upload tracking, AI worker tasks, and state versioning outside the 15-table domain schema.

UI-specific presentation fields used during live demonstrations are preserved in auxiliary JSON columns (payload_json, report_json, incident_json), ensuring full fidelity of the frontend JSON State without introducing redundant domain tables.

When the frontend submits an update request, the backend processes the mutation on the JSON State and writes the normalized records back to the relational tables. Each successful transaction increments the state version in app_meta, enabling clients to detect changes and synchronize state.

In the final release, DATA_BACKEND=relational is the default persistence mode, with DATA_BACKEND=json retained as a fallback option. Consequently, the JSON State serves as the frontend data-exchange contract rather than the underlying primary storage format.

3.5. Data Flow and Information Security Design

3.5.1. Session Management and Role-Based Access Control

SkyRescue implements session-based authentication in server/auth.py, distinguishing two portal roles— Reporter (/reporter/) and Dispatch Admin (/dispatch/)— and further classifying Reporter accounts into two operational trust tiers: (1) Verified / Authenticated Reporter Accounts (Verified Reporters, Specialist Collaborators / CTV, and Registered UAV Operators with verified = true), and (2) Unverified Citizen / Other Accounts (general citizens or anonymous reporters with verified = false).

User authentication is processed through the /api/login endpoint. Passwords are stored as cryptographic hashes computed using the scrypt algorithm with a random salt. During login,

the backend verifies the username, password, and target portal, while enforcing a rate limit of at most 10 failed login attempts per minute per IP address.

Authenticated sessions are managed via cryptographically random session tokens stored in the sessions table alongside the user ID, portal scope, and creation timestamp. The backend delivers the token to the browser via a cookie configured with HttpOnly and SameSite=Lax attributes (with the Secure flag controlled by COOKIE_SECURE). Upon logout, the session record is deleted from the database and the browser cookie is cleared.

Access control and verification routing are enforced according to account role and trust tier. On the Reporter Portal, both Verified Accounts and Unverified Citizen Accounts submit scene imagery and a single optional free-text field note; however, high-confidence emergency reports (Conf $\geq$ 85%, zero risk flags) from Verified Accounts are Auto-Verified by the Hybrid Trust Router directly to Matching Team status, whereas all submissions from Unverified Citizen Accounts strictly remain in Unverified status until a Dispatch Admin manually verifies them on C9R. On the Dispatch Station, Dispatch Admins verify Unverified reports, confirm single-primary-team dispatches, and retain the Edit Type guardrail (clicking Edit Type on an Auto-Verified incident routes it back to the Verify screen C9R for mandatory re-verification).

The system also restricts audit log visibility: audit records are included only in the JSON State returned to authenticated Dispatch accounts. While fine-grained per-record write permissions on the generic /api/sync endpoint remain simplified for prototype flexibility, portal-level RBAC and endpoint guards enforce clear operational separation.

3.5.2. Append-Only Audit Trail for Post-Incident Accountability

SkyRescue maintains an audit log to record user actions and data modifications across the system. Implemented in the backend persistence layer, audit entries are stored in the relational audit_log table.

Each audit record captures the entry ID, acting user, action type, target entity type, target entity ID, and timestamp, along with a human-readable summary, before/after attribute values, and the list of modified fields stored across structured columns and JSON payload fields.

Audit logging is triggered automatically whenever the backend processes a state update containing audit metadata. The backend resolves the acting user directly from the authenticated server session rather than trusting client-supplied user identifiers, and also supports logging automated system events under the System actor.

An append-only discipline is enforced within the application's logging workflow: new events are strictly appended to the historical log rather than overwriting prior entries, and the frontend is blocked from directly mutating the audit or auditSeq keys via /api/sync. Although

database-level cryptographic chaining or triggers against direct SQL tampering are not yet implemented, application-layer immutability ensures reliable operational traceability.

Audit log inspection is provided in the Dispatch Station under the Audit Log view, supporting search and filtering by event type, actor, and target entity. The backend returns up to the 1,000 most recent entries ordered chronologically from newest to oldest.

These audit records enable commanders to trace verification histories, parameter overrides, and dispatch authorizations, providing an authoritative reference during post-incident reviews.

3.5.3. Automated Incident Digital Dossier Generation

SkyRescue provides an automated Digital Dossier capability to compile the complete record of each incident. Integrated into the Dispatch Station under Reports & Export, each dossier is dynamically assembled from stored system records and linked to a unique Incident ID.

The dossier is structured into six sections. Section 1 captures the original photograph, reporting source, UAV device telemetry, capture timestamp, and GPS coordinates. Section 2 presents Layer 1 metadata validation results, verification status, and linked reports. Section 3 records the Coordinator's verification decision, confirmed incident category, and review notes.

Sections 4 and 5 detail dispatch and DSS ranking metrics, including the dispatch order code, assigned rescue unit, equipment inventory, geodesic distance, Team Match Score, and server-side AHP / TOPSIS priority score, alongside dispatch confirmation and mission completion timestamps.

Section 6 displays the final resolution status and authorizing Coordinator. When a Coordinator executes Finalize & Resolve to transition an incident to Resolved, the system records the closure timestamp, marks the dispatch order as Completed, and appends the closure event to the audit log.

Dossier export is supported in both CSV and PDF formats. The Export Dossier CSV function exports structured summary fields, while Print Full Dossier (PDF) invokes the browser's native print engine (window.print()) with a dedicated print stylesheet so operators can save a formatted FEMA ICS-209 compliant PDF document.

In the current implementation, the Digital Dossier is dynamically compiled on demand when opened by a user, aggregating live data from the report, prediction, dispatch, and audit tables without requiring pre-rendered static PDF files on the server.

CHAPTER 4. USER INTERFACE DESIGN AND SYSTEM EVALUATION

4.1. System User Interface Overview (Real-Time Synchronized Dual-Portal System)

SkyRescue is built as a web application featuring two primary portals: the Reporter Portal and the Dispatch Station. Each portal provides specialized interfaces and workflows while sharing a common FastAPI Backend and SQLite database, enabling near real-time data exchange and state synchronization across both workspaces.

The Reporter Portal is designed for field reporters across two account tiers: Verified Accounts (Verified Reporter, Specialist CTV, Registered UAV) and Unverified Citizen Accounts. Its streamlined interface requires only a scene image upload (with automated EXIF or browser GPS) and a single optional free-text field note (removing manual scene classification and manual victim counting). Validated emergency reports from Verified Accounts (AI confidence ≥ 85%, no flags) auto-verify directly to Matching Team, while Unverified Citizen reports queue for Coordinator verification.

The Dispatch Station is designed for rescue dispatch coordinators. Its interface provides a GIS Tactical Map, a Priority Queue featuring the Reinforced Code Red Life-Critical Fast-Track lane (#CR-n, controlled via 3 defense layers: High-Confidence Gate, Hazard Context Gate, and Golden Hour Escalation) and AHP / TOPSIS priority rankings, an AI Evidence Viewer (C3/C9R), and a split-view Unified Triage Workspace (C2) for 1-click human-confirmed dispatch. Coordinators verify Unverified Citizen reports, review Auto-Verified Matching Team incidents (routing them back to the Verify screen C9R for re-verification if Edit Type is clicked), and manage single-primary-team dispatch orders and Digital Dossiers.

The user interfaces are developed using HTML5, CSS, and vanilla JavaScript. Each portal features a dedicated login page, with the backend enforcing portal-scoped authentication and access control. Frontend interactions communicate with FastAPI via RESTful APIs exchanging JSON payloads.

Real-time synchronization combines Server-Sent Events (SSE) with fallback polling. Whenever operational data changes, the backend broadcasts the updated state version number via SSE. Upon receiving the notification, the frontend fetches the latest state via the API, while a parallel 2-second poll on the /api/version endpoint ensures continuous synchronization if the SSE stream is interrupted.

Domain records are persisted in SQLite and projected into the JSON State format served to the frontend. When a Reporter submits a new incident or a Coordinator updates mission statuses, changes are committed at the backend and synchronized across both portals without requiring a full page reload.

In the demonstration environment, the system comes pre-seeded with 20 aerial incident images spanning 7 provinces and cities. Seeded sample reports utilize pre-computed AI outputs for immediate display, whereas live uploaded images undergo the full asynchronous AI inference pipeline and dynamically update upon completion.

4.2. Field Reporter Portal Interface

4.2.1. Image Upload, Automated EXIF GPS Extraction, and Browser AML Geolocation Interface

The Reporter Portal provides the scene image upload interface under the Submit Rescue Report section, supporting two Reporter account types: (1) Verified / Authenticated Accounts (Verified Reporters, Specialist CTVs, and Registered UAV Operators using Direct Drone Telemetry Link or verified ground upload) and (2) Unverified Citizen / Other Accounts (using Personal / Ground Upload). This two-tier account distinction directly drives the downstream Hybrid Trust Verification Router.

Scene image uploads support JPEG and PNG formats up to 35 MB. Users can select a local image file or pick from pre-packaged sample images in the demo_samples directory. Once selected, the interface renders an image preview alongside its filename and file size, allowing users to replace or clear the image prior to submission.

EXIF metadata extraction triggers automatically upon image selection. The system parses available header tags—including GPS coordinates, capture timestamp, camera make, device model, and UAV serial number—and displays them inside the EXIF / GPS / Geofencing panel.

Location resolution prioritizes GPS coordinates embedded in the EXIF metadata. For personal photos lacking valid EXIF GPS tags, the system falls back to coordinates acquired via the Browser Geolocation API (AML fallback) using the Get Live GPS button. These coordinates determine the administrative district and verify compliance with the active operational Geofence.

For demonstration sample images, the interface supports metadata simulation when original capture timestamps are outdated or device tags are absent, while preserving authentic EXIF coordinates whenever available. Simulated fields are clearly badged on the UI to distinguish them from native EXIF metadata.

Upon submission, the frontend uploads the binary file and metadata to the /api/uploads endpoint. The backend re-verifies file headers and executes Layer 1 validation; if valid, the image is saved to the media/ directory and an uploadToken is returned to bind the media file to the incident report.

4.2.2. Supplemental Field Notes and Layer 1 Validation Feedback Interface

After selecting an image, the Reporter Portal provides a streamlined Supplemental Field Context section consisting of a single optional free-text field note (#r1-access-note / fieldNote) to capture auxiliary ground observations (such as narrow alley access, downed power lines, or a callback number) without imposing cognitive burden during an emergency. All manual scene classification dropdowns (sceneTag), manual victim count inputs (victims), and hint checkboxes have been removed from the Reporter Portal.

Disaster categorization and victim counting are 100% automated by Model 1 (EfficientNet-B0) and Model 2 (Faster R-CNN / YOLOv8m) within the Two-stage AI Pipeline, eliminating any 'reporter declaration vs. AI conflict' errors. Once AI inference completes, the Hybrid Trust Router applies two verification flows based on the reporter's account tier: (1) If submitted by a Verified Account and classified as an emergency (Fire, Flood, Collapsed Building, Traffic Accident) with AI confidence ≥ 85% and zero review flags, the incident is Auto-Verified (verificationStatus = verified) and pushed directly to Matching Team status; (2) If submitted by an Unverified Citizen account (or if Normal, confidence < 85%, or flagged), the report strictly remains in Unverified status awaiting manual Coordinator verification in the Admin Review Inbox, ensuring citizen submissions are never auto-verified under any condition.

Layer 1 gatekeeping validation is executed before a report is accepted into the queue. The system validates image format and size, GPS coordinates, geographic coverage boundaries, capture timestamp freshness (within 30 minutes of submission), and SHA-256 cryptographic deduplication against previously accepted photos.

In Drone mode, the system validates the selected registered UAV device, requiring the camera make and UAV serial number in the image metadata to match the registered aircraft profile. In Personal mode, submissions are bound to the Reporter account and allow browser geolocation fallback.

Validation status checks are displayed in real time inside the Submission Gate panel, indicating pass/fail states for each criterion. If the backend rejects an upload, the interface transitions to the Upload Rejected screen, displaying the rejected thumbnail, specific error code (ERR_IMAGE_INVALID_OR_MISSING, ERR_GEO_METADATA_UNREADABLE, ERR_CAPTURE_TIME_INVALID, or ERR_DUPLICATE_PHOTO), root cause, and remediation instructions.

Additionally, the frontend includes a client-side image sharpness check using Laplacian variance against a threshold of 100. This metric is displayed as an informational quality indicator on the client UI and is not currently enforced as a hard rejection gate on the backend.

When an upload fails Layer 1 validation, the report is blocked from entering the AI inference pipeline, and the failed attempt is logged with its corresponding error codes. The user can return to the upload screen to correct the issue and resubmit.

When an upload satisfies all Layer 1 criteria, the system accepts the report and generates a unique incident code (INC-XXXX). If the submitter is an Unverified Citizen account, the incident remains in Unverified status and advances to team matching only after a Coordinator verifies it on the Admin Review screen (C9R). If the submitter is a Verified Account and the AI pipeline detects a high-confidence emergency (≥ 85%, no flags), the incident is Auto-Verified directly to Matching Team status— though if the Coordinator subsequently wishes to change its incident type, clicking Edit Type routes the incident back to the Verify screen (C9R) for mandatory re-verification.

4.2.3. Submission History and Personal Rescue Progress Tracking Interface (My Reports)

The Reporter Portal provides the My Reports interface for users to review their submission history and track live rescue progress. Data is synchronized from the shared system state, automatically reflecting status updates whenever a Coordinator verifies, dispatches, or resolves an incident.

An overview summary displays four key counters: total upload attempts, successful reports, active incidents currently in dispatch or rescue, and resolved incidents. Rejected upload attempts are also listed so users can audit their submission outcomes.

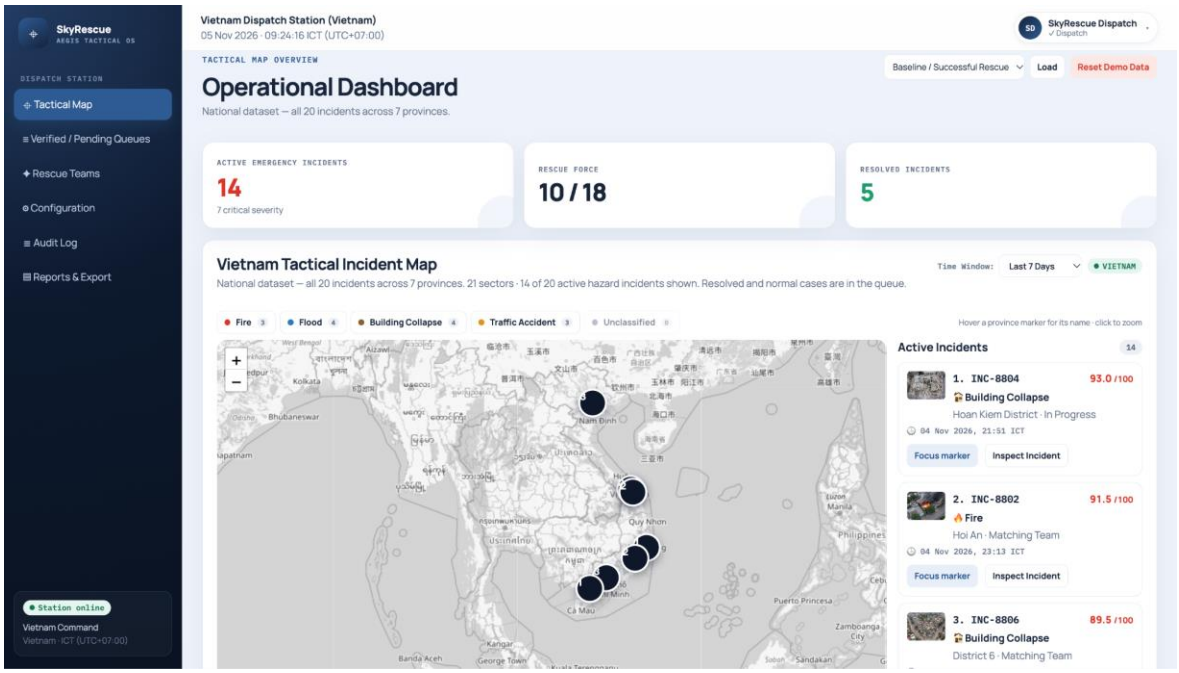
The report history is presented in a structured table displaying the Incident Code (e.g., INC-XXXX, representing the report's public operational identifier), submission timestamp, verification status, incident category, and rescue progress state. The interface supports filtering by upload outcome and rescue status, keyword search across incident IDs, categories, UAVs, and assigned teams, and pagination displaying up to six records per page.

Clicking the Open button launches the detailed report modal, showing the scene photograph, verification status, incident category, and GPS coordinates. Once a rescue unit is dispatched, the modal displays the assigned team's name, unit code, and team leader; for rejected uploads, it lists the specific Layer 1 error codes and rejection reasons.

Rescue progress is visualized across a four-stage timeline: Report Intake, Admin Review, Dispatch, and Rescue Outcome. Each stage highlights its live completion state, enabling reporters to follow their report from initial verification and team matching through active field response and final resolution (reports verified as Normal are closed directly without team dispatch).

4.3. Command Dispatch Station Interface

4.3.1. GIS Tactical Map Dashboard for Incident and Rescue Resource Monitoring



The Dispatch Station features an interactive Tactical Map interface to monitor the geographic distribution and real-time status of active incidents. Built with the Leaflet library using OpenStreetMap base tiles, the map plots each incident at its extracted GPS coordinates.

A top-level tactical summary displays three operational indicators: active emergency incidents, available rescue units, and resolved incidents. To prevent visual clutter, the tactical map displays only active, unresolved emergency incidents (excluding Normal and Resolved cases, which remain accessible in the queue views).

Incident locations are rendered using custom map markers. At the national zoom level, incidents are clustered by province or city, with each cluster badge indicating the number of active incidents in that region. Selecting a region or zooming in expands individual incident markers at their exact coordinates, color-coded by disaster category and scaled by severity score.

Clicking any incident marker opens an interactive popup card displaying the Incident ID, scene thumbnail, disaster category, district location, Severity Score, and report timestamp, along with an Inspect Incident button that navigates directly to the detailed inspection and verification view.

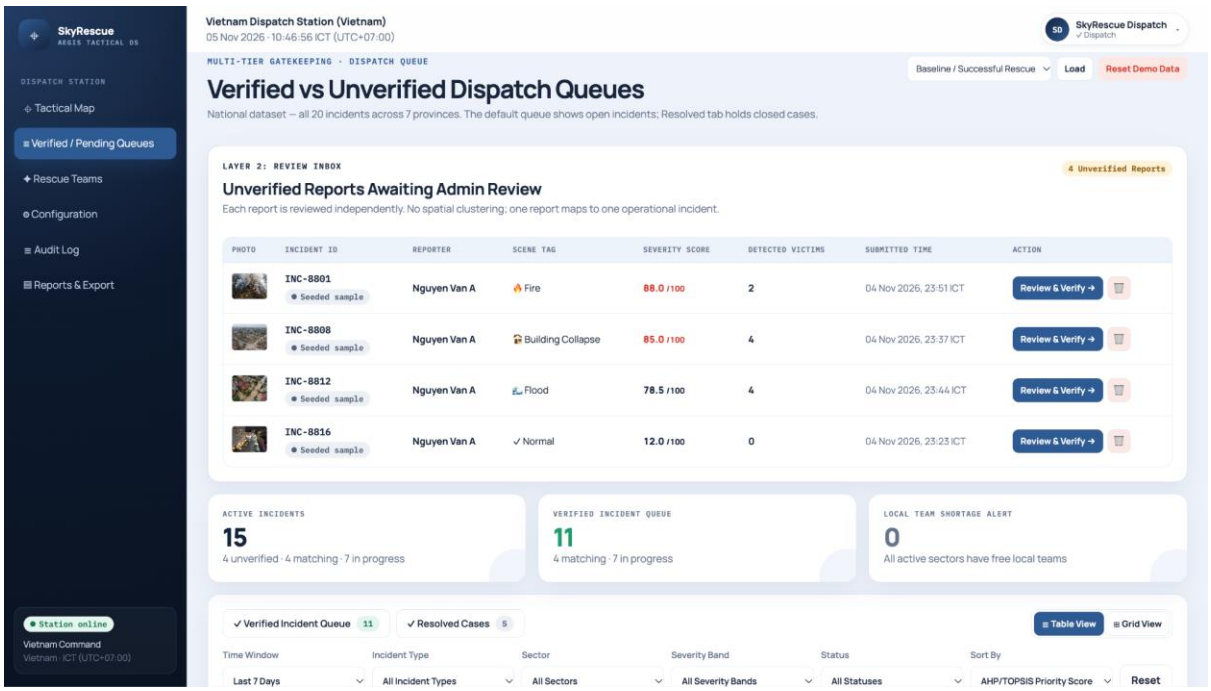
Map filtering controls allow operators to filter markers by disaster category, time window, and geographic region, switching seamlessly between the national overview and specific provinces or cities. An adjacent incident sidebar lists active cases sorted in descending order

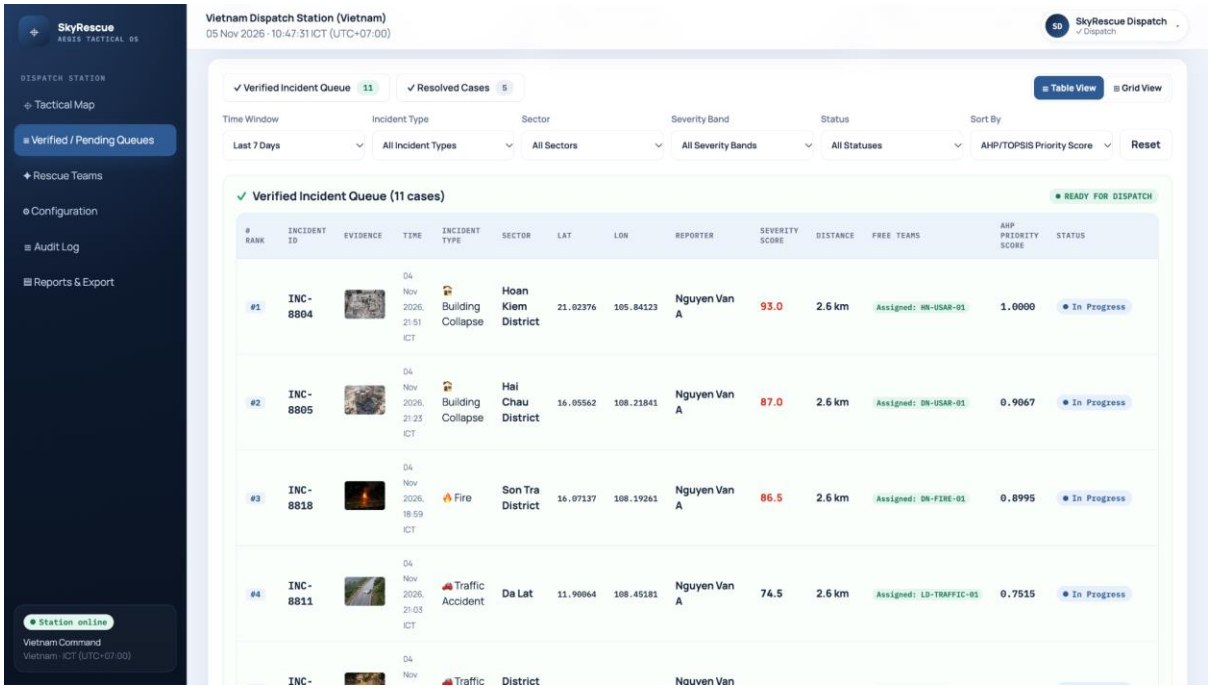
of Severity Score, providing a Focus Marker action to pan and zoom the map directly to any selected incident.

If external Leaflet assets or map tiles fail to load in an offline environment, the interface automatically activates a built-in fallback SVG tactical map that preserves marker plotting, regional filtering, and incident inspection capabilities.

In the as-built implementation, the Tactical Map provides five document hazard-type filters (Fire, Flood, Building Collapse, Traffic Accident, and Unclassified), colors incident markers by hazard type, plots active non-Normal incidents (14 in the default national seed), aggregates by province at national zoom with province drill-down and viewport-culled markers, and lists rescue unit base station locations in the Rescue Teams Roster (screen C7) rather than plotting team base pins on the map canvas.

4.3.2. Visual Priority Queue Interface





The Dispatch Station provides the Priority Queue interface (C2) to manage and rank incoming and active incidents alongside a split-view Unified Triage Workspace. At the top of the queue, a dedicated crimson Reinforced Code Red Life-Critical Fast-Track lane (#CR-1, #CR-2...) automatically pins life-critical emergencies satisfying the 3 defense layers (Layer 1: Non-Upright ≥ 1 with Conf ≥ 0.70 or MCI Victims ≥ 3 with Conf ≥ 0.75; Layer 2: Confirmed disaster context, blocking Normal; Layer 3: Golden Hour Escalation at ≥ 45 minutes with codeRedReason logging). The operational queue strictly follows the multi-tier hierarchy: Life Risk (Code Red) ⟶AHP Priority ⟶Waiting Time. Below the Code Red lane, the queue is organized into Unverified (Citizen or flagged reports awaiting Coordinator verification) and Verified / Matching Team (including both manually verified incidents and Hybrid Trust Auto-Verified incidents ready for team dispatch), complemented by a Resolved Cases archive.

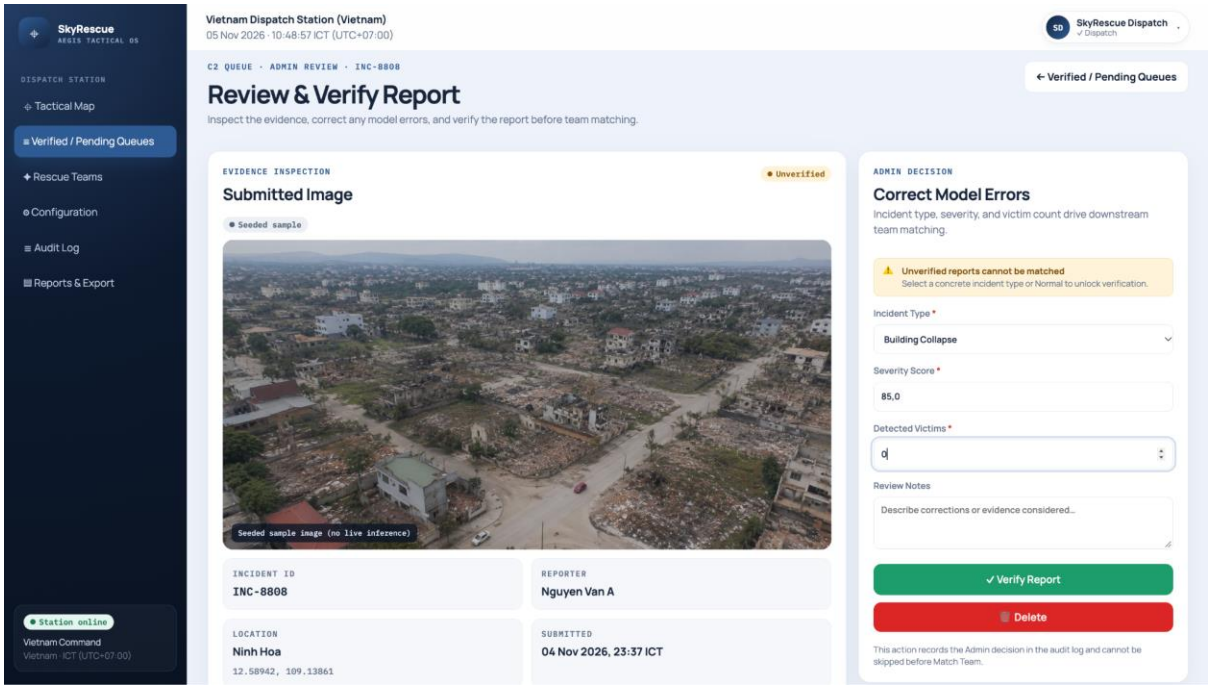
Queue prioritization is driven by the hybrid AHP / TOPSIS algorithm evaluating three criteria with fixed weights: Incident Severity S (Criterion C1, weight 0.648), Geodesic Distance to the Nearest Capable Free Team (Criterion C2, weight 0.230), and Available Specialized Teams Count (Criterion C3, weight 0.122), maintaining a Consistency Ratio of CR = 0.003. Each queue card transparently displays the criterion values and component AHP / TOPSIS scores.

Authoritative calculations are executed by the Python backend (backend_ml/ahp_topsis.py) and exposed via the /api/ahp/priority endpoint, assigning each active emergency incident a Priority Score and a Priority Rank across both Verified and Unverified emergency cases (Normal and Unclassified reports remain visible for administrative processing but are excluded from TOPSIS emergency ranking).

The queue interface supports both Table View and Grid View layouts, displaying each incident's ID, scene thumbnail, category, severity badge, priority score, and workflow state. Operators can filter records by time range, disaster category, region, severity tier, and status, or sort dynamically by priority rank, severity, distance, timestamp, and detected victim count.

While awaiting asynchronous server updates, the frontend renders an instantaneous client-side JavaScript priority preview. Once the authoritative Python AHP / TOPSIS calculation completes, the official score, rank, and computation timestamp are persisted into the incident record upon dispatch confirmation.

4.3.3. Detailed Incident Verification and AI Evidence Viewer Interface



The Dispatch Station integrates the AI Evidence Viewer within the Incident Detail and Admin Review workspace, enabling Coordinators to inspect photographic evidence, computer vision detections, and algorithmic traces prior to verifying an incident.

The visual evidence canvas displays the original scene image overlaid with interactive person detection Bounding Boxes. Each bounding box is annotated with an object ID, posture classification (Upright vs. Non-Upright), and detection confidence, synchronized with an adjacent tabular detection list where clicking any row highlights the corresponding person on the image.

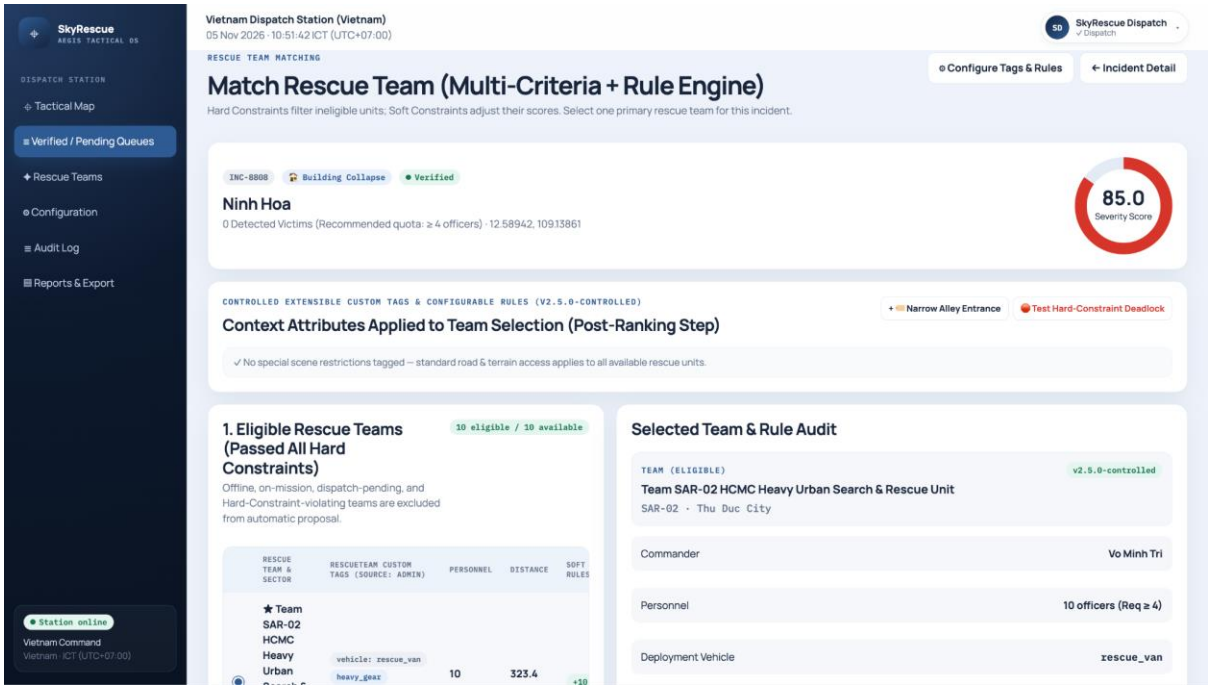
The interface explicitly distinguishes between pre-seeded demonstration samples and live uploaded images using Seeded Sample and Live Upload badges. Seeded samples display pre-computed detections for rapid demonstration, whereas live uploads invoke the backend AI inference worker to generate real-time bounding boxes.

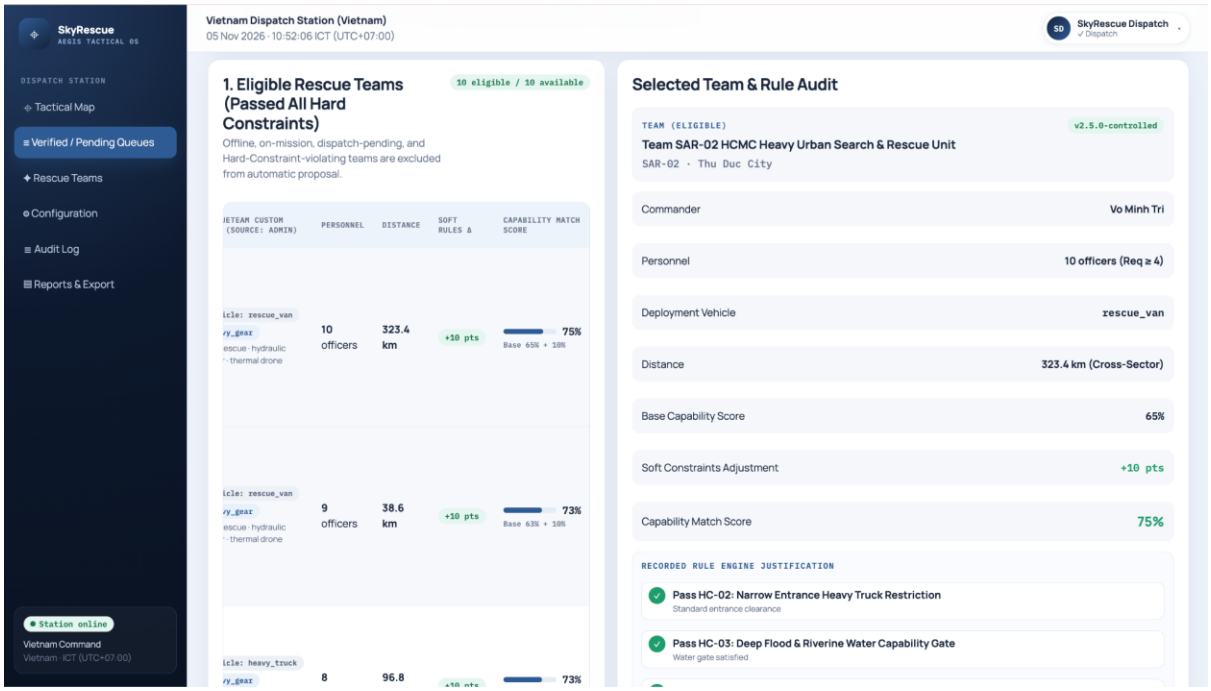
The AI assessment summary presents the predicted disaster category, AI Confidence, Severity Score, detected victim count (Upright vs. Non-Upright), and the Hybrid Trust verification badge (Auto-Verified Trusted Source vs. Pending Admin Review). During Admin Review on screen C9R, the Coordinator can override the incident category, adjust the severity score, or update the victim count; all modifications are logged alongside the verification sign-off.

For live-analyzed images, the DSS Trace panel exposes the full multi-stage execution trace—covering Stage 1 scene classification, Stage 2 human and pose detection, domain-specific severity wrapper outputs, inference latency, AHP weights, and review warnings—alongside a Re-run AI button to re-enqueue inference if needed.

After reviewing an Unverified report on C9R, the Coordinator clicks Verify Report to transition the incident to Matching Team. Crucially, for incidents that were Auto-Verified directly to Matching Team by the Hybrid Trust Router, if the Coordinator wants to change the incident type (Edit Type), clicking the Edit Type button routes the incident back to the Admin Review / Verify screen (C9R) and resets its verification gate so the Coordinator must select the new incident type and explicitly click Verify Report again before team matching and dispatch can proceed.

4.3.4. Rescue Team Recommendation and Dispatch Order Confirmation Interface





Once an incident is in Matching Team status (either after passing manual Admin Review on C9R or via Hybrid Trust Auto-Verification from a Verified Account), the Dispatch Station unlocks the Team Matching workflow to recommend the single optimal primary rescue unit. If the Coordinator clicks Edit Type on an Auto-Verified incident during team matching, the incident is immediately routed back to the Verify screen (C9R) for re-verification prior to dispatch.

Candidate rescue teams are evaluated across three weighted criteria: Specialty & Equipment Fit (50%), Geodesic Distance & Local District Bonus (35%), and Headcount Capacity (15%), producing a composite Team Match Score (Capability Match Score) on a 0–100% scale.

Prior to scoring, the engine filters out teams that are currently Offline or already On Mission, and evaluates remaining available units against active Hard Constraints (mandatory access and equipment requirements) and Soft Constraints (contextual bonus/penalty adjustments).

The recommendation panel lists eligible rescue units in descending order of their composite Team Match Score, displaying each unit's ID, specialty badge, distance, headcount, equipment tags, satisfied/violated constraints, and soft-constraint score adjustments so the Coordinator can select the most appropriate unit.

If zero available teams satisfy all Hard Constraints, the system halts automatic recommendation and presents the Manual / Deadlock Override option. To exercise an override, the Coordinator must select a unit and enter a mandatory operational justification, which is permanently recorded in the dispatch order and Audit Log.

Final dispatch authorization can be executed in two ways: (1) directly via the 1-Click Confirm Dispatch button inside the split-view Unified Triage Workspace on screen C2 (which pairs the selected incident's AI evidence and field note with the Top-1 recommended

primary rescue unit for rapid human-on-the-loop authorization), or (2) on the dedicated Dispatch Approval screen (C5), which summarizes the verified incident parameters, selected primary rescue unit, compatibility score breakdown, travel distance, and applied rules before clicking Confirm Dispatch.

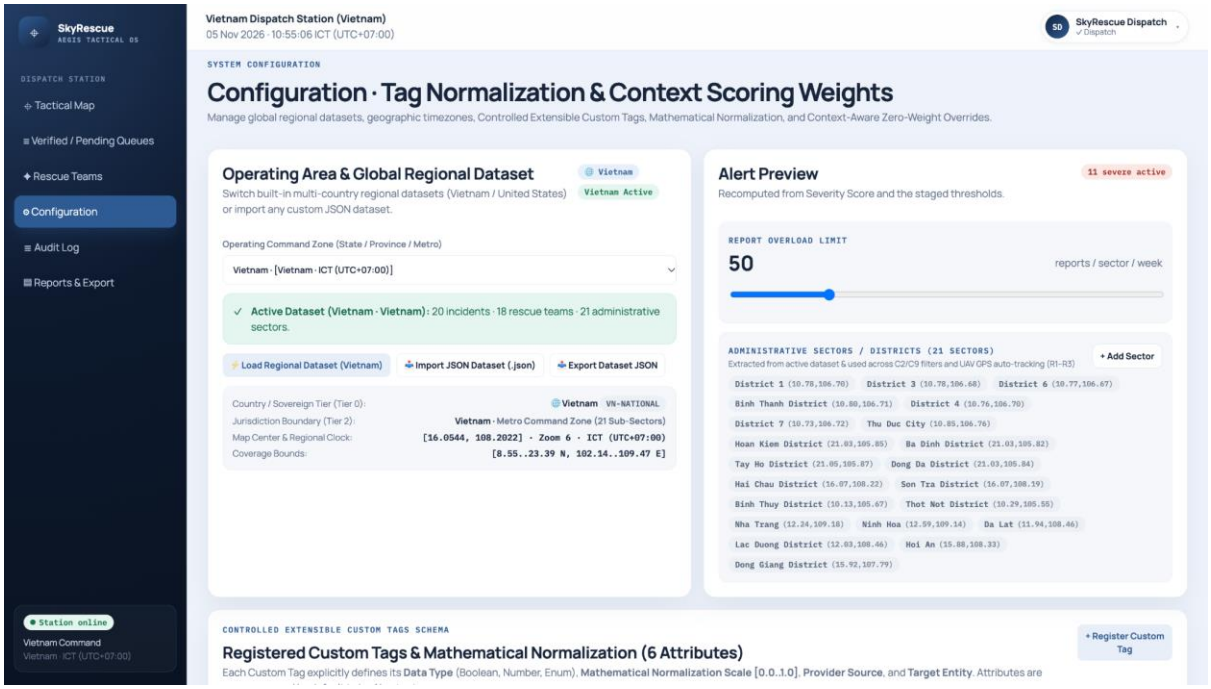
In accordance with the single-primary-unit business rule, confirming dispatch generates a unique dispatch order (DSP-XXXX), transitions the incident status to In Progress, sets the assigned rescue team's status to On Mission, and persists the transaction to the database.

When field operations conclude, the Coordinator executes the status-only Finalize & Resolve action, which transitions the incident to Resolved, marks the single dispatch order as Completed (recording completed_at and confirmed_by_user_id without casualty-tally or debrief forms), releases the rescue team back to Available status, records the audit event in the audit_log table, and updates the Digital Dossier.

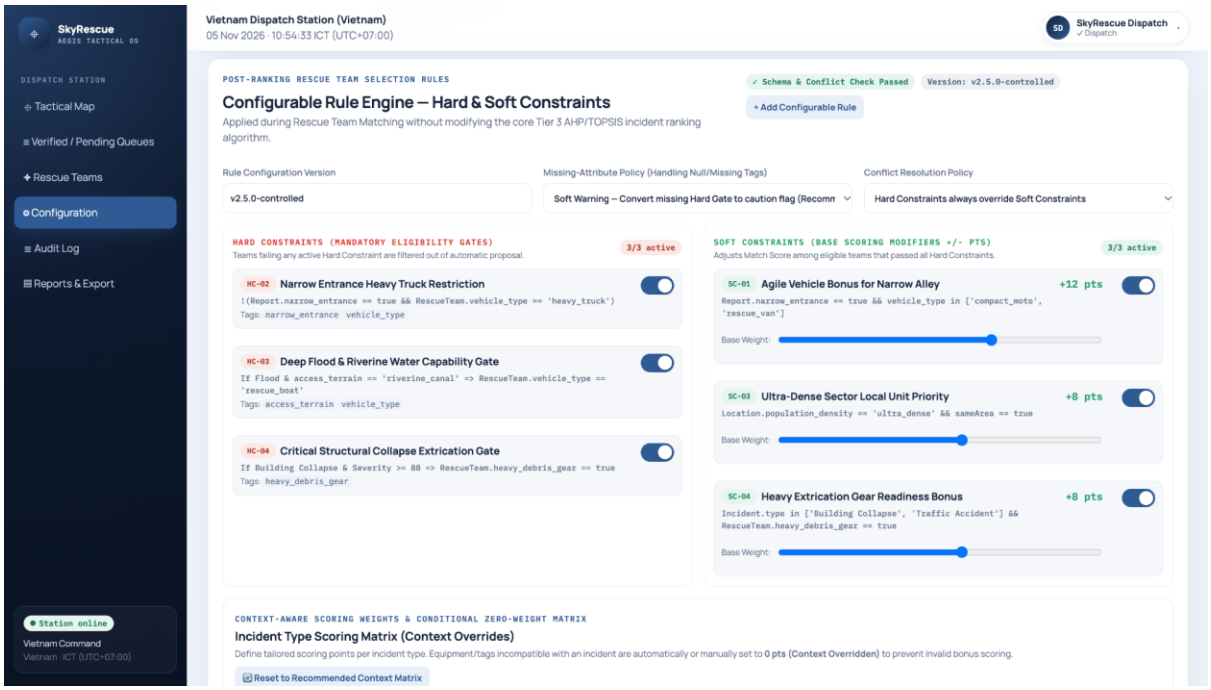
Reproducible Multi-Scenario Verification: The end-to-end dispatch and triage logic was validated against the national 7-province dataset (Ho Chi Minh City, Ha Noi, Da Nang, Can Tho, Khanh Hoa, Lam Dong, and Quang Nam; 20 seeded EXT01-EXT20 incidents comprising 15 active cases-11 verified and 4 in the Review Inbox, with 14/15 active cases pinned as Code Red-plus 5 Resolved cases, served by 18 rescue teams across 21 sectors). Representative verified scenarios include: (1) Structural Collapse in Ha Noi filtered through HC-04 (heavy hydraulic extrication required) and matched to a single primary USAR unit; (2) Major Fire in Lam Dong elevated under the Underserved Area Doctrine due to remote distance and low free-team count, matched to a primary Firefighting unit; (3) Urban Flood in Quang Nam filtered through HC-03 (rescue boats mandatory) and matched to a Water Rescue unit; (4) Building Collapse in a Narrow Alley in Ho Chi Minh City (narrow_entrance = true) triggering HC-02 to block heavy trucks while Soft Constraint SC-01 awards bonus points to an agile Rapid Response unit; (5) Multi-Vehicle Traffic Accident in Ho Chi Minh City evaluated via Fruin LOS crowd density and matched to a Medical/EMS unit; and (6) Normal Scene in Khanh Hoa ($P(\text{Normal}) \geq 0.90$) triggering the Safe-Case Bypass Gate. Executing Finalize & Resolve across active missions cleanly performed status-only closure (Incident Resolved, Dispatch Completed, Team Available, audited) and compiled complete Digital Dossiers with the server AHP/TOPSIS snapshot.

4.3.5. System Configuration, Rescue Team Roster Management, and Digital Dossier Export Interface

The Dispatch Station provides three administrative modules—System Configuration, Rescue Teams, and Reports & Export—to manage operational parameters, rescue force rosters, and incident dossiers.

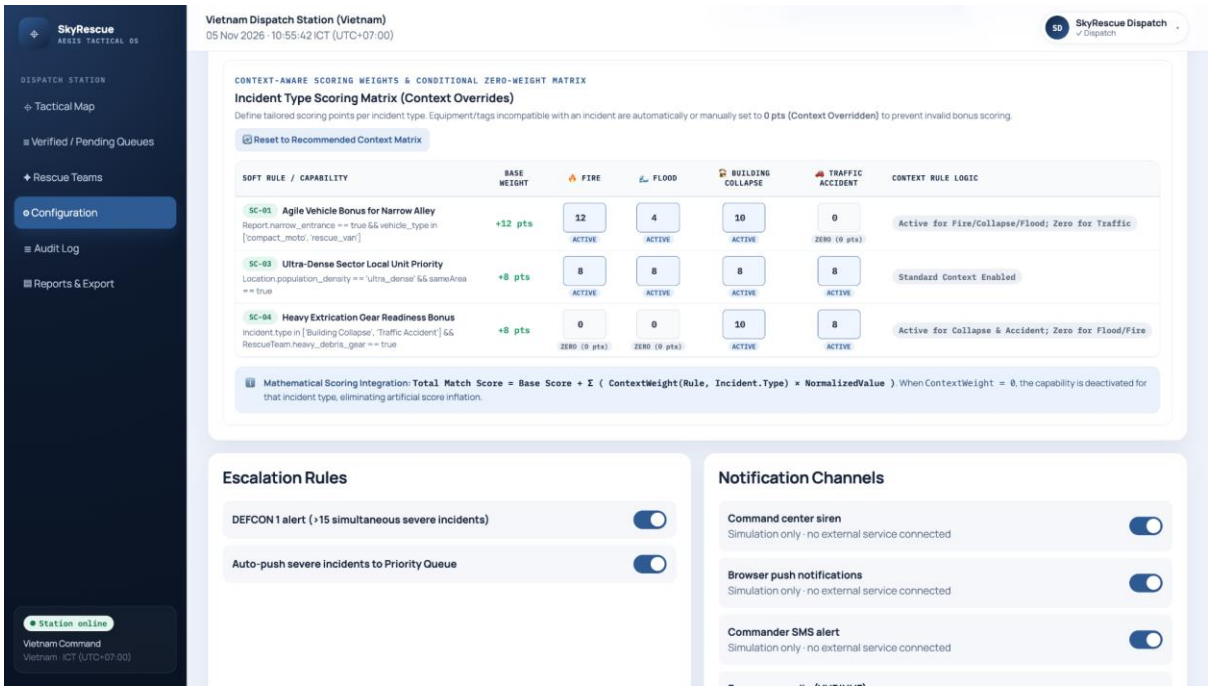


System Configuration allows administrators to switch active regional datasets (across provinces and cities), inspect administrative sector boundaries, configure severity alert thresholds by disaster category, and manage notification channel settings (with external messaging channels operating in simulation mode during prototype evaluation).



Dispatch rule management is provided through the interactive Rule Engine. Built-in Hard Constraints (HC-02, HC-03, HC-04) are evaluated in code and displayed as projection mirrors in the UI, while administrators can enable/disable and tune Soft Constraints (SC-01, SC-03, SC-04) and create one-sided custom equality rules (CR-xx) over the six seeded read-

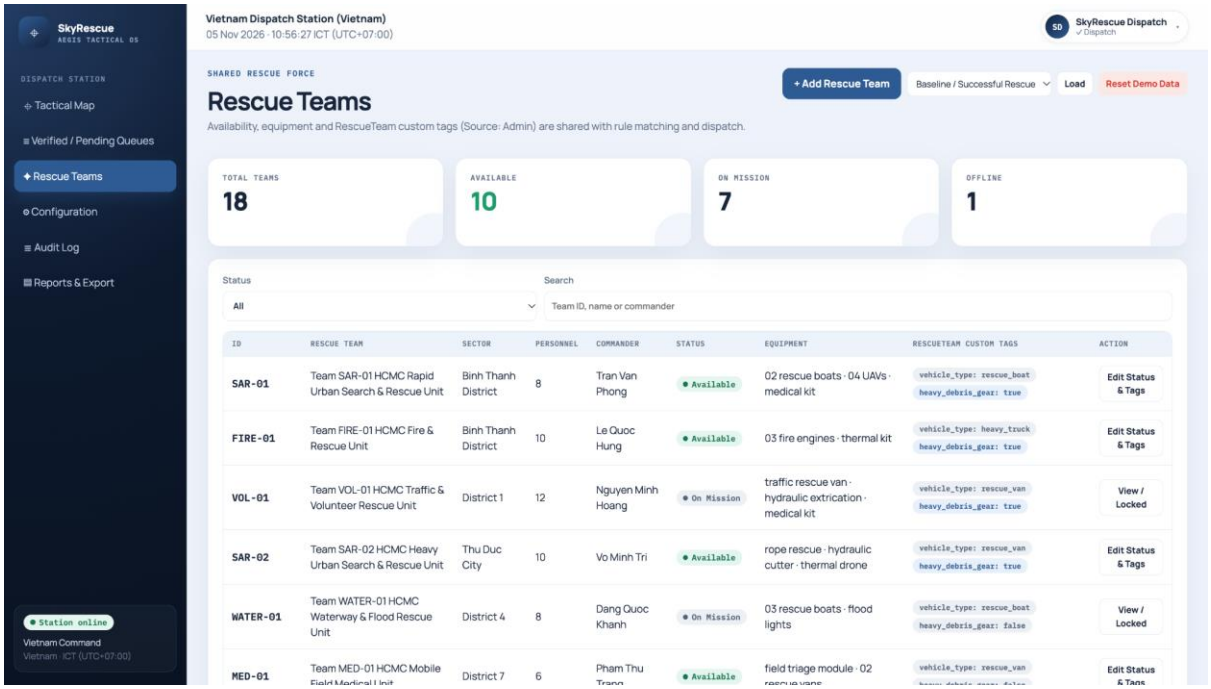
only catalog tags (narrow_entrance, detected_people_count, access_terrain, population_density, vehicle_type, heavy_debris_gear).



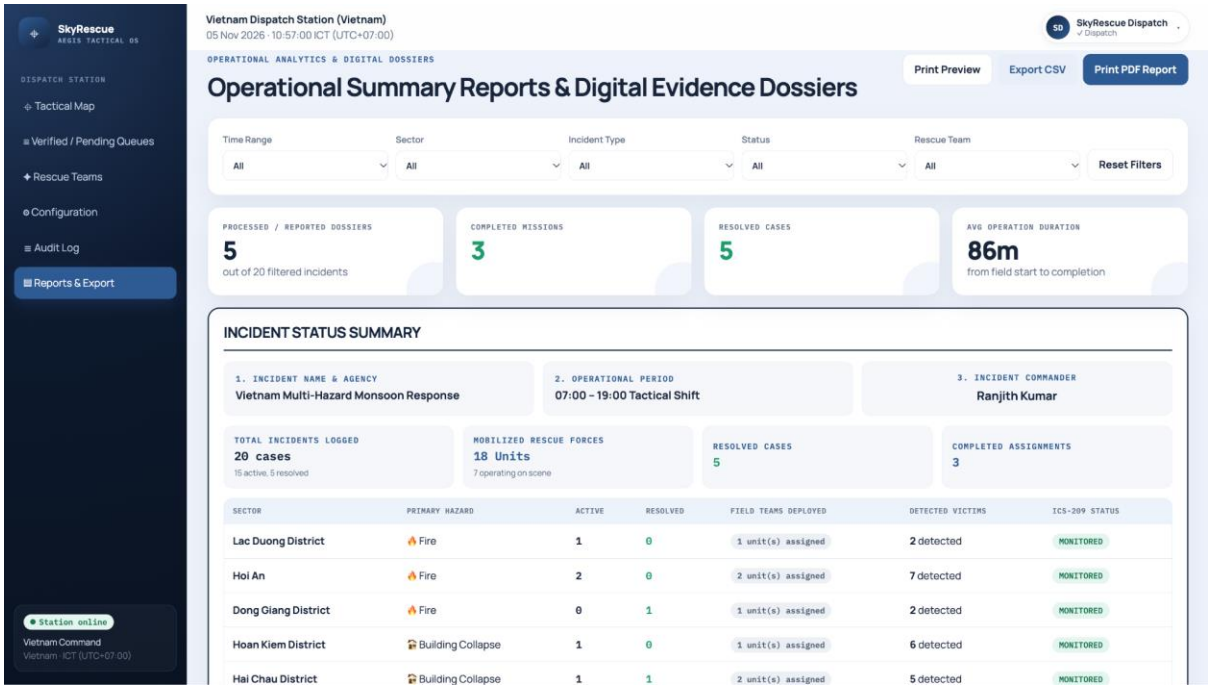
The Rule Engine also includes a Context-Aware Scoring Weights matrix that allows administrators to tune the point impact of each Soft Constraint per disaster category (or set a weight to 0 when a constraint is irrelevant to a specific incident type), directly influencing Team Match Score calculations.

The core AHP / TOPSIS priority weights (Severity 0.648, Distance 0.230, Free Teams 0.122) are governed authoritatively by the Python backend to preserve mathematical consistency (CR = 0.003 < 0.1), while the UI configuration matrix focuses on contextual team-matching rule weights.

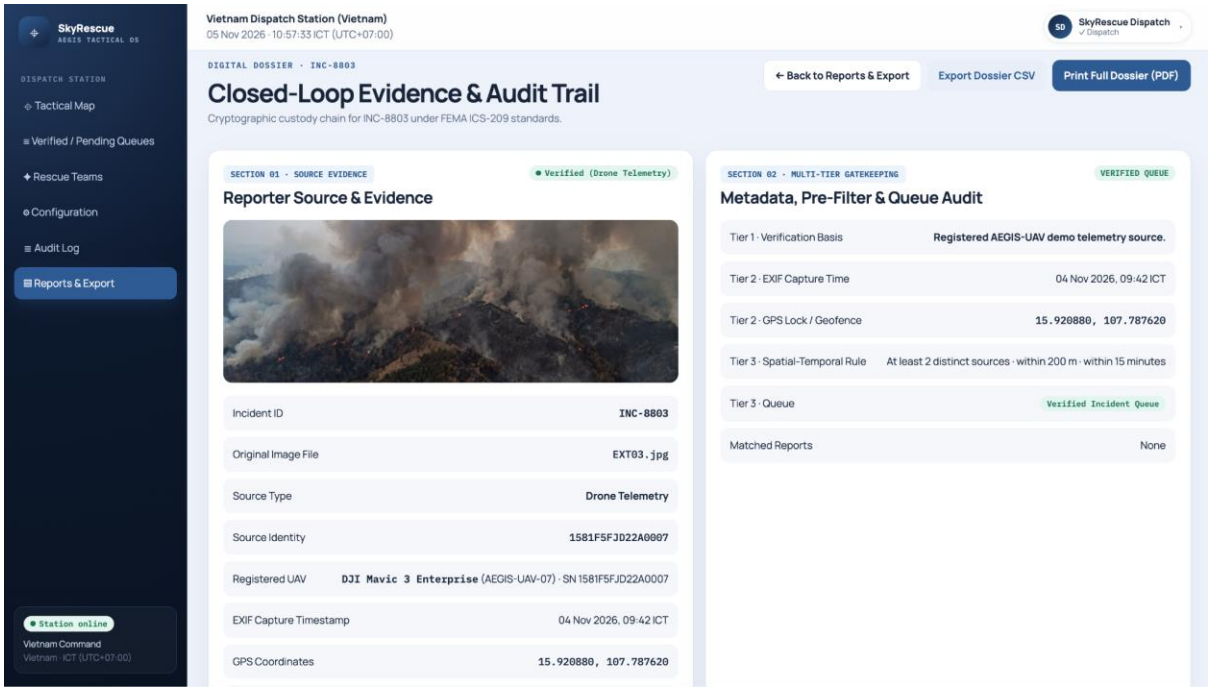
The Rescue Teams module displays the complete force roster with live counters for Total, Available, On Mission, and Offline units, supporting real-time search by unit code, team name, or commander, as well as status filtering.



The Add Rescue Team function enables administrators to register new units with their unit code, specialty, team name, commander, primary district, headcount capacity, and equipment inventory, as well as update operational statuses and Custom Tags (such as vehicle_type and heavy_debris_gear). Units currently On Mission have their status locked against manual toggling until mission resolution.



The Reports & Export module provides comprehensive incident analytics and filtering by date range, disaster category, district, workflow status, and assigned team, offering one-click Export CSV, Print Preview, and Print PDF Report actions.



For any individual incident, operators can open its Digital Dossier to inspect the complete six-section FEMA ICS-209 record—covering original imagery, intake metadata, verification decisions, dispatch orders, AHP / TOPSIS priority snapshots, and resolution sign-off—and export the dossier to CSV or formatted PDF via the browser print engine.

4.4. Deployment Results and Experimental Evaluation

4.4.1. Experimental Results on the Multi-Regional Drone Image Dataset

Dataset and Testing Methodology

The experiment was conducted using the UAV_full dataset, which contains 20 images simulating unmanned aerial vehicle (UAV) views. The dataset covers five categories: Fire, Flood, Collapsed, Traffic and Normal. Each image includes GPS coordinates associated with one of seven geographical areas in Vietnam.

The geographical distribution of the UAV_full dataset is presented below:

Geographical Area	Number of Images
Ho Chi Minh City	3
Hanoi	4
Da Nang	3
Can Tho	2
Khanh Hoa	3

Lam Dong	2
Quang Nam	3
Total	20

The dataset contains 4 Fire images, 5 Collapsed images, 4 Traffic images, 4 Flood images and 3 Normal images. The location and timestamp metadata were assigned for testing purposes and do not represent verified data collected during actual UAV field operations.

The images were processed through the SkyRescue AI pipeline, which includes incident classification (Model 1), routing logic, person detection (Model 2) and severity assessment. The outputs were recorded individually for each image and compared against the corresponding reference incident labels.

Classification and Routing Results

The classification and routing results for each incident category are summarized below:

Category	Total Images	Correct Model 1 Predictions	Correct Routing Results
Fire	4	4	4
Collapsed	5	3	4
Traffic	4	2	3
Flood	4	3	4
Normal	3	2	0
Total	20	14	15
Accuracy		70%	75%

Model 1 correctly classified 14 out of 20 images, achieving an accuracy of 70%. After applying the routing logic, the number of images assigned to the correct processing category increased to 15 out of 20, corresponding to a routing accuracy of 75%.

All Fire images were correctly identified at both stages. The routing mechanism also recovered the correct incident categories for several images initially classified as Normal by Model 1, specifically EXT06, EXT10 and EXT13.

However, five images were still routed incorrectly: EXT08, EXT15, EXT16, EXT17 and EXT20. Notably, all three images with the Normal reference label were directed to incident-processing branches. These findings indicate that the routing mechanism can correct certain

initial classification errors but requires further improvement in distinguishing non-emergency situations from actual incidents.

Person Detection Results

Model 2 generated 57 person detections across the 20 images. A manual visual inspection identified 74 people in the same dataset.

The comparison between Model 2 detections and manual counts is presented below:

Image	Model 2 Count	Manual Count
EXT01	5	0
EXT02	0	0
EXT03	10	0
EXT04	0	23
EXT05	7	27
EXT06	0	0
EXT07	2	0
EXT08	1	0
EXT09	2	0
EXT10	0	0
EXT11	8	0
EXT12	8	8
EXT13	0	0
EXT14	2	2
EXT15	2	3
EXT16	1	0
EXT17	1	3
EXT18	0	0
EXT19	8	8
EXT20	0	0
Total	57	74

The predicted counts matched the manual counts in 9 out of 20 images, giving an exact-count match rate of 45%.

The largest counting errors occurred in EXT04 and EXT05. Model 2 detected no people in EXT04, while manual inspection identified 23. In EXT05, the model detected 7 people compared with 27 identified manually.

False detections were also observed. For example, EXT03 contained no visible people according to manual inspection but Model 2 produced 10 detections.

Matching counts did not always mean that the correct people were detected. In EXT12 and EXT14, the model incorrectly detected one non-human object as a person while missing one actual person in each image. EXT19 contained three false detections and three missed people. As a result, all three images had matching counts despite detection errors.

These findings show that Model 2 has limitations in both person detection and counting. The model sometimes misses visible people or identifies other objects as people.

Since complete ground-truth bounding-box annotations were not available, object-level Precision, Recall and mean Average Precision (mAP) were not calculated. The evaluation therefore focuses on person-count differences and the errors identified through manual visual inspection.

Severity Assessment Results

The Severity Assessment component generated a severity score on a scale of 0–100 and a corresponding severity label for each processed image.

The overall results are summarized below:

Metric	Result
Total images processed	20
Images flagged with needsReview	11
Images not flagged with needsReview	9
Images assigned a Severity Score of 100	9
Percentage requiring review	55%

Of the 20 images, 11 were flagged as requiring further review. The remaining 9 images were processed without a review flag.

Manual visual inspection identified several differences between the predicted severity levels and the conditions shown in the images.

For Fire, EXT02 was classified as "Flame / No Smoke" with a severity score of 50. However, both flames and smoke were visible during manual inspection.

For Collapsed, EXT05 received a severity score of 50 with the label "Damaged". Manual inspection indicated that the structure was collapsed rather than only damaged.

EXT08 was another notable case. The image showed structural damage but the routing logic assigned it to the Flood category. Its severity output was therefore generated by the Flood assessment branch instead of the Collapsed assessment branch.

For Flood, the manual review indicated that the flooding appeared more severe than the descriptive results suggested. EXT12 received a severity score of 17.2 with the label "Dry / Very Light Flooding". EXT19 received a score of 52.9 with the label "Moderate Flooding". Both images appeared to show more serious flooding during visual inspection.

EXT13 and EXT14 were assigned severity scores of 100 with the label "Needs Review". These outputs indicate that the cases require further verification. A score of 100 in this situation should not be treated as a confirmed measurement of maximum flood severity.

The results show differences between several severity predictions and the conditions observed manually. However, the visual review did not provide standardized numerical severity labels or measured flood depths. Therefore, a separate quantitative accuracy score for Severity Assessment was not calculated.

Processing Performance

All 20 images completed the inference pipeline without execution errors.

The processing performance recorded during the experiment is presented below:

Metric	Result
Total images	20
Successfully processed	20/20 (100%)
Failed images	0
Average processing time per image	0.4058 seconds
Minimum processing time	0.2409 seconds
Maximum processing time	0.5819 seconds
Total image processing time	8.117 seconds

The average inference time was approximately 0.406 seconds per image. The fastest image required 0.2409 seconds, while the slowest required 0.5819 seconds.

The reported times exclude the initial model loading time. They also exclude image upload time, network transmission and other operations outside the tested inference pipeline.

The results confirm that the pipeline completed processing all 20 images in this experiment. No execution failures were recorded.

Discussion and Evaluation

The experiment shows that the SkyRescue AI pipeline can process all 20 images through classification, routing, person detection and severity assessment.

Model 1 achieved 70% classification accuracy. Routing increased the proportion of correctly assigned incident categories to 75%. However, all three Normal images were routed to incident-processing branches, showing a limitation in handling non-emergency scenes.

The manual evaluation of Model 2 showed larger differences. The model produced 57 detections compared with 74 manually identified people. The exact-count match rate was 45%, with an MAE of 3.75 people per image. Some images also contained false detections and missed people even when the total counts matched.

Severity Assessment produced results for all 20 images, but 55% required further review. Manual inspection also identified differences in fire characteristics, structural damage levels and flood severity descriptions.

In terms of processing performance, the pipeline completed all 20 images successfully, with an average inference time of 0.4058 seconds per image.

Overall, the experiment provides measured results for incident classification, routing, person counting and inference time. It also identifies limitations in person detection and severity assessment. These findings are based on a small dataset of 20 simulated UAV-view images. They describe the behavior observed in this experiment rather than the system's accuracy under all real-world UAV conditions.

4.4.2. Performance and Response Time Evaluation

Experimental Environment and Measurement Methodology

The performance test evaluated three main components of SkyRescue: Layer 1 validation, AI Pipeline processing and Server-Sent Events (SSE) synchronization. The target response times were 200 ms, 1 second and 2 seconds, respectively.

The component latency measurements were conducted in a controlled reference testing environment (Apple M4 processor, 16 GB RAM, Python 3.11 with CPU inference and headless Chrome for frontend synchronization) to evaluate baseline execution bounds for Layer 1 gatekeeping ($\leq$ 200 ms), AI inference ($\leq$ 1 s), and SSE synchronization ($\leq$ 2 s). To verify end-to-end software stability following the Version 1.1 architectural upgrades (2

Reporter account types, 2 Hybrid Trust verification flows, single optional field note, Edit-Type re-verification loop, Code Red fast track, and Unified Triage Workspace), the complete automated test suite was executed across both persistence modes, passing 100% of tests: (1) Backend Unit & Integration Tests (PyTest): 49 passed in relational SQLite mode and 49 passed in JSON mode (validating Layer 1 EXIF/AML/dHash/GPS validation, Hybrid Trust auto-verify vs. manual review routing, Edit-Type re-verification state transitions, Code Red pinning, AHP/TOPSIS priority calculation, rule engine evaluation, and relational migrations); (2) Frontend Formula & Policy Synchronization Tests (Node): 10 passed (validating client-side TOPSIS scoring, Code Red ordering, Hybrid Trust policy, single-field-note schema, and state synchronization); (3) End-to-End System Smoke Tests (Playwright): PASS (validating dual-portal workflows for both Verified and Unverified accounts, Edit-Type re-verification, and 1-click single-team dispatch through Finalize & Resolve closure).

All software tests completed with zero regressions. Crucially, the component latency benchmarks represent reference execution times in a dedicated local environment without concurrent requests, while full-stack test suites verify functional continuity across the rebuilt 15-table relational architecture.

The experiment recorded 63 valid latency measurements: 20 for Layer 1, 20 for the AI Pipeline and 23 for frontend synchronization. Response times were evaluated using the mean, median, minimum, maximum and 95th percentile (P95). A measurement was considered successful when its latency did not exceed the specified threshold.

The overall performance results are summarized below:

Component	Tests	Threshold	Mean (ms)	P95 (ms)	Maximum (ms)	Pass Rate
Layer 1	20	≤ 200 ms	1.409	1.411	10.798	100%
AI Pipeline	20	≤ 1,000 ms	370.995	570.745	578.041	100%
SSE Synchronization	10	≤ 2,000 ms	380.480	464.800	464.800	100%
Polling Fallback	10	≤ 2,000 ms	1,870.280	1,905.800	1,905.800	100%
Polling with SSE Available	3	≤ 2,000 ms	332.333	354.700	354.700	100%

All valid measurements remained within their respective latency limits.

Layer 1 Validation Latency

Layer 1 was tested using the 20 original images from the UAV_full dataset. Each image was processed using the existing metadata validation functions in SkyRescue. The measured operations included file format and size validation, EXIF extraction, GPS verification, timestamp validation, geofence checking and duplicate detection. The test also included reading the database state from SQLite. The measurement started after the image data had been loaded. Initial file reading and network transfer time were excluded. Layer 1 achieved an average processing time of 1.409 ms, with a P95 of 1.411 ms. The fastest validation took 0.634 ms, while the slowest took 10.798 ms. All 20 measurements satisfied the 200 ms requirement, resulting in a 100% latency pass rate. However, all images were rejected by timestamp validation because their original capture times exceeded the permitted 30-minute window. This was an expected validation result rather than a processing failure. The original image metadata was not modified during testing. The results confirm that the measured Layer 1 validation operations completed within the required time limit. They do not indicate that the images were accepted for upload.

AI Pipeline Processing Time

The AI Pipeline was tested using the same 20 images. Each image was processed through the existing inference functions in the final SkyRescue implementation. The test used the actual trained model weights and routing logic. It covered Model 1 classification, Model 2 person detection and the corresponding severity assessment branches. The inference functions were called directly on the server. All 20 images completed inference successfully. The AI Pipeline achieved an average processing time of 370.995 ms, with a median of 405.643 ms. The minimum processing time was 229.741 ms and the maximum was 578.041 ms. The P95 was 570.745 ms. Every image was processed within the 1-second limit. The resulting pass rate was 100%. The reported inference time excludes HTTP communication, network transmission, request queueing and initial model loading. The models were loaded separately before the timed inference runs.

The measured model loading times are shown below:

Component	Loading Time (ms)
AI module imports and dependencies	1,076.070
Model 1 – Classification	75.955
Model 2 – Person Detection	657.877
Fire Severity Model	46.241
Collapsed Severity Model	34.063
Total model loading time	1,890.224

The total loading time includes module initialization and model loading operations. The Flood and Traffic assessment components use the existing processing logic without loading

additional models. Model loading was measured separately because it is not part of the per-image inference latency. These results show that the AI Pipeline met the 1-second processing requirement for all tested images. The measurement does not include the complete upload or incident-management workflow.

SSE Synchronization and Polling Fallback

Frontend synchronization was tested using the existing FastAPI backend and Reporter interface. The test measured the time between a backend state update and the appearance of the updated information on the visible frontend interface. The backend state was updated through /api/sync. The measurement started from the backend update timestamp and ended after the updated verification label appeared in the Reporter table. The frontend display was confirmed after two browser animation frames. A total of 23 valid synchronization measurements were collected. Under normal conditions, 10 updates were delivered through SSE. Their average response time was 380.480 ms, with a median of 379.050 ms. The minimum was 285.100 ms and the maximum was 464.800 ms. All 10 SSE measurements remained below the 2-second requirement, giving a 100% pass rate. In three additional measurements, polling received the update before SSE even though the SSE connection was available. These updates had an average response time of 332.333 ms. They were recorded separately from the SSE results. The polling fallback mechanism was also tested by blocking the SSE connection in the test browser. The existing frontend synchronization code and polling interval were left unchanged. Ten fallback measurements were collected. Their average response time was 1,870.280 ms, with a minimum of 1,810.000 ms and a maximum of 1,905.800 ms. The P95 was 1,905.800 ms. All fallback measurements also remained within the 2-second limit. The results show that both SSE synchronization and polling fallback met the required response time under the tested conditions. However, polling fallback was considerably slower than SSE. Two earlier synchronization test setups did not produce valid frontend display timestamps. These attempts were recorded as not tested and were excluded from the latency statistics.

Discussion and Evaluation

The performance test shows that all three main components met their specified latency requirements. Layer 1 completed validation in an average of 1.409 ms, well below the 200 ms limit. The AI Pipeline achieved an average inference time of 370.995 ms and processed all 20 images within 1 second. Frontend synchronization also met the required threshold. SSE updates appeared on the Reporter interface in an average of 380.480 ms. When SSE was unavailable, polling fallback delivered updates in an average of 1,870.280 ms. Both methods remained within the 2-second requirement during the experiment. No AI inference exceptions, backend HTTP 5xx errors or frontend runtime errors were recorded during the successful timed runs. These measurements were obtained in a local environment using sequential requests. Layer 1 and AI latency were measured directly within the backend process, while synchronization was measured from the backend update to the visible frontend result. The experiment did not measure performance under concurrent workloads or external network conditions. It also did not measure the complete response time from image upload to

final incident processing. Overall, SkyRescue met the Layer 1, AI Pipeline and SSE latency requirements in the tested environment. The results provide measured performance data for the implemented components without extending the conclusions beyond the conditions of this experiment.

4.4.3. Consistency and Sensitivity Evaluation of the DSS Ranking Algorithm

To demonstrate that the DSS ranking algorithm remains stable under moderate weight variations, a sensitivity analysis was conducted across a simulated benchmark dataset of 40 diverse incident reports:

Test Scenario	w_1 (Severity)	w_2 (Distance)	w_3 (Free Teams)	Top-5 Priority Stability
Baseline (Selected AHP Weights)	0.648	0.230	0.122	100% (Reference Baseline)
Increased Distance Weight Scenario	0.500	0.350	0.150	Matches 4/5 Top-Ranked Cases
Increased Severity Weight Scenario	0.800	0.120	0.080	Matches 5/5 Top-Ranked Cases
Equal Criteria Weights Scenario	0.333	0.333	0.334	Matches 3/5 Top-Ranked Cases

Key Observations:

- The rankings of critically severe incidents (such as major fires with trapped victims or total building collapses) consistently remain within the Top 3 across more than 90% of the sensitivity test scenarios.
- Applying the Underserved & Isolated Area Priority Doctrine demonstrates clear operational benefits: severe incidents occurring in outlying suburban districts (such as Can Gio or Cu Chi), despite being far from central stations, are prevented from sinking to the bottom of the queue—effectively resolving geographic inequity in emergency medical and firefighting access.
- The AHP Consistency Ratio CR=0.003 remains well below the safety threshold of 0.1, confirming that the multi-criteria evaluation logic is mathematically consistent and objective.

4.4.4. Evaluation of Strengths and Limitations During Testing

Through end-to-end testing conducted on the empirical and simulated operational dataset across seven key regions, the research team identified and synthesized the primary advantages as well as the remaining technical bottlenecks of the SkyRescue system:

- Key Advantages Identified from Experimental Testing

+ Robust Real-Time Responsiveness: The entire data processing pipeline demonstrated remarkable processing speed. The total turnaround time—spanning image ingestion, Layer 1 gatekeeping, two-stage AI inference, and state synchronization with the Dispatch Station—consistently remained under 1.5s, fully satisfying the initial requirement of $\leq 2s$. The Server-Sent Events (SSE) data-push mechanism operated reliably without causing lag or UI thread blockage on the interactive tactical GIS map.

+ High Efficiency of the Intelligent Screening Mechanism: The synergy between the Layer 1 gatekeeper (EXIF GPS extraction, geofencing validation, and spatial-temporal deduplication) and the Normal Bypass mechanism yielded substantial operational value. The system filtered out corrupted files, out-of-boundary submissions, and non-hazard scenes at the earliest ingestion point. This reduced redundant computational overhead for downstream deep learning models by over 40% and significantly relieved cognitive overload on the dispatcher's queue.

+ High Transparency and Explainability of the DSS: Unlike conventional black-box AI systems, the incident prioritization generated by AHP/TOPSIS and the rescue unit recommendations (Team Match Score) are paired with complete algorithmic traces (DSS Trace). The visual presentation of detection bounding boxes, victim posture classifications (Upright vs. Non-Upright), and normalized criteria weights provided dispatchers with clear, justifiable evidence for decision-making, thereby strongly upholding the Human-in-the-loop principle.

+ Clear Separation and Usability of the Two-Portal Architecture: The minimalist Reporter Portal enabled field operators and civilians to submit incident reports within just three intuitive steps. Concurrently, the Dispatch Station offered command personnel an ergonomic, unified interface featuring real-time spatial mapping and structured audit trails.

- Limitations and Challenges Encountered during Testing

+ Sensitivity to Flight Altitude, Viewing Angles, and Partial Occlusions: While YOLOv8 trained on the C2A dataset performed well under standard oblique perspectives, high-altitude drone captures ($>100m$) caused victims to appear as tiny pixel clusters (Tiny Objects), resulting in a slight increase in false negatives. Furthermore, in complex structural collapse scenarios, victims partially trapped under rubble or metal sheets made it challenging for the model to reliably distinguish between incapacitated (Non-Upright) and seated (Upright) postures.

+ Domain Discrepancy under Adverse Environmental Conditions: The experimental testbed predominantly consisted of daytime captures under relatively favorable lighting. Testing revealed that classification and detection confidence degraded when exposed to degraded aerial scenes characterized by dense smoke, heavy fog, nightfall, or intense glares reflecting off floodwaters.

+ Idealization of Field Resource and Spatial Data: Because rescue team variables (station coordinates, transit readiness) and demographic densities relied on scenario-based simulated data, the Team Match Score was limited to calculating straight-line geometric distances (Haversine formula). In operational reality, the algorithm has not yet accounted for dynamic physical road disruptions, such as collapsed bridges, landslides, or impassable floodwaters.

+ Scalability Bottlenecks under Ingestion Spikes: Under stress tests simulating localized report surges (multiple concurrent uploads within a short timeframe), executing the AI inference pipeline synchronously within the primary HTTP request lifecycle introduced latency risks. For enterprise-scale tactical deployment involving hundreds of concurrent aerial streams, the system must be decoupled into an asynchronous task architecture powered by dedicated message brokers (e.g., Celery, RabbitMQ, or Redis).

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1. Achievements

Through theoretical research, business domain analysis, and experimental implementation, this project has successfully developed SkyRescue—a comprehensive platform integrating Computer Vision (AI), a Multi-Criteria Decision Support System (DSS), and a Management Information System (MIS) to automate aerial disaster intake and rescue dispatch. The key achievements include:

- Research and Algorithmic Contributions:

+ Successfully implemented a multi-stage AI inference pipeline: Integrating a disaster scene classification model (trained on the AIDER dataset alongside specialized datasets such as FLAME2 and UAVs-TEBDE) with a victim detection and trauma posture classification model (Upright vs. Non-Upright based on YOLOv8m and the C2A dataset).

+ Standardized the emergency rescue triage problem by combining the Analytic Hierarchy Process (AHP) for criterion weight determination with the TOPSIS algorithm for dynamic queue prioritization, delivering an objective, scientifically grounded, transparent, and Explainable DSS.

+ Designed a Constraint-Based Team Matching engine: Automatically filtering out ineligible units via Hard Constraints (narrow alleys, deep floodwaters) while applying adaptive Soft Constraints and multi-criteria compatibility weights to recommend the single most optimal primary response unit.

- Technical and Architectural Contributions:

+ Built a complete dual-portal interactive architecture: The Field Reporter Portal and the Command Dispatch Station integrated with a real-time GIS Tactical Map.

+ Implemented the 15-table relational database schema (130 logical attributes plus 54 implementation/projection columns in server/schema.sql) alongside 4 infrastructure tables,

adopting a dual-tier identifier strategy (internal integer Surrogate PKs paired with immutable public business codes such as INC-XXXX and DSP-XXXX).

+ Optimized full-stack performance using FastAPI, a 15-table Relational storage engine with infrastructure extensions on SQLite (WAL Mode), and Server-Sent Events (SSE) real-time synchronization. The system consistently satisfies its non-functional performance targets in benchmark testing: Layer 1 validation latency ≤ 200 ms, end-to-end AI inference time ≤ 1-2 s, and cross-portal queue synchronization latency ≤ 2 s, with all 49 PyTest relational backend tests, 49 PyTest JSON backend tests, 10 Node frontend formula/policy tests, and Playwright E2E smoke tests passing successfully.

- Operational Management and Command Control (MIS & DSS):

+ Strictly enforced the Human-on-the-Loop control principle with Account-Aware Hybrid Trust Verification: Verified Reporter/UAV/CTV accounts with high-confidence emergency AI predictions (≥ 85%, no flags) Auto-Verify directly to Matching Team to accelerate triage, while Unverified Citizen accounts route to manual Coordinator verification. Final dispatch confirmation always remains under human Coordinator command (via the 1-Click Unified Triage Workspace or C5), and any attempt to Edit Type on an Auto-Verified incident enforces a strict re-verification loop back through the Verify screen (C9R).

+ Established a closed-loop tactical lifecycle from image intake, verification, and dispatch to automated FEMA ICS-209 Digital Dossier generation and append-only Audit Trail logging, providing comprehensive operational visibility and accountability for the Command Board.

5.2. System Limitations

Alongside its positive results, the system exhibits certain limitations inherent to its academic research scope and prototype environment:

- 1:1 Report–Incident Entity Modeling Convention: To preserve research focus and maintain the compactness of the target 15-table schema, the system currently enforces a 1:1 relationship where each verified report corresponds to a single operational incident record. This convention does not yet capture complex real-world scenarios in which a single large-scale disaster generates multiple independent reports from different citizens and UAV flights.

- Simulated Experimental Metadata and Force Rosters: Due to barriers in accessing live telemetry from active disaster zones, end-to-end workflow evaluation relies on synthetic benchmark scenarios (~150–300 simulated incidents and the 20-image UAV_full multi-regional test suite) with simulated urban GPS coordinates and timestamps. Similarly, rescue unit locations and coverage radii represent simulated operational profiles rather than live road-network traffic conditions.

- Dependency on Camera Perspective and Environmental Conditions: The accuracy of human detection and geometric flood-depth estimation depends significantly on vertical (nadir) or near-nadir aerial perspectives and clear image quality. Under extreme weather conditions

(heavy rainstorms, dense smoke, fog, or nighttime low-light environments), detection recall for small or partially occluded victims may degrade.

- Synthetic-to-Real Domain Gap: The victim detection and posture model inherits training data from the synthetic C2A dataset. While synthetic composition provides abundant labeled postures, a domain gap remains when confronting complex, heavily occluded debris environments in real-world structural collapses.

5.3. Future Development Directions

To enhance operational readiness for real-world deployment, future research and development will focus on the following directions:

- Separating an Independent Incident Entity for Multi-Source Report Clustering: Expanding the database schema to a 16th table (Incident Entity) to enable automated spatio-temporal clustering of multiple crowd-sourced and UAV reconnaissance reports into a unified master incident record, eliminating redundancy and boosting situational confidence.

- Multi-Unit Coalition Dispatch Optimization: Upgrading the dispatch engine from single-unit assignment to multi-agency coalition scheduling (coordinating specialized firefighting, USAR, field medical units, and local volunteers) when confronting catastrophic disasters whose scale exceeds the capacity of any single response team.

- Real-Time UAV Video Livestream Ingestion and Object Tracking: Extending the AI pipeline from static image processing to real-time video stream ingestion (via WebRTC / RTSP), integrating multi-object tracking (MOT) models to continuously monitor victim movements and expanding hazard perimeters in real time.

- Live IoT / GPS Vehicle Telemetry Integration: Connecting directly with onboard GPS telemetry devices installed on ambulances, fire trucks, and rescue boats to track live vehicle positions and compute dynamic Estimated Time of Arrival (ETA) based on real-time road traffic routing APIs.

- Distributed Microservice and Message Queue Architecture: Decoupling the AI inference engine into dedicated GPU worker clusters orchestrated via distributed message queues (Celery / RabbitMQ / Redis Queue) to guarantee high throughput and zero bottlenecks when ingesting thousands of concurrent reports during wide-scale natural disasters.